A TEXTBOOK OF

DATABASE MANAGEMENT SYSTEMS

Concepts | Design | Implementation

LURNEXA PUBLICATIONS

DATABASE MANAGEMENT SYSTEMS: CONCEPTS, DESIGN AND IMPLEMENTATION

- Dr. H Balaji

Ms. J Saritha

Ms. B Pallavi

LURNEXA PUBLICATIONS

Title: *DATABASE MANAGEMENT SYSTEMS: CONCEPTS, DESIGN AND IMPLEMENTATION*

Authors: Dr. H Balaji, J Saritha, B Pallavi

Edition: First Edition, 2026

Published by:
Lurnexa Publications Private Limited
(An ISO 9001:2015 Certified Organization)
130–187, Ramulavari Gudi Centre,
Gorantla, Guntur – 522034, Andhra Pradesh, India

Email: lurnexapublication@gmail.com

Website: www.lurnexa.in

ISBN: 978-81-903315-4-8

Cataloguing-in-Publication Data: Available on request

Printed and bound in India

ENDORSEMENT

I am pleased to note that this textbook on **Database Management Systems** authored by **Prof. Dr. H. Balaji**, **J. Saritha**, and **Pallavi B**. has been developed with a clear academic vision and student-friendly approach. The book effectively covers important DBMS concepts such as database architecture, ER modeling, relational model, SQL, normalization, transaction processing, concurrency control, and recovery mechanisms with clarity and academic depth.

The inclusion of examples, and illustrations, and practical and explanations enhances learning and supports both classroom teaching and self-study. The book maintains a good balance between theoretical foundations and practical applications, aligning well with current university curricula and helping students build strong analytical and technical skills.

I appreciate the dedicated efforts of the authors in bringing out this valuable academic contribution. I believe this textbook will serve as a useful resource for undergraduate students, faculty members, and learners in Computer Science, and Information Technology, Artificial Intelligence, Data Science, and related fields. I extend my best wishes for the success of this publication and its contribution to academic and technical education.

Dr. A. Govardhan
Senior Professor, Department of Computer Science & Engineering,
Jawaharlal Nehru Technological University Hyderabad (JNTUH),
Vice Chancellor,
Rajiv Gandhi University of Knowledge Technologies (RGUKT Basar).

FOREWORD

It gives me immense pleasure to present this textbook on **Database Management Systems**, prepared with a sincere academic vision to support students and learners in understanding the fundamentals of database technology. DBMS is an essential subject in Computer Science and Information Technology, playing a vital role in areas such as software development, analytics, cloud computing, artificial intelligence, and enterprise applications.

The authors have presented the concepts of DBMS in a systematic, student-friendly, and application-oriented manner. The book covers important topics including database architecture, ER modelling, relational model, normalization, SQL, transaction processing, concurrency control, recovery mechanisms, and advanced database concepts with suitable examples and illustrations.

A notable strength of this book is its balance between theoretical understanding and practical relevance. The clear presentation style helps students strengthen conceptual clarity while also preparing for university examinations and industry requirements.

I congratulate the authors and publishing team for their dedicated efforts in bringing out this valuable academic work. I am confident that this textbook will serve as a useful learning resource for students, faculty members, and academic institutions. I wish the authors continued success in their future academic and research endeavours.

Dr. K. F. Bharati
Professor & Head
Department of Computer Science and Engineering
Jawaharlal Nehru Technological University Anantapur (JNTUA)
Anantapur, Andhra Pradesh

PREFACE

The rapid growth of information technology has made data management an essential part of modern computing. Databases are widely used in education, banking, healthcare, e-commerce, industries, and research, making Database Management Systems (DBMS) an important subject in Computer Science and related fields.

This book is written to provide students with a clear, systematic, and practical understanding of database concepts. Special emphasis has been given to conceptual clarity, and structured presentation, and examination-oriented coverage while maintaining academic depth.

The book covers key topics such as database architecture, and ER modeling, and relational model, and SQL, normalization, transaction management, concurrency control, recovery techniques, and database design concepts with suitable examples and illustrations.

Efforts have also been made to align the content with undergraduate engineering curricula and industry requirements. The objective is to help students strengthen both theoretical knowledge and practical understanding of database systems.

I sincerely hope this book will serve as a valuable learning resource for students, faculty members, and self-learners. I express my gratitude to all faculty members, reviewers, well-wishers, and the publishing team for their support and encouragement in completing this book.

Prof. Dr. H. Balaji
Professor of CSE
Sreenidhi Institute of Science and Technology (SNIST)
Hyderabad

AUTHORS

Dr. Halavath Balaji

Dr. Halavath Balaji is a dedicated academician and Professor at Sreenidhi Institute of Science and Technology. He is recognized for his expertise in teaching, research guidance, and student mentorship. His commitment to academic excellence, continuous learning, and creating an intellectually stimulating environment has earned him respect among students and colleagues. Through his guidance and mentorship, he continues to inspire aspiring engineers and researchers.

Jogu Saritha

Jogu Saritha is an Assistant Professor at Sreenidhi Institute of Science and Technology and a Research Scholar at National Institute of Technology Jalandhar. She has strong expertise in teaching, mentoring, and research, with a commitment to academic excellence and continuous learning. Her dedication to quality education and student development has made her a respected educator. She dedicates her work to her parents and expresses sincere gratitude to her husband for his constant support and encouragement throughout her professional journey.

Pallavi B

Pallavi B is an experienced academician serving as an Assistant Professor at Sreenidhi Institute of Science and Technology and a Research Scholar at SR University. With 18 years of teaching experience, she has developed strong expertise in delivering quality education, mentoring students, and supporting their academic and career growth. She creates a positive learning environment that encourages curiosity, critical thinking, and continuous learning. Her dedication to both teaching and research reflects her strong commitment to education and professional excellence.

ACKNOWLEDGEMENTS

The completion of ***Database Management Systems: Concepts, Design and Implementation*** has been possible through the support and contributions of many individuals and institutions. We express our sincere gratitude to everyone who contributed directly and indirectly to this work.

We warmly acknowledge the dedicated efforts of **Dr. H. Balaji**, **J. Saritha**, and **Pallavi B**., whose academic insight, research support, and contributions greatly strengthened this book. We also express our heartfelt gratitude to Vice-Chancellor **Prof. A. Govardhan** of the International Institute of Information Technology for his valuable guidance and academic mentorship.

We are thankful to our supervisors and for their continuous encouragement and support throughout this journey. Our sincere appreciation also goes to the **Lurnexa Publications**, and especially **Narendra Kumar**, for their professional support for the during the publication process.

We also thank the peer reviewers, domain experts, and seminar participants whose valuable suggestions helped improve this manuscript. Finally, we dedicate this work to our families and parents for their constant love, patience, and encouragement.

We hope this textbook serves as a valuable resource for students, educators, researchers, and learners in the field of database systems.

— Dr.H Balaji, J Saritha, Pallavi B

CONTENTS

CHAPTER - 1

INTRODUCTION

Chapter Overview

This Chapter lays the conceptual and architectural foundation of database management systems. It begins by contrasting traditional file-based data management with modern database approaches, then systematically introduces the actors in a database environment, the variety of data models available, and the critical concepts of schema, instances, and data independence. The second half of the chapter is dedicated to the Entity–Relationship (ER) model the primary tool for conceptual database design covering entities, attributes, relationships, constraints, and advanced features such as specialization and generalization.

Learning Objectives	**By the end of this chapter, the reader will be able to:** 1. Define a DBMS and explain its role as an intermediary between users and physical data storage 2. Identify and differentiate the categories of database users and their interaction modes 3. Describe the advantages of database systems over traditional file systems 4. Distinguish between different data models including hierarchical, network, relational, and ER 5. Explain the three-schema

	architecture and the two levels of data independence 6. Design ER diagrams using entities, attributes, relationships, and cardinality constraints 7. Apply specialization and generalization to model superclass–subclass hierarchies

Section-by-Section Roadmap

Section	Topic	Key Concepts
1.1	**Introduction to Database Systems**	DBMS definition, role as intermediary, data management objectives
1.2	**Database vs File System**	Redundancy, inconsistency, integrity, isolation, security, access difficulty
1.3	**Database Users**	Naïve users, application programmers, sophisticated users, DBAs
1.4	**Advantages of Database Systems**	Centralized control, reduced redundancy, improved sharing and security
1.5	**Database Applications**	Banking, airlines, universities, manufacturing, HR, e-commerce
1.6	**Data Models**	Hierarchical, network, relational, ER, object-oriented, semi-structured, physical

Section	Topic	Key Concepts
1.7–1.8	**Schema & Instance**	Schema as structure definition; instance as current data state
1.9–1.10	**Data Independence**	Physical and logical independence; three-tier schema architecture
1.11–1.12	**Database System Structure**	Storage/query/transaction managers; centralized vs client-server architecture
1.13–1.16	**ER Model: Entities & Attributes**	Entity types, simple/composite/multi-valued/derived/key attributes, entity sets
1.17–1.19	**Relationships & Constraints**	Relationship sets, key and cardinality constraints, participation constraints
1.20	**Superclass and Subclass**	Inheritance, specialization, generalization, disjoint/overlapping constraints

1.1 Introduction to Database Systems

A database management system includes a collection of related data along with a collection of software referred to as the **Database Management System (DBMS)**. This DBMS enables the storage, searching, updating, and manipulation of data. Data have become crucial assets for every organization due to the computerized era. Organizations including schools, universities, banks, hospitals, government, and many others deal with massive amounts of data as part of their day-to-day operations. It would be highly cumbersome and practically impossible to handle these data either manually or through conventional means such as keeping files on papers.

In a DBMS, an entity named DBMS acts as an interface between the physical database and the users of the database or applications that use the database. Management of data is done through a series of processes including data abstraction, data independence, security, data integrity checks, concurrent control, and failure recovery among others.

Suppose there is a university scenario whereby it is required to manage the data related to students, subjects, lecturers, examination and results among other data. If different departments maintain their database separately, then it is very likely that data duplication will occur. **But this is not so with a database system.**

1.2 Characteristics (Database Vs File System)

Data redundancy involves duplicating data within several files or databases, and it is a common occurrence in file systems. Since

each individual application will have its own specific file where data will be stored, there will always be duplicating data to be stored. In a database management system of a university, basic student information like names, roll numbers, and addresses will occur in admission files, exam files, and dormitory files.

Data inconsistency is caused by data redundancy, which refers to the existence of different data values for the same item in different files. Data inconsistency will be generated when changes are made to one of the data item files without making corresponding changes to other copies. For example, if the address of a student is modified in the admission records but is not modified in the examination records, then the student will have two addresses. The database system solves this problem by storing only one version of the data.

Data integrity is concerned with ensuring that data is correct, accurate, and consistent. In the file system environment, integrity constraints such as range checks, uniqueness constraints, and mandatory constraints are maintained via the use of application programs. This may lead to redundancy in the enforcement of integrity constraints, as well as increase the possibility of mistakes, since some applications might not enforce certain constraints. For instance, whereas one program might allow data to exceed the maximum value, another program might not do so. On the other hand, integrity constraints are defined in the schema in a database environment.

Data isolation is experienced in file-based systems, since the data is scattered into several files, which makes data integration very difficult. Each application runs on its own and maintains its own files; therefore, the data cannot be shared between the applications. Data integration is required, for instance, in producing a report showing the student's performance along with his fee paying status, since this will involve manually integrating data from various files.

Another aspect that comes to mind is that of **data security**, which has something to do with file systems, since data protection is only applied to the file. After accessing the file, one can easily acquire the information stored in the file without worrying about its confidentiality. For instance, after an employee accesses the file of a particular student, he or she can acquire private financial and personal details that were supposed to be secured. Database systems, on the other hand, employ more sophisticated forms of data security.

Limited access to data is the biggest disadvantage in the use of the file system approach because of the rigid data access approach employed. The major problem is that in order to get a different form of data, it will be necessary to create new applications which will be both expensive and time consuming. For example, in order to get the information about the students who had scored more than 80 percent but had not paid their dues for tuition, a new application needs to be created.

1.3 Database Users

Database system is built for various types of users with different ways of interacting with the system. You need to be aware of different user types that are well known by their own database system and help in making their building & management easier. In contrast, we have the naive user or the end-user which is defined as a human who makes use of the system under the guidance of application programs that were prepared ahead of time.

The **end users** who utilize the database by filling a few simple forms or selection menus don't care about the inner workings of the system. ATM machine, ATM operator and ATM user, Student running Homepage online. Now, Application programmers then develop application programs to communicate with database systems and other parts of the organization

Application programmers are professionals who develop application programs using Java; Python, C++ and other languages. Application Programmers - Application programmers create application programs to implement business logic and leverage the power of DBMS for data manipulation. –As a programmer/technologist, they tend to use the database applications that give them some insight into query language in order to analyze the data within the database. Example : Data analyst, data engineers and data scientist.

The advanced users have direct access to the database by utilizing query languages such as SQL. This category consists of users like data analysts, engineers, and scientists who carry out intricate database querying activities. These users need a thorough knowledge of the database architecture and the processes involved in query processing.

Some **specific applications**, designed for particular purposes by specialized users, do not fall into the category of standard data processing. These include computer-aided design (CAD) systems, geographic information systems (GIS), artificial intelligence systems, and multimedia databases.

The **Database Administrator (DBA)** plays an important part in making a database system function effectively. Some of the functions performed by the Database Administrator include determining the database structure, choosing suitable storage structures, giving access privileges to users, and ensuring that the database performs well. In addition to these duties, the DBA also secures the database by performing backups and implementing recovery mechanisms.

1.4 Advantages of Database Systems

One of the major advantages that can be attributed to database management systems is the elimination of data redundancy and data inconsistencies. This is made possible by the centralized storage and sharing of data by various applications. In a case of an online business management system, data about the customers will be stored once and accessed for various purposes.

Another tickets simultaneously, while the DBMS makes sure no two users end up with the important advantage is data sharing with concurrency control. Multiple users can access and update the database at the same time without affecting each other. For example, in a railway reservation system, thousands of people can check seat availability and book same seat.

Another great benefit is improved data integrity. Database systems automatically enforce rules like primary keys, foreign keys, and domain constraints. For instance, the DBMS ensures a student can't be registered for a course unless that course actually exists in the course table, keeping the data accurate and reliable.

Enhanced data security is another important advantage of database systems. They offer authentication, authorization, and access control features that limit data access based on user roles. For example, in a hospital database, doctors can access patient medical records, billing staff can handle payment information, and patients can only view their own reports.

Additionally, database systems include backup and recovery mechanisms to protect data from loss caused by system failures, hardware crashes, or power outages. For instance, in a financial institution, transaction logs and regular backups help restore the database to a consistent state after any unexpected failures.

Finally, data independence is a key advantage of database systems. This means that changes to the data structure or how data is stored don't force changes in the application programs. For example, if you add a new attribute like an email ID to a customer table, existing applications won't be affected unless they specifically use that new attribute. This flexibility makes it easier to evolve the database without disrupting the software that relies on it.

1.5 Database Applications

Data base systems are now practically enforced as a core essential of many organizations today, so fast, that the organization can gather and maintain hundreds of gigabytes of data without breaking a sweat. Middle Bankers Uses Databases to Store Customer Information In their accounts. The most popularly database applications used in banking. Including specifics (loans, transactions)

Databases play a vital role in various sectors by efficiently managing large amounts of information.

In Education Systems, they handle student records, course registrations, exam results, attendance, and fee details. For instance, a university database lets

students enroll in courses online, allows faculty to upload grades, and helps administrators generate academic reports.

Airline and railway reservation systems also depend heavily on databases to manage schedules, seat availability, ticket bookings, and cancellations in real time. These systems require strong concurrency control and reliability to support many users at once.

In healthcare, databases store patient records, medical histories, diagnostic reports, prescriptions, and billing information. Electronic health record systems enable doctors to access patient history instantly while maintaining confidentiality and security.

E-commerce platforms use databases to manage product catalogs, customer profiles, orders, payments, and delivery tracking. When a customer places an order, the database updates inventory, generates invoices, and records transaction details seamlessly.

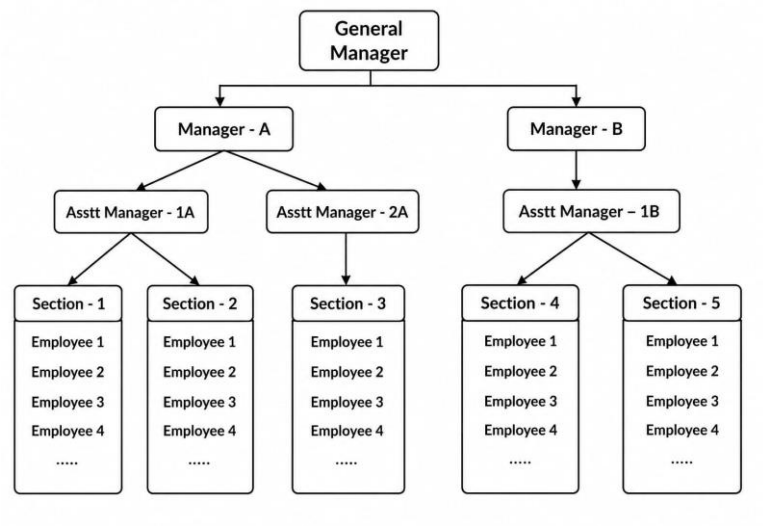
In **government and public administration**, databases are used for applications such as census data management, taxation systems, voter registration, and national identity databases. These applications require high data accuracy, security, and scalability.

Overall, database systems form the backbone of modern information systems by supporting efficient data storage, secure access, reliable processing, and decision-making across diverse application areas.

1.6 Data Models

A **data model** is 'a collection of conceptual tools for describing data, data relationships, data constraints and data manipulation'. Data models gives 'an interface between the real-world application domain and the database system'. Data models act as the abstraction that is used by designers to interface with the user requirements as a formal database structure. Based on the DBMS literature the most used classification of data models are **conceptual models** (high level), **logical models** (record-based) and physical data models.

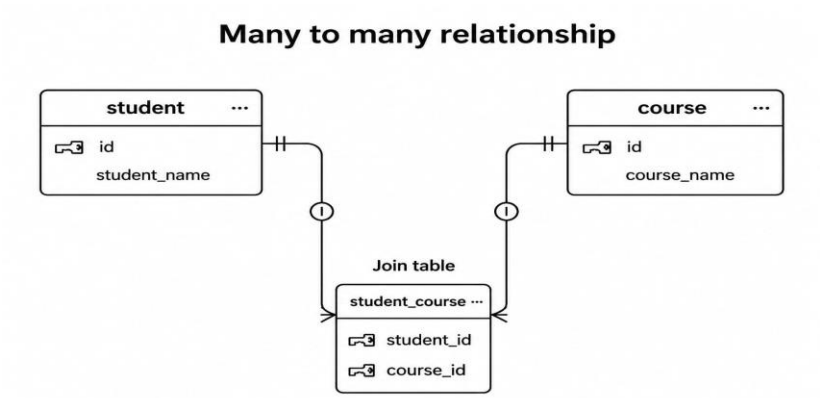
1. Hierarchical Data Model



The hierarchical model is a structure where data is represented using records in a hierarchical tree structure with each child record having a single parent, but each parent can have many children. Data relationships are one to many. Data is accessed by traversing through the tree beginning at the root node. This was employed in some early DBMSs including IBM IMS.

E. g. the root entity in organization database could be Company, under it there are Departments; and, under each department, there are Employees. Employee can be in only one Department. This will provide quick access to data along predetermined paths; Still, it can be cumbersome to change hierarchy dynamically, and it may not be optimal for many-to-many relationships.

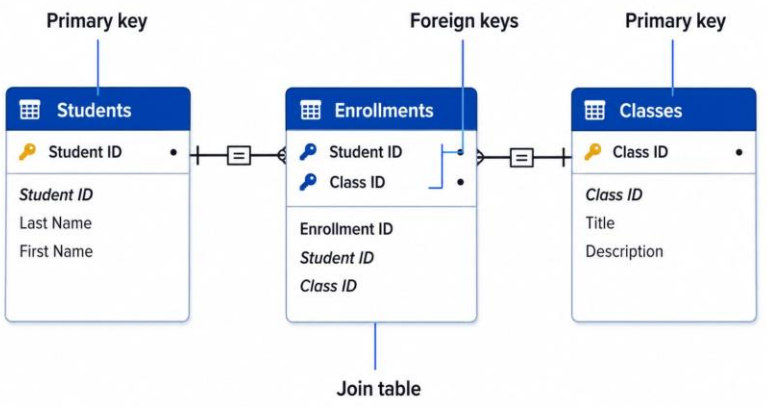
2. Network Data Model



Network data model -It is an extension to the hierarchical data model. Record type may have more than one owner i. e. its own record type. Data are modeled as a graph consisting of a nodes (records) and links (edges), which support many-to-many relationships.

For example, in a university database, a student can enroll in multiple courses, and each course can have many students. The network model handles this kind of many-to-many relationship efficiently. However, because it relies on pointer-based navigation, designing and using the database can be complex. Users need to have a clear understanding of the access paths to work with the data effectively.

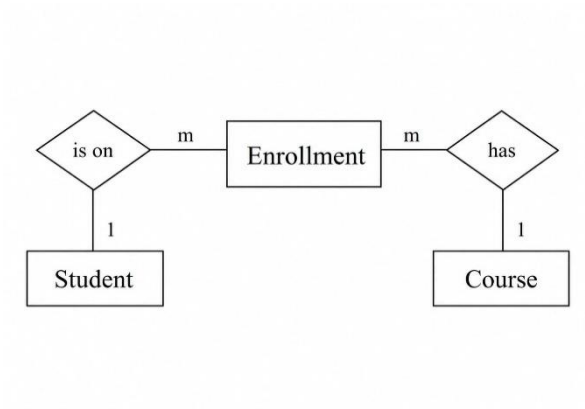
3. Relational Data Model



The relational data model organizes data into tables, also called relations, where each table is made up of rows (tuples) and columns (attributes). Relationships between tables are created using common attributes like primary keys and foreign keys. This model is grounded in strong mathematical principles and offers a high level of data independence.

For example, a Student table might store student details, while a Marks table holds subject-wise scores. The Roll Number attribute serves as a key that links these two tables. Thanks to its simplicity, flexibility, and support for SQL, the relational model has become the most widely used data model in modern database systems.

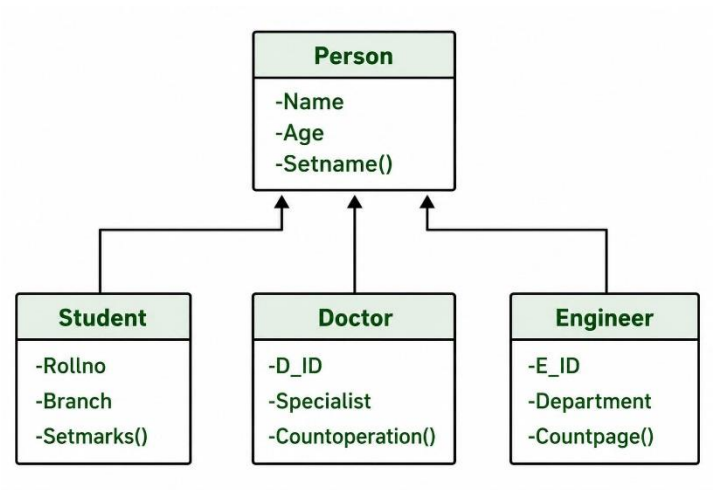
4. Entity–Relationship (ER) Data Model



The Entity–Relationship (ER) model is a high-level conceptual data model primarily used during the database design phase. It represents data through entities, attributes, and relationships. Entities represent real-world objects, attributes describe their properties, and relationships show how entities are connected.

For example, in a college database, Student and Course are entities, and Enrolment represents the relationship between them. ER diagrams offer a clear and intuitive visual overview of data requirements and act as a blueprint for translating the design into a relational database schema.

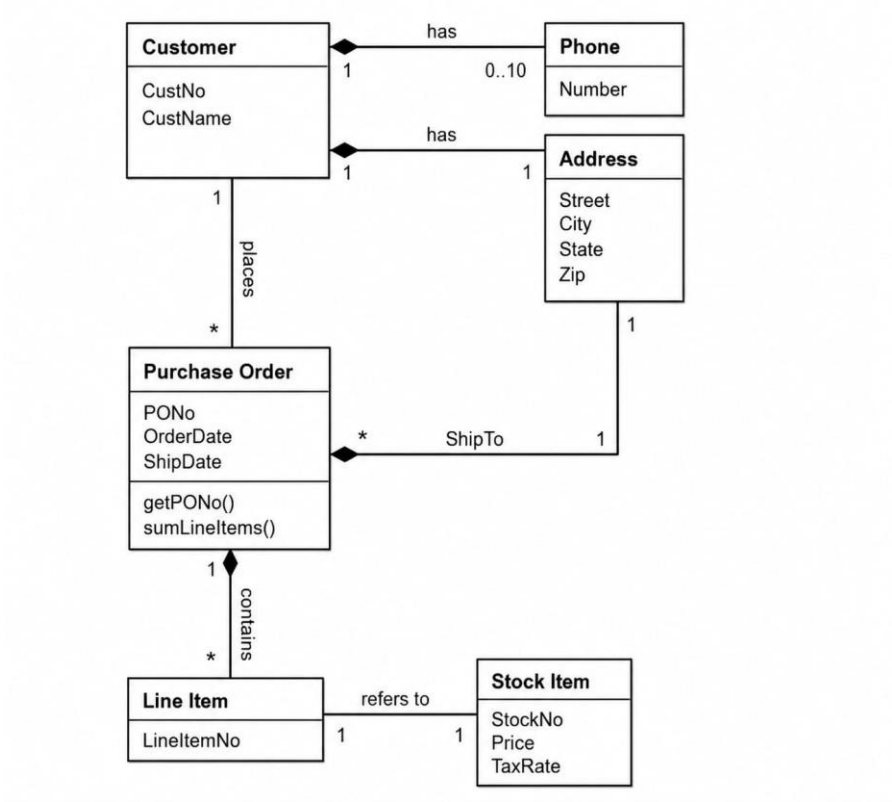
5. Object-Oriented Data Model



The object-oriented data model integrates database technology with object-oriented programming concepts like encapsulation, inheritance, and polymorphism. In this model, data and the operations that act on the data are bundled together as objects. Objects are instances of classes, and classes can inherit properties and behaviors from other classes.

For example, a Person class might include attributes such as name and address, while a Student class inherits these attributes and adds specific details like roll number and marks. This model is particularly well-suited for complex applications such as multimedia systems, CAD/CAM applications, and scientific databases.

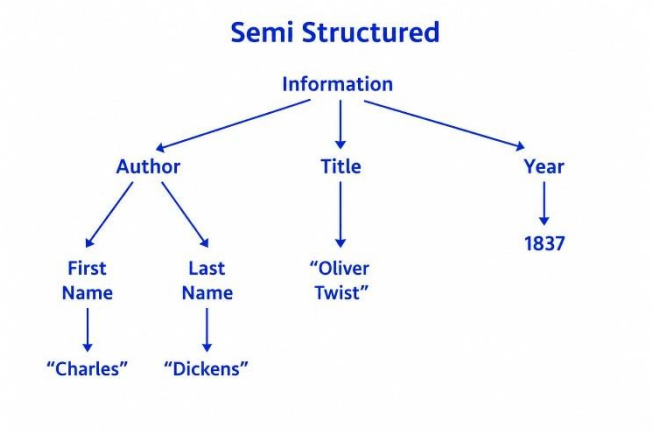
6. Object-Relational Data Model



The **object-relational data model** extends the traditional relational model by adding object-oriented features. It supports complex data types, inheritance, and user-defined types, while still maintaining the familiar table-based structure of relational databases.

For example, an Address can be defined as a complex data type that includes street, city, and pin code, and this type can be used as an attribute within a Customer table. This model effectively bridges the gap between conventional relational databases and object-oriented applications.

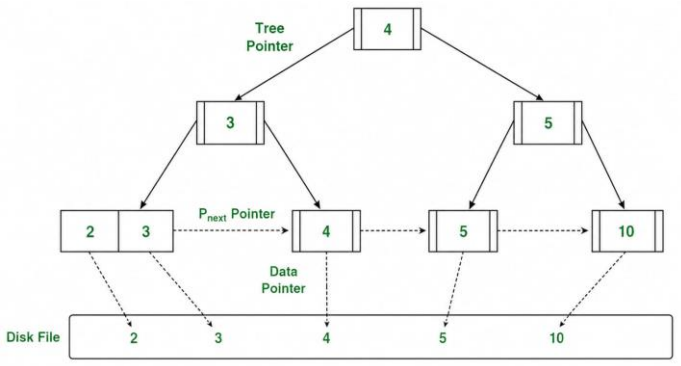
7. Semi-Structured Data Model



Semi-structured data model is suitable for flexible description of data that does not fit easily into one cohesive systematic static structure. There is an inherent structure in semi-structured data in form of its representation. Data in XML, JSON is semi-structured.

An example of profile information done through web application is less standardized data because people different (e. g. some people will send his/her cell-phone number, Facebook or Twitter link) ; semi-structured data model is very common to web services, data exchange and big data.

8. Physical Data Model



The physical data model brings a comprehensive specification of physical storage of the data; the specifics of storage on media (access paths, organization

of files, index structures and hashing algorithms) are defined at this level. The model is very much concerned with efficiency of storage.

For example: a database table may be index on the B+ tree to reduce the delay of retrieving database data. The physical data models are non-physical objects are physical object which is maintained by the DBA (Database Administrator).

1.7 Schema

A **schema** is the logical structure or blueprint of a database that defines how data is organized. It specifies the tables, attributes, relationships, constraints, and data types within the database. Unlike actual data, the schema itself doesn't change frequently and is established during the database design phase. It describes the kinds of data the database will hold, rather than the data values themselves.
For example, in a university database, a schema might define a STUDENT table with attributes like Roll.No, Name, Department, and Year. Even if there are no student records yet, the schema remains in place. Schemas are stored in the data dictionary and help the DBMS understand and manage the data effectively.

1.8 Instance

An instance is the actual content of the database at a specific moment in time. It represents the current collection of data values stored in the database. Unlike the schema, which remains mostly stable, instances change frequently as data is added, deleted, or updated.

For example, when new students are admitted or marks are updated, the database instance changes, but the schema stays the same. In short, the schema defines the structure of the database, while the instance reflects its current state.

1.9 Data Independence

Data independence is the ability to have the schema at one level modified without having to change the schema at a higher-level. By Using of DBMS data independence is one of the major advantage you got. It changes the least in the application and makes most flexible system. The two types of data independence are as follows:

1. Physical Data Independence
2. Logical Data Independence

1 . Physical Data Independence

Physical data independence is the ability to modify the internal (physical) schema of a database without impacting the conceptual or external schemas. These modifications typically involve changes to storage structures, file organization, indexing methods, or access paths.

For example, switching the storage of a table from a heap file to a B+ tree index to boost performance won't require any changes to application programs or user views. Users can continue querying the database in the same way, without needing to know how the data is physically stored behind the scenes.

2 . Logical Data Independence

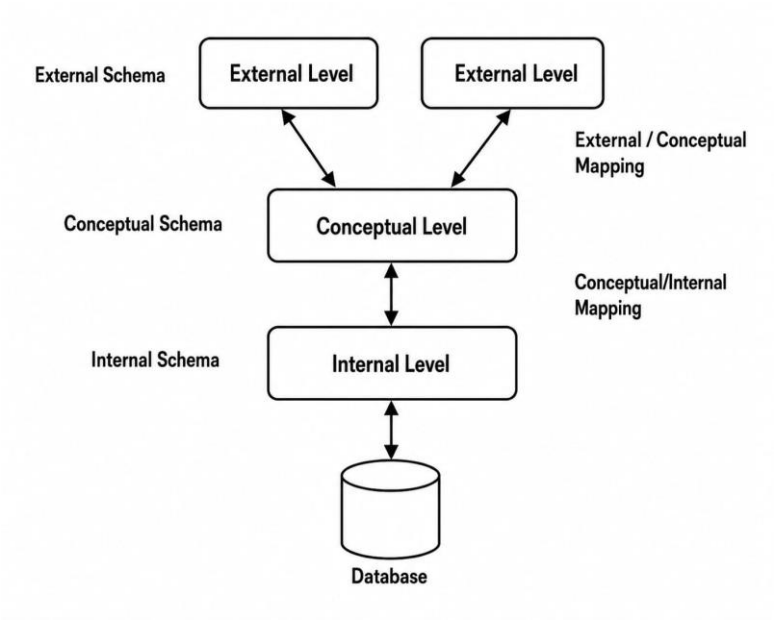
Logical data independence is the ability to change the conceptual schema without impacting the external schemas or application programs. This can involve actions like adding or removing attributes, splitting tables, or creating new relationships.

For example, adding an Email_ID attribute to the STUDENT table should not affect existing applications that don't use this attribute. Achieving logical data independence is generally more challenging than physical data independence.

1.10. Three-Tier Schema Architecture for Data Independence

The Three-Tier (Three-Schema) Architecture was proposed to achieve data independence by separating the database system into three distinct levels: External, Conceptual, and Internal schemas.

1. External Schema (View Level)



An external schema how a database appears to year individual users or applications. Users of the database may see something entirely different from it, and which depends upon all what is in need. External schema is used to conceal additional data, or for security wise.

For instance, a student can view their own credential and ongoing marks but an administrator might be able to see the data of all students, including fees.

2. Conceptual Schema (Logical Level)

The conceptual schema describes the complete logical view of the database. It defines all the entities attributes relationships and constraints without regard to how they are physically implemented. There is only one conceptual schema for the whole database.

It defines entities like STUDENT, COURSE and ENROLLMENT.

3. Internal Schema (Physical Level)

Internal schema: This is concerned with how the data is physically stored in the database. This consists of file structures, how records are formatted, indexing techniques and even-

placement on disk. The level is worried about performance and now the storage efficiency.

The method includes details like whether the data is contained through hashing, B+ tree indexing or sequential files.

4. Mapping Between Schema Levels

- **External–Conceptual Mapping** ensures that changes in the conceptual schema do not affect user views.
- **Conceptual–Internal Mapping** ensures that changes in physical storage do not affect the logical structure.

These mappings are the key reason data independence is achieved in DBMS.

In a **college database system**, the three-tier schema architecture includes the external, conceptual, and internal levels, which together provide data independence.

At the external level, different users have customized views of the database. For example, a student's view might display Roll_No, Name, and Marks, while a faculty view shows Roll_No, Subject, and Grade. Each user accesses only the data relevant to their role.
The conceptual level defines the complete logical structure of the database. For instance, it includes tables like STUDENT (Roll_No, Name, Dept, Email), COURSE (Course_ID, Course_Name), and ENROLLMENT (Roll_No, Course_ID, Grade). There is a single conceptual schema that applies to the entire database.

At the internal level, the physical storage details are specified, such as using a B+ tree index on Roll_No to store the STUDENT table.

If changes are made to the storage method, the external and conceptual schemas remain unchanged—this is known as physical data independence. Similarly, if a new attribute like Email is added to the STUDENT table, existing user views are not affected, demonstrating logical data independence.

1.11 Database System Structure

The architecture of a database system describes the structure of the Database Management System (DBMS), explaining how individual components work together to perform operations on the data in terms of efficiency, security,

and reliability. A typical database system is made up of the user, application program, database management system, database, and hardware operating system. The user uses either the query language to interact with the database or the application programs that interact with the database using the query language.

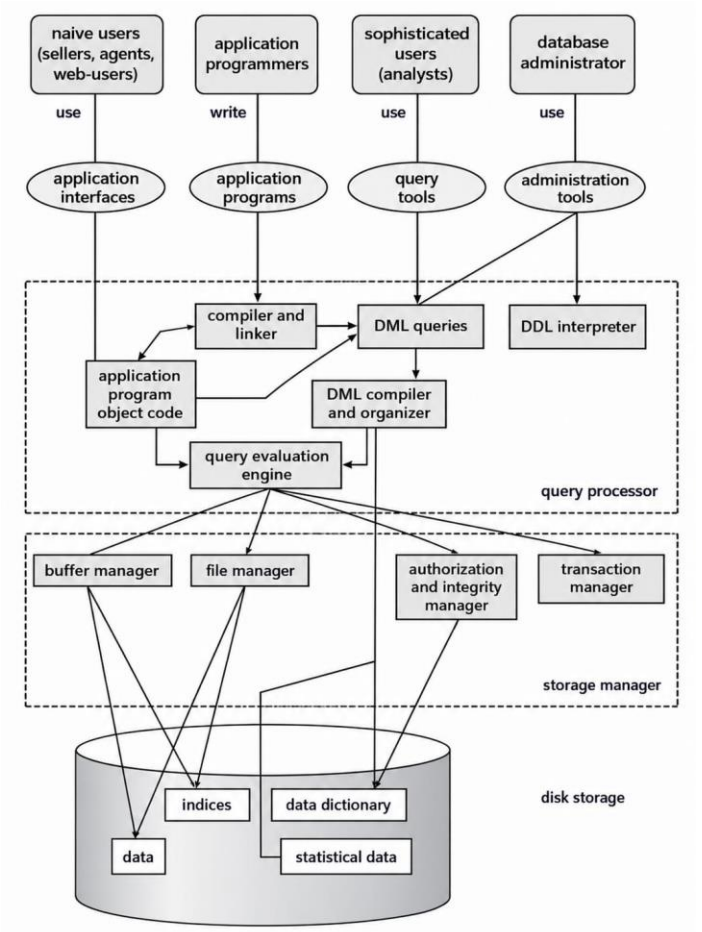
Query Processor, the most important component of the database management system architecture is the query processor, which oversees the entire process of query processing. After the receipt of the query, the query processor first evaluates whether the query is valid from the syntactical and semantic perspective. For this purpose, the **DDL interpreter** uses schema definition information and creates metadata, which is stored in the data dictionary. Simultaneously, the **DML compiler** translates the user query into the commands understandable to the database management system.

Storage manager is considered to be one of the most critical components of any relational database management system (RDBMS), as this manager organizes data physically on secondary storage devices. The storage manager links the **query processing** with data. In particular, the storage manager includes such components as the file manager, which is concerned with the organization of data files on the secondary storage media and allocation of disk space; buffer manager, which is responsible for transferring data from the disk into the main memory; and the transaction manager, among others.

The **database** includes the physical data file as well as the **data dictionary**, which includes metadata such as the definition of tables, constraints, permissions of users, and other storage forms. The data dictionary plays an essential role in database management, as it helps the DBMS understand how the database is structured. Data stored in the data dictionary is essential for any database operation.

The database management system operates within an extended environment of hardware and system software, such as servers, storages, network connections, and the operating system. All these components work together to make the data management process more efficient among many users. The whole combination forms an organized hierarchy where data can be abstracted, protected, managed concurrently, and recovered

Figure 1.1 Database System Environment



The **database system environment** is defined as the complete setting in which a Database Management System (**DBMS**) works, as illustrated in Figure 1.1. The database system environment consists of hardware, software, data, processes, and people, all contributing to the performance of **database-related tasks**. Hardware is the hardware components required to perform the database tasks, which include servers, storage

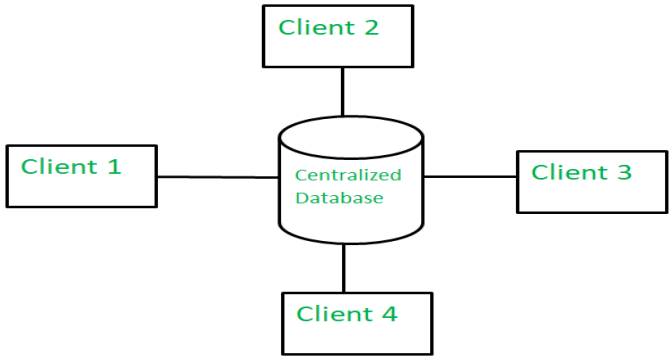
devices, and other computer hardware. Software is the computer software used, including the **DBMS software**, **operating system**, **and application software**

that interacts with the DBMS software. Data refers to both the actual data and the metadata contained in the data dictionary, which offers information about the database structure.

1.12 Centralized and Client–Server Architecture for the Database

1.12.1 Centralized Database Architecture

Figure 1.2:



Centralized Database

As indicated in **Figure 1.2**, the database and **Database Management System (DBMS)** exist in one central server in centralized database architectures. Users can access the database via terminals or client machines connected to the central server. All processes concerning database processing such as querying, transactions, maintaining consistency of transactions, and data recoveries are carried out by the central machine. Centralized architecture provides a number of strengths in relation to implement ability and security because both data administration and data backups are carried out from a centralized platform. However, as illustrated in Figure 1.2, there exist some weaknesses of centralized architecture in relation to scalability and availability. Should the central server fail, then the whole database application becomes unavailable. Centralization is suitable for organizations operating with one server managing the databases.

1.12.2 Client–Server Database Architecture

Client-Server Database Architecture, as shown in Figure 1.3, database processes are divided between **client computers** and the **database server**. The client manages all matters relating to the user interface and presentation in this architecture, while the database server manages database processes such as storing and retrieving data, managing transactions, and recovering data.

Commands from the client computer are sent to the database server, which performs the commands and provides the resulting output

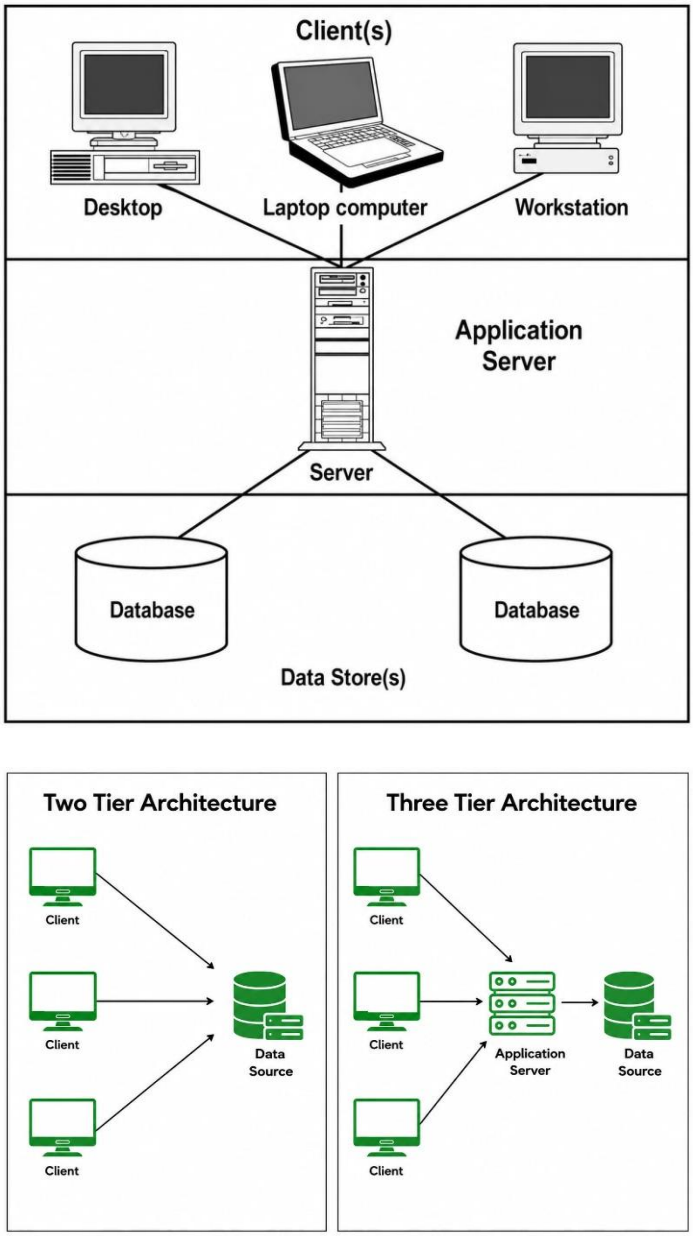


Figure 1.3: Client–Server Database Architecture

The implementation of the client-server architecture in **Figure 1.3** can be done either using a two-tier system or a three-tier system. In the two-tier system, the connection will

be made directly from the client and the database server; hence, it is applicable in small and medium-sized applications. In the three-tier system, there will be

an extra server, known as the application server, which will act as a mediator between the client and the database server.

1.13 Entity–Relationship (ER) Model: Introduction

The **Entity–Relationship (ER) Model** is a **high-level conceptual data model** used to describe the structure of a database in a graphical and intuitive manner. It was proposed by Peter Chen and is widely used during the **database design phase** to model real-world systems before converting them into relational schemas. As shown in **Figure 1.5**, the ER model represents data using three basic concepts: **entities**, **attributes**, and **relationships**.

The main objective of the ER model is to capture data requirements in a form that is easy to understand for both technical and non-technical users. It helps database designers identify data objects, their properties, and the associations among them. ER diagrams serve as a blueprint for transforming conceptual designs into tables, keys, and constraints in relational databases.

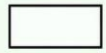
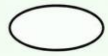
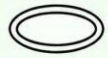
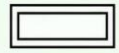
Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

Figure 1.5 Symbol used in ER

1.14 Representation of Entities

An **entity** is a real-world object or concept that has an **independent existence**. In ER diagrams, entities are represented using **rectangles**, as shown in **Figure 1.6**. Each entity is described using attributes.

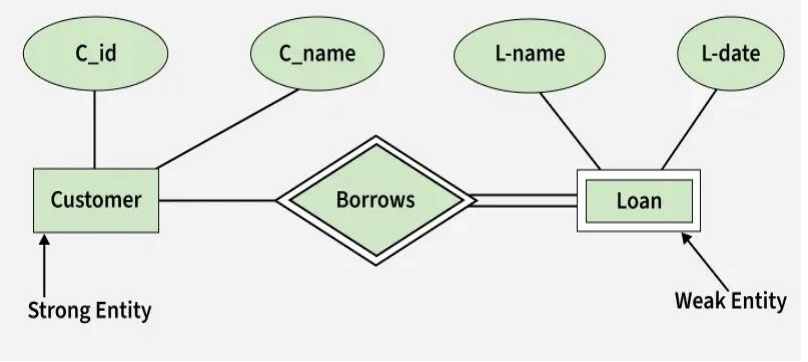


Figure 1.6 Strong Entity and Weak Entity

Entities are classified into **strong entities** and **weak entities**. A **strong entity** has its own primary key and exists independently, whereas a **weak entity** does not have a primary key of its own and depends on a strong entity for identification. Weak entities are represented using **double rectangles**. The figure contains two entities: **Customer** and **Loan**. **Customer** is a **strong entity** with

an independent existence and is uniquely identified by **C_id**. **Loan** is a **weak entity** that does not have an independent identity and exists only in association with the Customer entity.

1.15 Attributes

An **attribute** is a property that describes an entity or a relationship. Attributes are represented using **ovals** connected to entities. There are seven types of attributes. They are

1. Simple (Atomic) Attribute

A **simple attribute** cannot be divided into smaller meaningful components. As shown in **Figure 1.7**, attributes such as *Age* and *Roll_No* are atomic in nature.

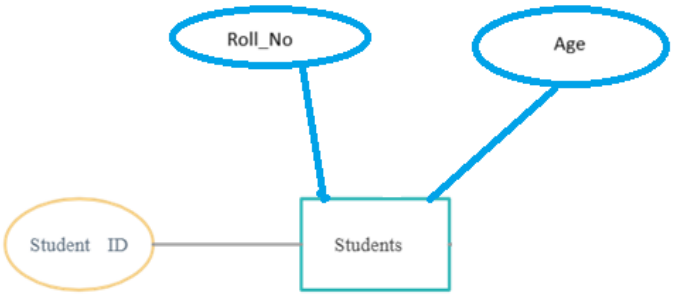


Figure 1.7: Simple Attribute

2. Composite Attribute

A **composite attribute** can be divided into sub-attributes. In **Figure 1.8**, *Address* is divided into *Street*, *City*, and *State*.

Example: Address of STUDENT

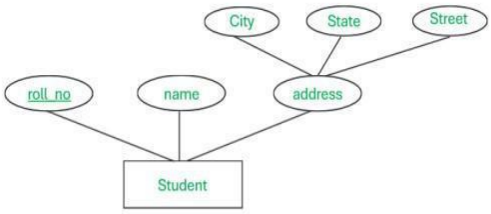


Figure 1.8: Composite Attribute

3. Single-Valued Attribute

A **single-valued attribute** holds only one value for each entity instance, as shown in **Figure 1.9**.

Example: Date_of_Birth of STUDENT.

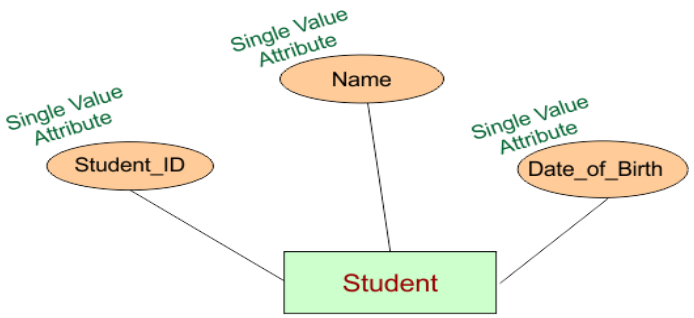


Figure 1.9: Single-Valued Attribute

4. Multi-Valued Attribute

A **multi-valued attribute** can store multiple values for a single entity and is represented using a **double oval**, as shown in **Figure 1.10**.

Example: Phone_Number of STUDENT.

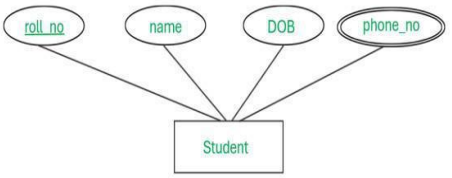


Figure 1.10: Multi-Valued Attribute

5. Derived Attribute

A **derived attribute** is calculated from another attribute and is represented using a **dashed oval**, as shown in **Figure 1.11**

Example: Age derived from Date_of_Birth.

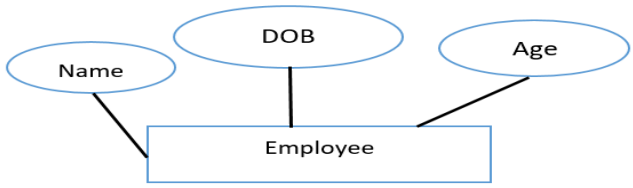


Figure 1.11: Derived Attribute

6. Stored Attribute

A **stored attribute** is physically stored in the database and used to derive other attributes.

Example: Date_of_Birth.

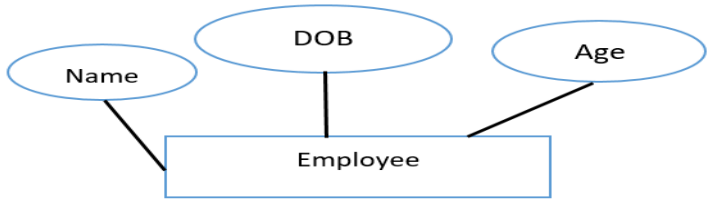


Figure 1.12: Stored Attribute

7. Key Attribute

A **key attribute** uniquely identifies each entity in an entity set and is represented by **underlining the attribute name**, as shown in **Figure 1.13**.

Example: Roll_No of STUDENT.

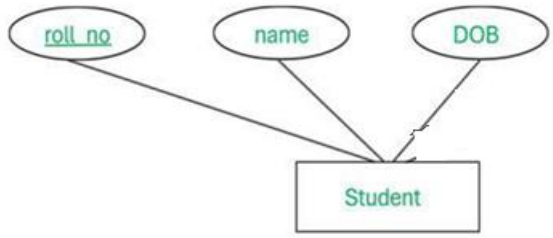


Figure 1.13: Key Attribute

1.16. Entity Set

An entity set is a collection of similar entities that share the same attributes. An entity represents a real-world object or concept that has an independent existence, whereas an entity set represents a group of such entities. All entities within an entity set have the same structure but differ in their attribute values.

For example, the STUDENT entity set represents all students in a university. Each individual student is an entity, while STUDENT as a whole is an entity set. Entity sets are represented in ER diagrams using rectangles, and their attributes are connected to them using ovals.

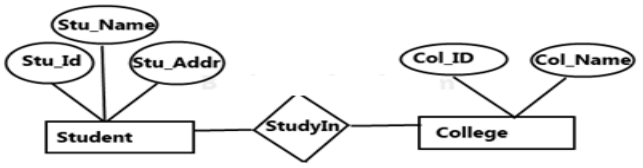


Figure 1.14: Representation of an Entity Set

As shown in Figure 1.14 shows two **entity sets**, **STUDENT** and **COLLEGE**. The STUDENT entity set consists of all student entities described by attributes such as *Stu_Id, Stu_Name,* and *Stu_Addr*. The COLLEGE entity set represents all colleges and is described by attributes such as *Col_ID* and *Col_Name*. The **StudyIn** relationship set represents the association between the STUDENT and COLLEGE entity sets, indicating that students study in colleges. Each instance of the relationship set connects one student entity with one college entity.

1.17 Relationship

A relationship represents an association among two or more entities. While entities model objects, relationships model how those objects are connected in the real world. A relationship instance represents a specific association among entity instances. For example, when a student enrolls in a course, this association is captured using a relationship. Relationships are represented in ER diagrams using diamond-shaped symbols.

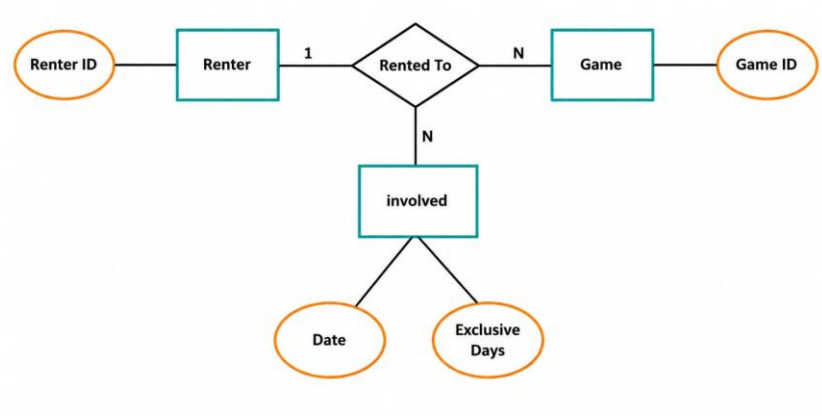


Figure 1.15: Representation of a Relationship

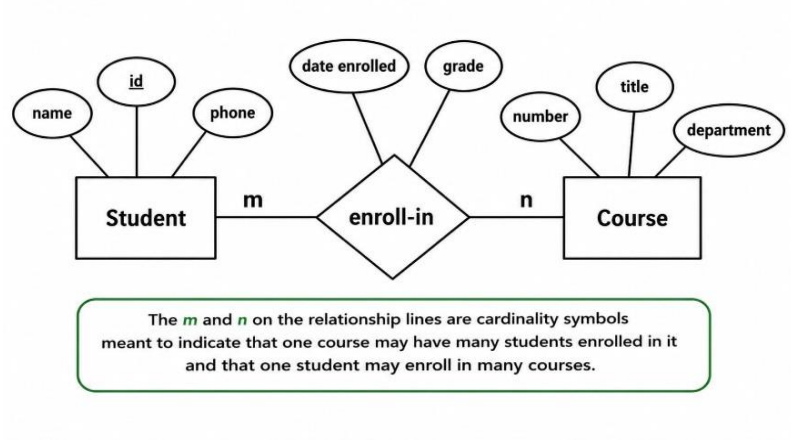
In Figure 1.15, the relationship ENROLLED IN connects the STUDENT and COURSE entity sets. Each instance of the ENROLLS relationship links one specific student to one specific course. Relationships may also have their own attributes, such as EnrollmentDate, which describe properties of the association rather than the entities themselves. Relationships are essential for representing interactions and dependencies among entity sets in a database.

1.18. Relationship Set

A relationship set is a collection of similar relationship instances involving the same participating entity sets. Just as an entity set is a collection of entities, a relationship set is a collection of relationships of the same type.

For example, all instances of students enrolling in various courses together form the ENROLLS relationship set. Each instance in the relationship set connects one STUDENT entity with one COURSE entity.

Figure 1.16: Relationship Set Representation



As illustrated in Figure 1.16, the ENROLLS relationship set consists of all enroll-in associations between STUDENT and COURSE entity sets. The relationship set captures the overall pattern of interaction between the participating entity sets rather than a single association. Relationship sets help in defining constraints such as cardinality (one-to-many, many-to-many) and participation (total or partial), which further refine the database design.

1.19 Constraints

Constraints in the Entity–Relationship (ER) model are rules that restrict the data that can be stored in a database. They represent semantic properties of the real-world enterprise and ensure the integrity and validity of data. Constraints are specified at the conceptual design stage and are reflected in the ER diagram.

1. Key Constraint

A key constraint specifies that an entity set must have an attribute, or a combination of attributes, whose values uniquely identify each entity in the set. Such an attribute is called a key attribute. No two entities in an entity set can have the same value for the key attribute.

For example, in a *Student* entity set, the attribute ***Student_ID*** **uniquely identifies each student.**

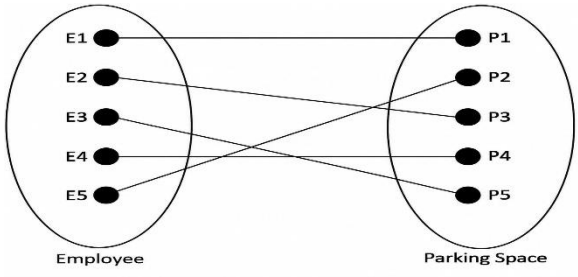
2. Cardinality Constraint

A cardinality constraint specifies the minimum and the maximum number of entities that can participate in a relationship. The common cardinality ratios are:

- One-to-One (1:1)
- One-to-Many (1:N)
- Many-to-One (N:1)
- Many-to-Many (M:N)

These constraints describe how entities are associated in the real world.

Example for Cardinality – One-to-One (1:1)

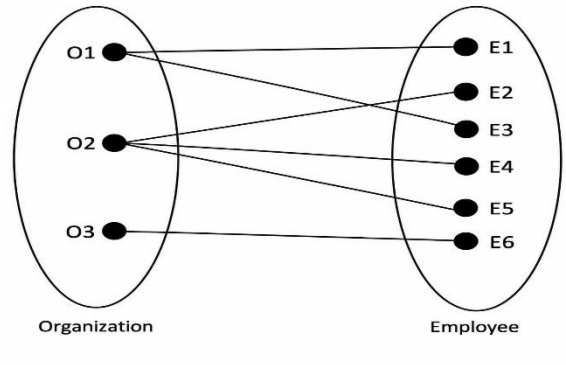


Employee is assigned with a parking space.

One employee is assigned with only one parking space and one parking space is assigned to only one employee. Hence it is a 1:1 relationship and cardinality is One-To-One (1:1)

In ER modeling, this can be mentioned using notations as given below

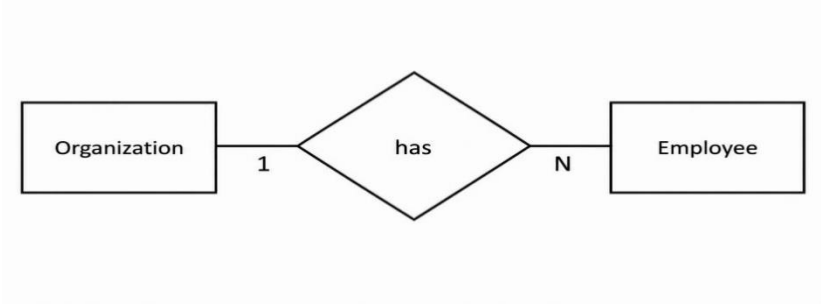
Example for Cardinality – One-to-Many (1:N)



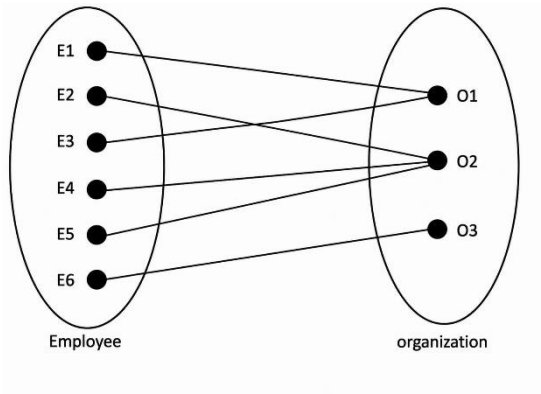
Organization has employees

One organization can have many employees, but one employee works in only one organization. Hence it is a 1:N relationship and cardinality is One-To-Many (1:N)

In ER modeling, this can be mentioned using notations as given below

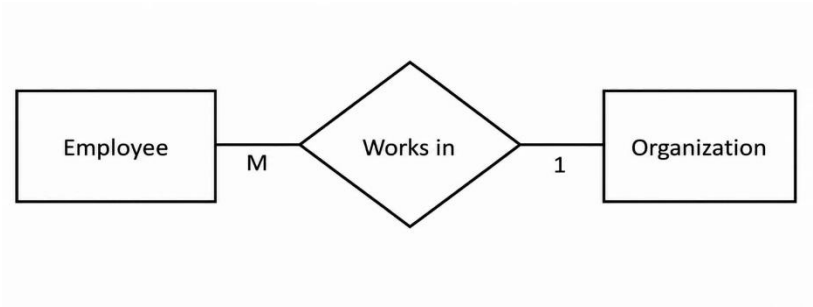


Example for Cardinality – Many-to-One (M :1)

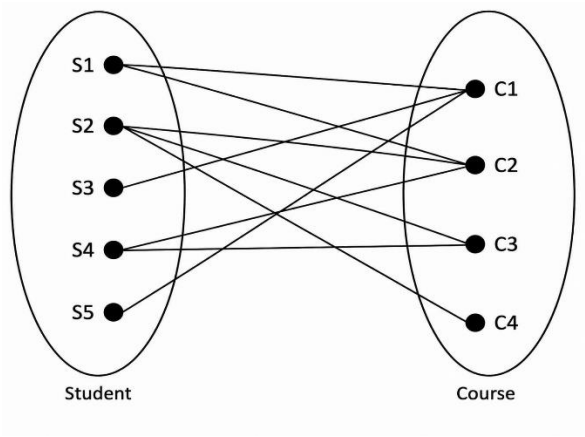


It is the reverse of the One to Many relationship. employee works in organization One employee works in only one organization But one organization can have many employees. Hence it is a M:1 relationship and cardinality is Many-to-One (M :1)

In ER modeling, this can be mentioned using notations as given below.



Cardinality – Many-to-Many (M:N)

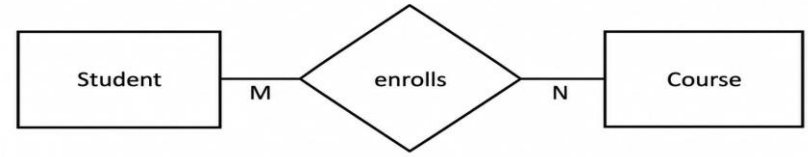


Students enrolls for courses

One student can enroll for many courses and one course can be enrolled by many students. Hence it is a M:N relationship and cardinality is Many-to-Many (M:N)

In ER modeling, this can be mentioned using notations as given sbelow

3. Participation Constraint



A participation constraint determines whether participation of an entity set in a relationship is mandatory or optional.

- Total participation: Every entity participates in the relationship (represented by a double line).

- Partial participation: Some entities may not participate (represented by a single line).

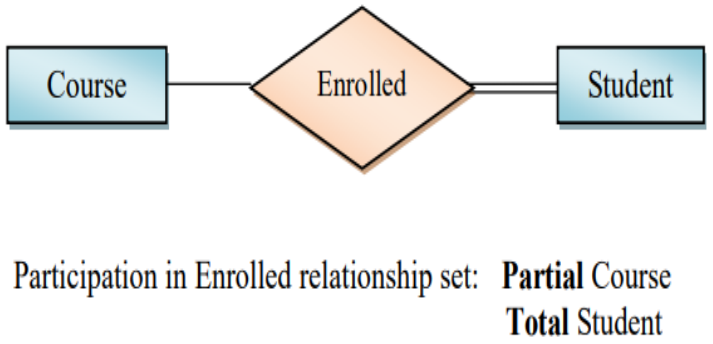


Figure 1.2: Participation Constraints

1.20. Superclass and Subclass

The Enhanced Entity–Relationship (EER) model extends the ER model by introducing superclasses and subclasses, allowing the representation of hierarchical relationships among entities.

1 Superclass

A superclass is a generalized entity set that contains common attributes and relationships shared by multiple subclasses. It represents a higher-level abstraction.

Example:
Employee(Emp_ID, Name, Salary) is a superclass.

2 Subclass

A subclass is a specialized entity set derived from a superclass. It inherits all the attributes of the superclass and may have additional attributes specific to the subclass.

Example:

- Teaching_Staff(Subject, Qualification)
- Non_Teaching_Staff(Department)

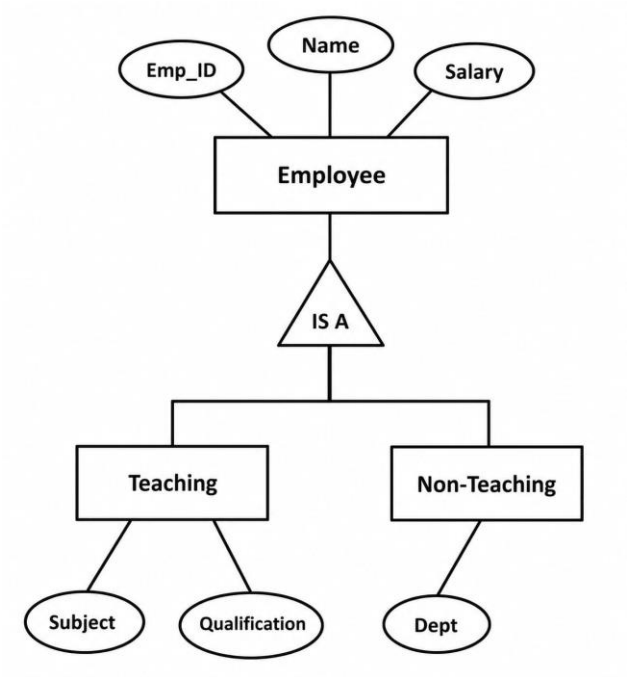


Figure 1.17: Superclass–Subclass Relationship

1.21 Inheritance

Inheritance is a fundamental concept of the EER model by which a subclass automatically acquires:

- All attributes of the superclass
- All relationships in which the superclass participates

Inheritance avoids redundancy and improves schema clarity.

For example, if *Professor* is a subclass of *Employee*, then *Professor* inherits attributes such as Employee_ID, Name, and Salary from *Employee*, in addition to its own attributes like Research_Area.

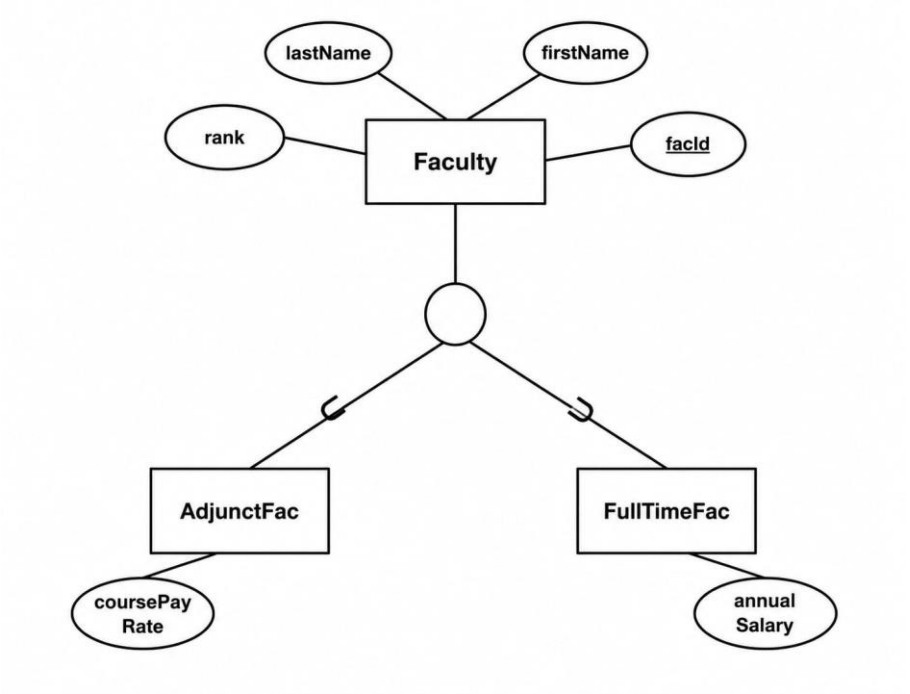


Figure 1.18: Inheritance in EER Diagram

1.22 Specialization

Specialization is a top-down design process in which a higher-level entity set is divided into lower-level entity sets based on distinguishing characteristics. Each subclass represents a subset of the entities in the superclass.

Example:
Employee → Teaching_Staff, Non_Teaching_Staff

Specialization Constraints

a) Disjointness Constraint

Specifies whether an entity of the superclass can belong to more than one subclass.

- Disjoint specialization: Entity belongs to only one subclass.
- Overlapping specialization: Entity can belong to multiple subclasses.

b) Completeness Constraint

Specifies whether all superclass entities must belong to at least one subclass.

- Total specialization: Every superclass entity must be in a subclass.
- Partial specialization: Some superclass entities may not belong to any subclass.

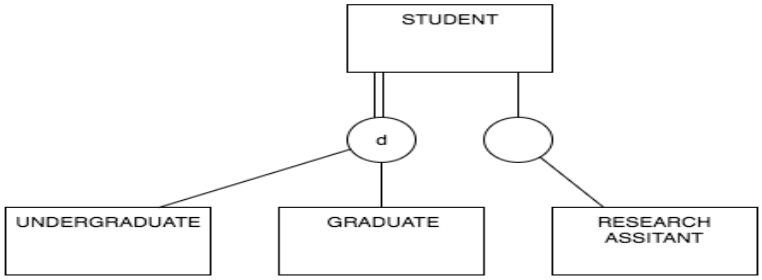


Figure 1.19: Total and Partial Specialization

This diagram shows multiple specialisations of the superclass STUDENT, where two separate IS-A circles divide students into

different subclasses with different constraints. The left circle, marked with "d" and connected to STUDENT via a double line, represents a disjoint and total specialisation meaning every student must belong to either UNDERGRADUATE or GRADUATE, but cannot belong to both at the same time. The right circle, which is empty (no "d") and connected via a single line, represents an overlapping and partial specialisation — meaning a student can belong to both GRADUATE and RESEARCH ASSISTANT simultaneously, and some students may not fall into either subclass at all. Notice that GRADUATE appears in both specialisations, which means a graduate student can also be a research assistant (due to the overlapping right circle), but can never be an undergraduate at the same time (due to the disjoint left circle). In short, the two specialisation circles operate independently, each enforcing its own disjointness and completeness constraints on the STUDENT superclass.

1.23 Generalization

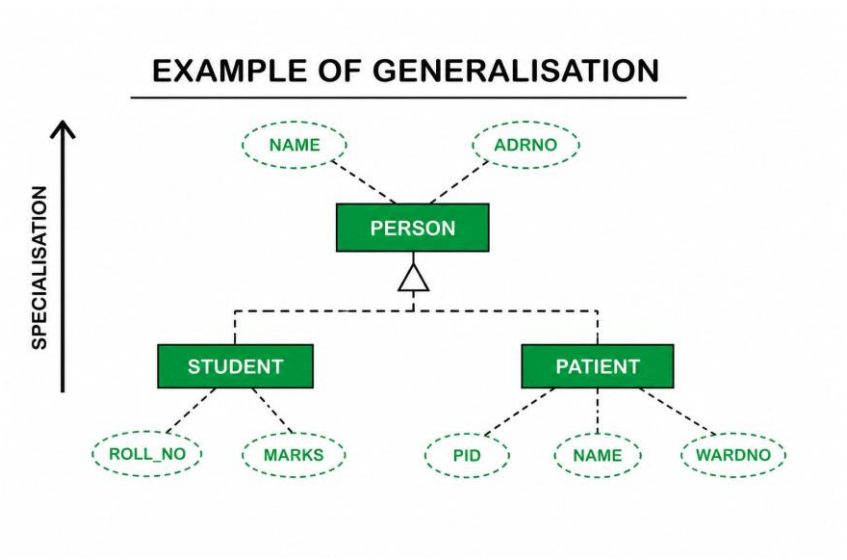
Generalization is the bottom-up design process, which is the reverse of specialization. In generalization, multiple lower-level entity sets with common features are combined into a single higher-level entity set (superclass).

Example: This diagram demonstrates **Generalisation** in ER (Entity-Relationship) modeling, which follows a **bottom-up approach** where common attributes from lower-level entities are combined into a higher-level generalised

entity. Here, two specific entities — **STUDENT** (with attributes ROLL_NO, NAME, MOBNO) and **PATIENT** (with attributes PID, NAME, MOBNO) share common attributes NAME and MOBNO, so these are "generalised" upward into a new parent entity called **PERSON**. The hollow triangle/arrow pointing upward between PERSON and its sub-entities represents an **IS-A relationship**, meaning a STUDENT *is a* PERSON and a PATIENT *is a* PERSON. Through **inheritance**, both STUDENT and PATIENT automatically acquire NAME and MOBNO from PERSON, eliminating redundancy. The arrow on the left labeled "Bottom Up Approach" reinforces that in generalisation, we start from specific entities and move upward to form a more general entity, which is the opposite of specialisation.

Generalization improves abstraction, reduces duplication, and simplifies database design by identifying common attributes and relationships.

Figure 1.20: Generalization in EER Diagram



Specialization is a top-down approach, whereas generalization is a bottom-up approach, as illustrated in **Figure 1.21**

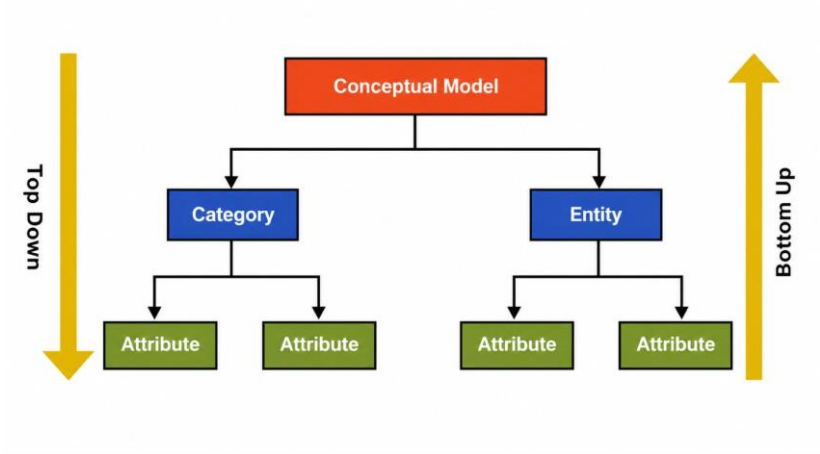


Figure 1.21: Specialization and Generalization

Chapter Summary

1.1 Introduction to Database Systems

A database system is an organized collection of interrelated data combined with a set of programs (DBMS) that enables efficient storage, retrieval, updating, and management of data. The DBMS acts as an intermediary between the physical database and users/applications, providing data abstraction, data independence, security, integrity enforcement, concurrency control, and recovery from failures.

1.2 Database vs File System

Key problems with the file system approach that databases solve:

- Data Redundancy: Same data duplicated across multiple files, increasing storage cost.
- Data Inconsistency: Inconsistent values when updates are not propagated uniformly.
- Data Integrity: In file systems, constraints enforced by programs; in DBMS, enforced at schema level.
- Data Isolation: Scattered files make combining information difficult.
- Data Security: DBMS provides fine-grained access control at table/record/attribute level.
- Difficulty in Accessing Data: File systems require new programs for new query types; DBMS allows flexible SQL queries.

1.3 Database Users

- Naive Users: End users interacting via predefined applications (e.g., ATM users).
- Application Programmers: Developers building software on top of the DBMS.
- Sophisticated Users: Data analysts querying directly using SQL.
- Specialized Users: Designers of non-traditional applications (CAD, GIS, AI).
- Database Administrator (DBA): Defines schema, grants authorization, monitors performance, and ensures backups and recovery.

1.4 Advantages of Database Systems

- Reduction of data redundancy and inconsistency
- Data sharing with concurrency control
- Improved data integrity through schema-level constraints

- Enhanced data security via authentication and access control
- Backup and recovery mechanisms
- Data independence: changes in storage structure do not affect applications

1.5 Data Models

Data models describe database structure, relationships, constraints, and operations. Major types include:

- Hierarchical Model: Tree structure; parent-child relationships; one-to-many only.
- Network Model: Graph structure; supports many-to-many; pointer-based navigation.
- Relational Model: Tables (relations) with rows and columns; SQL-based; most widely adopted.
- Entity-Relationship (ER) Model: High-level conceptual model using entities, attributes, and relationships.
- Object-Oriented Model: Combines OOP concepts (encapsulation, inheritance) with database technology.
- Object-Relational Model: Extends relational model with complex data types and user-defined types.
- Semi-Structured Model: Flexible schema (XML, JSON) for varying attribute structures.
- Physical Data Model: Describes how data is stored (file organization, indexing, hashing).

1.6 Schema and Instance

Schema: The logical blueprint/structure of the database (tables, attributes, constraints). Does not change frequently. Instance: The actual data stored at a particular point in time. Changes with every insert, update, or delete operation.

1.7 Data Independence

- Physical Data Independence: Ability to change internal (physical) schema without affecting conceptual or external schemas.
- Logical Data Independence: Ability to change conceptual schema without affecting external schemas or applications. Harder to achieve.

1.8 Three-Tier Schema Architecture

- External Schema (View Level): How individual users see the database. Multiple views possible.
- Conceptual Schema (Logical Level): Complete logical structure of the

entire database.

- Internal Schema (Physical Level): Physical storage details (file structures, indexing, hashing).

Mappings between levels enable data independence: External-Conceptual mapping ensures view stability; Conceptual-Internal mapping ensures physical changes are transparent.

1.9 ER Model

Proposed by Peter Chen, the ER model represents data using entities, attributes, and relationships. Key concepts:

- Strong Entity: Has its own primary key (represented by a rectangle).
- Weak Entity: Depends on another entity for identification (double rectangle).
- Attribute Types: Simple, Composite, Single-Valued, Multi-Valued, Derived, Stored, Key.
- Relationship: Association between entities (diamond shape).
- Cardinality Constraints: 1:1, 1: N, M:1, M: N define how many entities participate.
- Participation Constraints: Total (mandatory) vs. Partial (optional) participation.

1.10 Enhanced ER (EER) Model

- Superclass/Subclass: Generalized vs. specialized entity sets with inheritance.
- Specialization: Top-down process dividing a superclass into subclasses.
- Generalization: Bottom-up process combining subclasses into a superclass.
- Disjoint vs. Overlapping: Whether an entity can belong to multiple subclasses.
- Total vs. Partial Specialization: Whether every superclass entity must belong to a subclass.

1.11 Centralized vs. Client-Server Architecture

- Centralized Architecture: Entire DBMS and database at one server. Simple but single point of failure.
- Two-Tier Client-Server: Client handles UI; server manages data. Suitable for small-medium applications.
- Three-Tier Architecture: Client (UI) + Application Server (business logic) + Database Server. Scalable and widely used in modern systems.

Review Questions

Section A: Conceptual Questions

1. Define a Database Management System (DBMS). How does it differ from a traditional file system? List at least five limitations of the file system approach that DBMS overcomes.
2. Explain the three-tier schema architecture (External, Conceptual, Internal). How does this architecture achieve both physical and logical data independence? Provide an example for each.
3. Describe all seven types of attributes in the ER model with examples and their diagrammatic representation (simple, composite, single-valued, multi-valued, derived, stored, key).
4. Differentiate between a strong entity and a weak entity in the ER model. What is the significance of a discriminator (partial key) for a weak entity?
5. Compare the Hierarchical, Network, and Relational data models. What are the limitations of each, and why has the relational model become the most widely adopted?
6. Explain the roles of different categories of database users: naive users, application programmers, sophisticated users, and the Database Administrator (DBA). What are the key responsibilities of the DBA?
7. What is the difference between schema and instance in a database system? Why is it important that schema changes are less frequent than instance changes?
8. Describe specialization and generalization in the EER model. What are disjoint and overlapping specializations? Provide a real-world example for each.
9. What are the four types of cardinality constraints in the ER model? Draw ER notation for each and provide a real-world scenario illustrating each type.
10. Compare centralized database architecture with client-server architecture. Under what conditions would each be preferred?

Section B: Analytical Problems

11. Draw a complete ER diagram for a Hospital Management System that includes: Patients, Doctors, Nurses, Departments, Wards, and Treatments. Include appropriate attributes, primary keys, cardinality constraints, and participation constraints. Identify any weak entities.
12. A university database stores information about Students, Courses,

Professors, and Departments. Each student can enroll in many courses; each course can be taught by multiple professors; each professor belongs to one department; each department offers many courses. Design an ER diagram with all appropriate relationships and constraints.

13. Consider a relation EMPLOYEE (EmpID, Name, Salary, DeptID, ManagerID). Identify which attributes represent different types. What functional dependencies can you derive? How would you represent the 'reports-to' relationship (manager-employee) in an ER diagram?
14. An online retail store needs to track Products, Categories, Customers, Orders, OrderItems, and Shipments. Model this system with an ER diagram. Identify any derived attributes (e.g., total order amount). Specify cardinality and participation for each relationship.
15. Given the following information: A library has Books, Members, and Authors. A book can have multiple authors. A member can borrow multiple books; a book can be borrowed by multiple members over time, with borrow and return dates recorded. Model this with an ER diagram, including a weak entity if applicable.

Section C: Applied and Design Questions

16. Design a three-schema architecture for an E-commerce system. Describe what the external schema for a Customer would look like versus the external schema for an Administrator. What would the conceptual and internal schemas contain
17. A hospital is migrating from a file-based system to a DBMS. Identify at least four specific problems they would face with the file system and explain exactly how each problem would be resolved by implementing a DBMS.
18. Compare the use of total vs. partial participation constraints in ER modeling. Give a scenario where incorrectly assigning participation type could lead to incorrect data entry. How does the DBMS enforce these constraints?
19. Propose an EER design for a university system with superclass Person and subclasses Student, Faculty, and Staff. Specify which attributes are inherited and which are specific to each subclass. Is the specialization disjoint or overlapping? Total or partial?
20. Discuss why logical data independence is considered harder to achieve than physical data independence. Provide a concrete scenario where a change in the conceptual schema could break existing applications, and explain how views (external schemas) help mitigate this risk.

CHAPTER - 2

RELATIONAL MODEL

Chapter Overview

This Chapter provides a rigorous treatment of the relational model the mathematical foundation of modern SQL databases. Beginning with the formal definitions of domains, attributes, tuples, and relations, the chapter establishes the precise conceptual vocabulary. It then covers constraint types (domain, key, entity integrity, referential integrity), followed by an extensive study of Relational Algebra — the procedural query language that underpins SQL. The chapter concludes with Relational Calculus (tuple and domain) and a practical section on simple database schema design.

Learning Objectives	**By the end of this chapter, the reader will be able to:** 1. Define domain, attribute, tuple, relation, degree, and cardinality in formal relational model terms 2. Explain the significance of NULL values and how they differ from zero or empty string 3. Apply domain, key, entity integrity, and referential integrity constraints to database schemas 4. Write and evaluate relational algebra expressions using selection, projection, union, difference, and Cartesian product 5. Perform natural joins and understand join semantics formally 6. Apply aggregate functions (COUNT, AVG, MAX) in relational algebra expressions 7. Distinguish between tuple relational calculus and domain relational calculus 8. Design a simple multi-table database schema with appropriate keys and relationships

Section-by-Section Roadmap

Section	Topic	Key Concepts
2.1	**Concept of Domain**	Domain as named set of atomic values; domain constraints on attributes
2.2	**Attribute**	Attribute definition, domain association, NULL values as unknown/inapplicable
2.3	**Tuple**	Ordered list of attribute values; one tuple = one real-world fact
2.4	**Relation**	Relation schema, relation instance, degree, cardinality, no duplicate tuples
2.5	**NULL Values**	Significance of NULL; distinction from zero/blank; impact on queries
2.6	**Constraints**	Domain, key, entity integrity, referential integrity constraints
2.7–2.10	**Relational Algebra**	Selection (σ), projection (π), union, difference, Cartesian product with worked examples
2.8–2.11	**Join Operators**	Natural join (⋈), theta join, equijoin; join semantics and results
2.9	**Aggregate Functions**	COUNT, AVG, MAX in relational algebra; grouping semantics
2.12	**Relational Calculus**	Tuple relational calculus (TRC) and domain relational calculus (DRC)
2.13	**Simple Database Schema**	Schema design, purpose, multi-table example with primary and foreign keys

The **relational model** is a logical data model that represents data in the form of **relations**, which are mathematically defined structures based on set theory. Although relations are commonly visualized as tables, conceptually they are **sets of tuples**, where each tuple represents a real-world fact. The relational model was proposed to overcome the limitations of earlier data models by providing a simple, uniform, and theoretically sound framework for data storage and retrieval.

In the relational model, both data and relationships among data are represented using relations. This uniform representation simplifies database design and enables powerful query languages such as SQL. A key advantage emphasized in Ramakrishnan is that users interact with the database at a **logical level**, without concern for how data is physically stored. This abstraction leads to **data independence**, which is a central goal of database systems.

In the relational model, data is represented using **relations**, which are formally defined as **sets of tuples**. Each relation has a **relation schema** that specifies the relation name and a fixed set of attributes with associated domains, and a **relation instance** that contains the actual data at a particular point in time. The **degree** of a relation is the number of attributes in its schema, while the **cardinality** of a relation is the number of tuples present in its instance. Since a relation is a set, the order of tuples and attributes has no significance, and duplicate tuples are not permitted. Each tuple represents a single real-world fact and must contain exactly one atomic value for each attribute, drawn from the corresponding domain.

Example Illustrating Relation, Degree, and Cardinality

Consider the relation:

STUDENT (rollno, name, age, dept)

rollno	name	age	dept
101	Ravi	20	CSE
102	Anu	21	ECE
103	Kiran	19	EEE

In this relation STUDENT, the **degree** is **4**, because the relation schema consists of four attributes: *rollno*, *name*, *age*, and *dept*. The **cardinality** of the relation is **3**, since there are three tuples present in the relation instance. Each row represents a single tuple, each column represents an attribute, and all attribute values are atomic and drawn from their respective domains, satisfying the requirements of the relational model.

2.1. Concept of Domain

A **domain** is a fundamental concept in the relational model. It is defined as a **set of atomic values**, all drawn from the same underlying data type. Every attribute in a relation is associated with exactly one domain, and all values stored in that attribute must belong to the specified domain.

Domains enforce **type correctness** and semantic validity of data. They may be based on standard data types such as integers, characters, or dates, or they may be **user-defined**, representing application-specific value sets such as department codes or grade values.

Formally, if an attribute *A* has domain *D*, then for every tuple *t* in the relation, the value *t[A]* must be an element of *D*.

Example of Domains

Consider the following domains:

- **D_rollno** = Positive integers
- **D_name** = Character strings of length ≤ 30
- **D_age** = Integers between 17 and 60
- **D_dept** = {CSE, ECE, EEE, MECH}

These domains restrict the kind of values that can be stored in corresponding attributes.

2.2. Attribute

An **attribute** is a named property or characteristic of an entity or relationship. In a relation, attributes correspond to **columns** of a table, but conceptually they represent properties defined over domains.

Each attribute:

- Has a unique name within a relation

- Is associated with exactly one domain
- Takes a single atomic value in each tuple

Attributes give **meaning** to the data stored in a relation by defining what kind of information each column represents.

Example

Consider the relation schema:

STUDENT (rollno, name, age, dept)

Here:

- *rollno* represents the student identification number
- *name* represents the student name
- *age* represents the student's age
- *dept* represents the department

Each of these attributes draws values from its respective domain.

2.3. Tuple

A **tuple** is an ordered collection of attribute values that represents a **single record** or fact in a relation. Each tuple contains exactly one value for each attribute defined in the relation schema, and the value must belong to the attribute's domain.

Tuples are conceptually unordered, meaning there is no inherent ordering among tuples in a relation. Each tuple represents one real-world entity or association.

Example

For the STUDENT relation, a tuple may be:

rollno	name	age	dept
101	Ravi	20	CSE

This tuple represents a specific student and is valid because each value conforms to its domain.

2.4. Relation

A **relation** is a finite set of tuples that all conform to the same relation schema. Since relations are sets, duplicate tuples are not allowed. The order of tuples and attributes is irrelevant in the relational model, even though tables may display them in a particular order.

Formally, a relation R over schema $(A_1, A_2, ..., A_n)$ is a subset of the Cartesian product of the domains of those attributes:

$$\mathbf{R \subseteq D_1 \times D_2 \times \ldots \times D_n}$$

This formal definition highlights the mathematical basis of the relational model.

Example of a Relation STUDENT

rollno	name	age	dept
101	Ravi	20	CSE
102	Anu	21	ECE
103	Kiran	19	EEE

Each row is a tuple, each column is an attribute, and the entire table represents a relation.

2.5. NULL Values and Their Importance

In the relational model, a NULL value is used to represent missing, unknown, or inapplicable information. As emphasized in Ramakrishnan, NULL is not a value in the usual sense; rather, it is a marker indicating the absence of a known value. NULL values commonly arise when information is unavailable at the time of data entry or does not apply to a particular tuple.

NULL values introduce complexity in query processing and constraint enforcement. Any comparison involving NULL does not evaluate to TRUE or FALSE, but to UNKNOWN. Therefore, special predicates such as **IS NULL** and **IS NOT NULL** are required to test for NULL values. Despite these complications, NULL values are essential for modeling real-world situations where complete information is not always available.

Example

rollno	name	age	dept
104	**Meena**	**20**	**NULL**

Here, the department of the student is not known or not yet assigned, and hence NULL is used.

2.6. Introduction to Constraints

In the relational model, **constraints** are rules that restrict the set of valid relation instances for a given relation schema. They represent **semantic rules of the real world** that must be enforced on the database to ensure correctness, consistency, and reliability of stored data.

Constraints are defined at the schema level and are automatically enforced by the database management system. The most important constraints in the relational model are **domain constraints**, **key constraints**, and **integrity constraints**.

2.6.1 Domain Constraints

A **domain constraint** specifies that the values of an attribute must belong to a predefined domain. Since every attribute in a relation is associated with a domain, domain constraints ensure that only **valid and meaningful values** are stored in that attribute. Domains are usually defined using data types, value ranges, formats, or fixed sets of permissible values.

Example

Consider the relation:

STUDENT (rollno, name, age, dept)

Let the domains be defined as:

- *rollno*: positive integers
- *name*: character strings of length ≤ 30
- *age*: integers between 17 and 60
- *dept*: {CSE, ECE, EEE, MECH}

If an attempt is made to insert a tuple with *age = –5* or *dept = 'CIVIL'*, the domain constraint is violated and the operation is rejected.

Importance of Domain Constraints

Domain constraints can also be utilized to make sure the data that was entered is accurate, prohibit the inserting of meaningless data and fix define exactly how you want the type of your data as well. Another area they simplify query

processing. Formulating the conditions is straightforward because attribute values exist within the specified domain. A key constraint is that each tuple must have a unique identity. There is a key like a set of attribute values.

Domain constraints improve data correctness by not only 'removing' impossible data enters but also doing type checking. They enable questions with OPL to be evaluated since attribute values must follow the constraints and send to proper domains. The database constraints guaranteeing that no two tuples in relation are equal to each other. Key is a concatenated set of attributes that can identify all its tuples in relation.

2.6.2 Key Constraints

A key constraint states that tuples of a relation can be identified uniquely. A set of one or more attributes is a key if the value of those attributes can uniquely identify the tuple. If K is a key of R, then for every pair of the tuples t 1 and t 2 t 1 t 2, then values of attributes 0…K should not match between t 1 and t 2. From several key candidates, one key is selected as primary key and all other keys are called candidate keys.

Among all possible keys, one is chosen as the primary key, while others are referred to as candidate keys.

Importance of Key Constraints

Fundamental Constraints · Prevent Duplication of records. · Provide Efficient Data Access. · Support defining relationships between relations using foreign keys.

2.6.3 Integrity Constraint

Integrity constraints are high-level abstraction constraints that ensure consistency of the database. Entity integrity and referential integrity are the two main types of integrity constraints.

2.6.3.1 Entity Integrity Constraint

Entity integrity constraints the primary keys attributes must not have NULL values. NULL Values are Not Allowed as Primary Key: A primary key uniquely identifies a single tuple, and if NULL is allowed in the primary key then the entity cannot be uniquely identified.

Importance of Entity Integrity

Entity integrity states that all tuples owe to a real and identifiable entity. It avoids partial or fuzzy identification of records.

2.6.3.2 Referential Integrity Constraint

The **referential integrity constraint** ensures consistency between related relations. It requires that a **foreign key** value in one relation must either:

- Match a primary key value in the referenced relation, or
- Be NULL

Example

Consider the relations:

STUDENT(rollno,name,age,dept)
ENROLL (rollno, courseid)

Here, *rollno* in ENROLL is a **foreign key** that references *rollno* in STUDENT.

rollno	courseid
105	DBMS

If there is no student with *rollno = 105* in the STUDENT relation, this tuple violates referential integrity.

Importance of Referential Integrity

Referential integrity prevents **dangling references**, ensures consistency across relations, and preserves the correctness of relationships among data.

2.7 RELATIONAL ALGEBRA

2.7.1 Introduction to Relational Algebra

Relational Algebra is a **procedural query language** for the relational data model. It consists of a collection of **operations that take relations as input and produce relations as output**. Since every operation yields a relation, relational algebra satisfies the **closure property**.

Relational algebra defines **how** a query is executed and forms the **foundation for SQL query processing and optimization** in database systems.

2.7.2 Properties of Relational Algebra

1. Operates on relations (tables)
2. Output of every operation is a relation

3. Supports formal reasoning about queries
4. Independent of physical data storage
5. Expressively equivalent to relational calculus

Common Relations Used in Examples

STUDENT

SID	Name	Dept	Age
1	Anil	CSE	20
2	Ravi	ECE	21
3	Meena	CSE	22
4	Kiran	ME	23

COURSE

CID	Title	Dept
C1	DBMS	CSE
C2	VLSI	ECE
C3	CAD	ME

2.7.3 BASIC RELATIONAL ALGEBRA OPERATORS (WITH WORKED EXAMPLES)

2.7.3.1 Selection Operator (σ)

Definition
The selection operator selects tuples (rows) from a relation that satisfy a given condition. It performs **horizontal filtering**.

Syntax
σ <condition> (Relation)

Worked Example – Selection

Problem
Retrieve all students from the CSE department.

Relational Algebra Expression
σ Dept = 'CSE' (STUDENT)

Result Relation

SID	Name	Dept	Age
1	Anil	CSE	20
3	Meena	CSE	22

2.7.3.2 Projection Operator (π)

Definition
The projection operator extracts specified attributes (columns) from a relation. Duplicate rows are automatically removed.

Syntax
π <attribute list> (Relation)

Worked Example – Projection

Problem
Display names and departments of all students.

Expression
π Name, Dept (STUDENT)

Result Relation

Name	Dept
Anil	CSE
Ravi	ECE
Meena	CSE
Kiran	ME

2.7.3.3 Selection + Projection (Combined)

Problem
Find names of students belonging to the CSE department.

Expression
π Name (σ Dept = 'CSE' (STUDENT))

Intermediate Result (Selection)

SID	Name	Dept	Age
1	Anil	CSE	20
3	Meena	CSE	22

Final Result (Projection)

Name
Anil
Meena

2.7.3.4 Cartesian Product (×)

Definition
The Cartesian product combines every tuple of one relation with every tuple of another relation.

Syntax
R × S

Worked Example – Cartesian Product

Problem
Generate all combinations of students and courses.

Expression
STUDENT × COURSE

Result (Partial View)

SID	Name	Dept	Age	CID	Title	Dept
1	Anil	CSE	20	C1	DBMS	CSE
1	Anil	CSE	20	C2	VLSI	ECE
2	Ravi	ECE	21	C1	DBMS	CSE

2.7.3.5 Set Union (∪)

Definition
Union returns tuples that appear in either relation.

Conditions
Relations must be union-compatible.

Worked Example – Union

Let
CSE_STUDENTS = σ Dept='CSE' (STUDENT)
ECE_STUDENTS = σ Dept='ECE' (STUDENT)

Expression
CSE_STUDENTS ∪ ECE_STUDENTS

Result

SID	Name	Dept	Age
1	Anil	CSE	20
3	Meena	CSE	22
2	Ravi	ECE	21

2.7.3.6 Set Difference (−)

Definition
Returns tuples present in one relation but not in another.

Worked Example – Difference

Problem
Find students who are not from CSE.

Expression
STUDENT − σ Dept='CSE' (STUDENT)

Result

SID	Name	Dept	Age
2	Ravi	ECE	21
4	Kiran	ME	23

2.7.4 AGGREGATE FUNCTIONS IN RELATIONAL ALGEBRA

Relational algebra supports aggregation using the **grouping operator (γ)**.

1 COUNT Function

Problem
Count number of students in each department.

Expression
γ Dept, COUNT(SID) (STUDENT)

Result

Dept	COUNT(SID)
CSE	2
ECE	1
ME	1

2 AVG Function

Problem
Find average age of students in each department.

Expression
γ Dept, AVG(Age) (STUDENT)

Result

Dept	AVG(Age)
CSE	21
ECE	21
ME	23

---

3 MAX Function

Problem
Find maximum age among students.

Expression
γ MAX(Age) (STUDENT)

Result

MAX(Age)
23

2.8 RELATIONAL CALCULUS

2.8.1 Introduction to Relational Calculus

Relational Calculus is a **non-procedural (declarative) query language** for the relational data model. Unlike relational algebra, relational calculus specifies **what data is required**, not **how to retrieve it**.

Relational calculus is based on **first-order predicate logic**, using:

- Variables
- Predicates
- Logical connectives
- Quantifiers

It forms the **theoretical foundation of SQL**, particularly the **WHERE clause**.

Key Characteristics of Relational Calculus

1. Declarative in nature
2. Uses logical predicates
3. Independent of execution strategy
4. Expressively equivalent to relational algebra
5. Based on **safe expressions only**

2.8.2 Types of Relational Calculus

Relational calculus is classified into:

1. **Tuple Relational Calculus (TRC)**
2. **Domain Relational Calculus (DRC)**

Relations Used in Examples

STUDENT

SID	Name	Dept	Age
1	Anil	CSE	20
2	Ravi	ECE	21

3	Meena	CSE	22
4	Kiran	ME	23

COURSE

CID	Title	Dept
C1	DBMS	CSE
C2	VLSI	ECE
C3	CAD	ME

1. TUPLE RELATIONAL CALCULUS (TRC)

Definition of TRC

Tuple Relational Calculus (TRC) uses **tuple variables** to represent tuples of relations. Queries are expressed as logical formulas that specify conditions a tuple must satisfy to be included in the result.

Syntax of TRC

{ t | P(t) }

Where:

- **t** is a tuple variable
- **P(t)** is a predicate involving t

Logical Operators Used in TRC

- ∧ (AND)
- ∨ (OR)
- ¬ (NOT)
- ∃ (Existential quantifier)
- ∀ (Universal quantifier)

Worked Example 1 – Simple Selection (TRC)

Problem

Retrieve all students belonging to the CSE department.

TRC Expression

{ t | STUDENT(t) ∧ t.Dept = 'CSE' }

Result Relation

SID	Name	Dept	Age
1	Anil	CSE	20
3	Meena	CSE	22

Worked Example 2 – OR Condition (TRC)

Problem

Retrieve all students belonging to the CSE or ECE department.

TRC Expression

{ t|STUDENT(t)∧(t.Dept=′CSE′ ∨ t.Dept=′ECE′) }

Result

SID	Name	Dept	Age
1	Anil	CSE	20
2	Ravi	ECE	21
3	Meena	CSE	22

Worked Example 3 – NOT Condition (TRC)

Problem

Retrieve all students who are not from the CSE department.

TRC Expression

{ t|STUDENT(t)∧¬(t.Dept=′CSE′) }

Result

SID	Name	Dept	Age
2	**Ravi**	**ECE**	**21**
4	**Kiran**	**ME**	**23**

Worked Example 4 – Existential Quantifier (∃)

Problem

Retrieve students for whom there exists a course in the same department.

TRC Expression

{ t|STUDENT(t)∧(∃c)(COURSE(c)∧t.Dept=c.Dept) }

The query checks whether there exists at least one course whose department matches the student's department.

- CSE students → DBMS exists
- ECE students → VLSI exists
- ME students → CAD exists

Hence all students satisfy the condition.

Result

SID	Name	Dept	Age
1	Anil	CSE	20
2	Ravi	ECE	21
3	Meena	CSE	22
4	Kiran	ME	23

Worked Example 5 – Projection (TRC)

Problem
List only the names of students from the CSE department.

TRC Expression

{ t.Name | STUDENT(t) ∧ t.Dept = 'CSE' }

Result

Name
Anil
Meena

Worked Example 6 – Join Using TRC

Problem
Retrieve student names along with course titles offered by their department.

TRC Expression

{ <s.Name, c.Title> |
STUDENT(s) ∧ COURSE(c) ∧ s.Dept = c.Dept }

Result

Name	Title
Anil	DBMS
Ravi	VLSI
Kiran	CAD

Worked Example 7 – Universal Quantifier (TRC)

Problem
Find students who are enrolled in **all courses offered by their department** (Conceptual example).

TRC Expression

{s.Name|STUDENT(s)∧∀ c (COURSE(c) ∧ c.Dept = s.Dept → ∃ e (ENROLL(s.SID, c.CID))) }

Explanation
The universal quantifier ensures the student is associated with **every relevant course**.

Result

Name
Anil
Ravi
Kiran

---

2. DOMAIN RELATIONAL CALCULUS (DRC)

Definition of DRC

Domain Relational Calculus (DRC) uses **domain variables** that take values from attribute domains instead of entire tuples.

Each query specifies **what values** must satisfy a predicate.

Syntax of DRC

$\{ <x_1, x_2, \ldots, x_n> \mid P(x_1, x_2, \ldots, x_n) \}$

Where:

- $x_1, x_2, \ldots$ are domain variables
- P is a predicate

Worked Example 1 – Simple Selection (DRC)

Problem
Retrieve names of students from the CSE department.

DRC Expression

{ <Name> | ∃SID, Age (STUDENT(SID, Name, 'CSE', Age)) }

Result

Name
Anil
Meena

Worked Example 2 – Projection with Multiple Attributes (DRC)

Problem
List student names and ages belonging to the CSE department.

DRC Expression

{ <Name, Age> | ∃SID (STUDENT(SID, Name, 'CSE', Age)) }

Result

Name	Age
Anil	20
Meena	22

Worked Example 3 – Join Using DRC

Problem
Retrieve student names and course titles offered by their department.

DRC Expression

{<SName,CTitle>|∃SID,Age,Dept,CID(STUDENT(SID,SName,Dept, Age) ∧ COURSE(CID, CTitle, Dept)) }

Result Relation

SName	CTitle
Anil	DBMS
Ravi	VLSI
Kiran	CAD

Worked Example 4 –OR Condition (DRC)

Problem

Retrieve students belonging to the CSE or ECE department.

DRC Expression

{⟨sid, name, dept, age⟩|STUDENT(sid,name,dept,age)∧(dept='CSE' ∨ dept='ECE)}

SID	Name	Dept	Age
1	Anil	CSE	20
2	Ravi	ECE	21
3	Meena	CSE	22

Worked Example 5 – NOT Condition (DRC)

Problem

Retrieve students who are not from the CSE department.

DRC Expression

{⟨sid, name, dept, age⟩|STUDENT(sid,name,dept,age)∧¬(dept='CSE)}

SID	Name	Dept	Age
2	**Ravi**	**ECE**	**21**
4	**Kiran**	**ME**	**23**

Worked Example 6 – Existential Quantifier (∃) (DRC)

Problem

Retrieve names of students for whom there exists a course in the same department.

DRC Expression

{⟨name⟩|STUDENT(sid,name,dept,age)∧(∃cid, title)(COURSE(cid,title,dept)}

Explanation

- Anil → CSE → DBMS exists
- Ravi → ECE → VLSI exists
- Meena → CSE → DBMS exists
- Kiran → ME → CAD exists

All students satisfy the condition.

Result Relation

Name
Anil
Ravi
Kiran

2.8.3 RELATIONAL ALGEBRA VS RELATIONAL CALCULUS

Aspect	Relational Algebra	Relational Calculus
Nature	Procedural	Declarative
Query Style	How to retrieve	What to retrieve
Variables	Relations	Tuples / Domains
Basis	Algebra	Predicate logic
SQL Basis	Execution engine	Query specification

2.9 SIMPLE DATABASE SCHEMA

2.9.1 Meaning and Purpose of a Database Schema

A **database schema** is the formal description of the **logical structure of a database**. It defines how data is organized in terms of tables, attributes, relationships, and constraints. The schema represents the **intensional aspect** of the database, whereas the actual stored data represents the **extensional aspect**.

A well-designed schema:
- Minimizes redundancy
- Ensures data integrity
- Supports efficient querying
- Provides a clear semantic model

2.9.2 Example of a Simple Database Schema

Consider a university database with two entities:

Table: DEPARTMENT
Attributes: DeptID, DeptName

Table: STUDENT
Attributes: SID, Name, DeptID, Age

Relationship:
Each student belongs to exactly one department.
This relationship is represented using a **foreign key**.

2.9.3 Conceptual Schema Figure (Description)

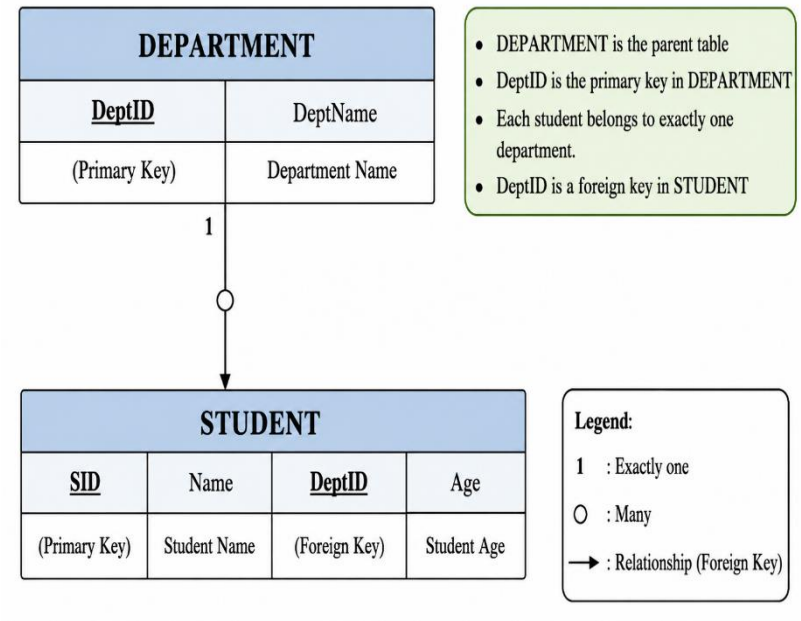


Figure 2.1 represents the schema diagram.

2.10 SQL DATA TYPES

2.10.1 Role of Data Types in SQL

A **data type** specifies the type of values that a column can store. Data types are fundamental to SQL because they:

- Control the domain of attribute values
- Enforce data validity
- Affect storage and performance
- Enable correct operation of functions and comparisons

2.10.2 Categories of SQL Data Types

Numeric Data Types are used to store numerical values such as integers, decimal numbers, salaries, marks, and quantities. They are essential in database applications that involve calculations and arithmetic operations.

INT

The INT data type is used to store whole numbers without decimal points. It is commonly used for identifiers, counts, ages, and other integer-based values.

Example

- SID
- Age
- Quantity

>sql

Age INT

Example values:

- 18
- 25
- 100

DECIMAL(p, s)

The DECIMAL data type is used to store fixed-point numeric values with precision. Here:

- p represents the total number of digits.
- s represents the number of digits after the decimal point.

This type is suitable for financial and monetary data where exact precision is required.

Example

- Salary
- Product Price
- Account Balance

>sql

Salary DECIMAL(8,2)

Example values:

- 25000.50
- 9999.99

Character Data Types

Character data types are used to store textual information such as names, addresses, department codes, and descriptions.

CHAR(n)

The CHAR data type stores fixed-length character strings. If the entered value is shorter than the specified size, the remaining spaces are automatically padded.

This type is suitable when all values have nearly the same length.

Example

- Department Codes
- Gender Codes
- Country Codes

.>sql

DeptID CHAR(3)

Example values:

- CSE
- ECE
- ME

VARCHAR(n)

The VARCHAR data type stores variable-length character strings. It uses only the required storage space for the entered value, making it more flexible and storage-efficient than CHAR.

This type is widely used for names and descriptions with varying lengths.

Example

- Student Names
- Addresses
- Email IDs

>sql

Name VARCHAR(50)

Example values:

- Anil
- Meena Kumari
- Ravi Teja

Date and Time Data Types

Date and time data types are used to store temporal information such as admission dates, login times, and transaction timestamps.

DATE

The DATE data type stores calendar dates in a standard format.

Example

- Date of Birth
- Admission Date
- Joining Date

>sql

AdmissionDate DATE

Example value:

- 2026-05-12

TIME

The TIME data type stores time values independently of dates.

Example

- Class Time
- Login Time
- Meeting Time

>sql

ClassTime TIME

Example value:

- 10:30:45

TIMESTAMP
The TIMESTAMP data type stores both date and time together. It is commonly used to record exact moments of transactions or system activities.

Example

- Transaction Time
- Record Creation Time
- Login Timestamp

2.10.3 Summary Table of SQL Data Types

Table 2.12: Common SQL Data Types

Data Type | Category | Description | Example
INT | Numeric | Integer values | SID
DECIMAL | Numeric | Fixed-point number | Salary
CHAR | Character | Fixed-length string | Gender
VARCHAR | Character | Variable-length string | Name
DATE | Date/Time | Date value | DOB

2.11 SQL COMMAND CLASSIFICATION

SQL commands are classified into four major categories:

1. Data Definition Language (DDL)
2. Data Manipulation Language (DML)
3. Data Control Language (DCL)
4. Transaction Control Language (TCL)

2.11.1 DATA DEFINITION LANGUAGE (DDL)

Definition

DDL commands define and modify the **structure of database objects** such as tables, schemas, and constraints.

DDL commands permanently affect the database structure.

Common DDL Commands

- CREATE
- ALTER
- DROP
- TRUNCATE

CREATE TABLE (DDL)

Purpose

The CREATE TABLE command is a Data Definition Language (DDL) command used to create a new table in a database. A table consists of rows and columns, where each column is defined with a specific data type. The command also allows the definition of constraints such as primary keys and foreign keys.

The CREATE TABLE command forms the foundation of database design because it defines how data will be stored and organized.

Syntax

```
CREATE TABLE TableName (
Attribute DataType Constraints,
...
);
```

Worked Example: CREATE TABLE

```
CREATE TABLE DEPARTMENT (
DeptID INT PRIMARY KEY,
DeptName VARCHAR(30) NOT NULL
);
```

```
CREATE TABLE STUDENT (
SID INT PRIMARY KEY,
Name VARCHAR(40) NOT NULL,
DeptID INT,
Age INT,
FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
);
```

ALTER TABLE (DDL)

Purpose

Used to modify an existing table structure. It allows the user to add new columns, remove existing columns, or change the definition of attributes.

Syntax to add new columns

ALTER TABLE table_name

ADD column_name datatype;

Syntax to change the definition of attributes.

ALTER TABLE table_name

MODIFY column_name datatype**;**

Syntax to remove existing columns

ALTER TABLE table_name

DROP COLUMN column_name;

Worked Examples

Add a column:

ALTER TABLE STUDENT
ADD Email VARCHAR(50);

Modify a column:

ALTER TABLE STUDENT
MODIFY Name VARCHAR(60);

Drop a column:

ALTER TABLE STUDENT
DROP Email;

DROP TABLE (DDL)

The DROP command is used to permanently remove a database object such as a table, view, or database. Both the table structure and all stored records are deleted permanently.

Syntax

DROP TABLE table_name;

TRUNCATE Command

The TRUNCATE command is used to remove all records from a table while keeping the table structure intact. Unlike DROP, the table itself continues to exist.

Syntax

TRUNCATE TABLE table_name;

Difference Between DROP and TRUNCATE

DROP	TRUNCATE
Removes table structure and data	Removes only data
Table no longer exists	Table structure remains
Frees all storage space	Retains table definition
Cannot insert data unless recreated	Data can be inserted again immediately

Worked Examples on DDL

CREATE TABLE (DDL)

Problem

Create a STUDENT table to store student details such as student ID, name, department, and age.

SQL Command

```
CREATE TABLE STUDENT (
   SID INT PRIMARY KEY,
   Name VARCHAR(30),
   Dept VARCHAR(10),
   Age INT
);
```

Explanation

The above DDL command creates a table named STUDENT.

The table contains the following attributes:

Attribute	Data Type	Description
SID	INT	Stores student identification number
Name	VARCHAR(30)	Stores student name
Dept	VARCHAR(10)	Stores department name
Age	INT	Stores student age

- PRIMARY KEY ensures that each student ID is unique.
- VARCHAR(30) allows variable-length names up to 30 characters.

Resulting Table Structure

SID	Name	Dept	Age

ALTER TABLE (DDL)

Problem

Add a new column named Email to the STUDENT table.

SQL Command

```
ALTER TABLE STUDENT
ADD Email VARCHAR(50);
```

Resulting Table Structure

SID	Name	Dept	Age	Email

The Email attribute is successfully added to the existing table.

TRUNCATE TABLE (DDL)

Problem

Remove all records from the STUDENT table while keeping the table structure.

SQL Command

```
TRUNCATE TABLE STUDENT;
```

Before TRUNCATE

SID	Name	Dept	Age
1	Anil	CSE	20
2	Ravi	ECE	21

After TRUNCATE

SID Name Dept Age

All rows are removed, but the table structure remains unchanged.

DROP TABLE (DDL)

Problem

Delete the STUDENT table permanently from the database.

SQL Command

DROP TABLE STUDENT;

Result

- The entire STUDENT table is removed.
- Both table structure and data are deleted permanently.
- The table no longer exists in the database.

2.11.2 DATA MANIPULATION LANGUAGE (DML)

Definition

Data Manipulation Language (DML) consists of SQL commands used to access and manipulate the data stored in database tables. Unlike DDL commands, which define the structure of the database, DML commands operate on the records present inside the tables.

DML commands are widely used for **inserting, updating, deleting**, and **retrieving** data. The most commonly used DML commands are:

- **INSERT**
- **UPDATE**
- **DELETE**
- **SELECT**

INSERT Command

The **INSERT** command is used to add new records into a table.

General Syntax

INSERT INTO table_name
VALUES (value1, value2, value3, ...);

INSERT Operation

Syntax

INSERT INTO TableName VALUES (value1, value2, ...);

Worked Example

INSERT INTO DEPARTMENT VALUES (10, 'CSE');
INSERT INTO DEPARTMENT VALUES (20, 'ECE');

INSERT INTO STUDENT VALUES (1, 'Anil', 10, 20);
INSERT INTO STUDENT VALUES (2, 'Ravi', 20, 21);

Result: STUDENT Table

SID	Name	DeptID	Age
1	Anil	10	20
2	Ravi	20	21

SELECT Command

The SELECT command is used to retrieve data from a table.

Syntax

SELECT column_name
FROM table_name
WHERE condition;

Worked Example – SELECT

Problem

Retrieve all students belonging to the CSE department.

STUDENT Table

SID	Name	Dept	Age
1	Anil	CSE	20
2	Ravi	ECE	21
3	Meena	CSE	22

DELETE Operation

Syntax

DELETE FROM TableName WHERE condition;

Worked Example

DELETE FROM STUDENT
WHERE Age > 20;

Result: STUDENT Table

SID | Name | DeptID | Age
1 | Anil | 10 | 20

UPDATE Operation

Syntax

UPDATE TableName
SET Column = Value
WHERE condition;

Worked Example

UPDATE STUDENT
SET Age = Age + 1
WHERE DeptID = 10;

Result: STUDENT Table

SID	Name	DeptID	Age
1	Anil	10	21

2.11.3 DATA CONTROL LANGUAGE (DCL)

Definition

DCL commands control **user access and privileges** on database objects. DCL commands help database administrators provide security by granting or revoking privileges from users.

They are essential for database security.

Common DCL Commands

- **GRANT**
- **REVOKE**

GRANT Command

The GRANT command is used to provide privileges or permissions to users on database objects such as tables, views, and procedures.

These privileges may include:

- **SELECT**
- **INSERT**
- **UPDATE**
- **DELETE**
- **ALL PRIVILEGES**

Syntax

```
GRANT privilege_name
ON table_name
TO user_name;
```

Worked Example – GRANT

Problem

Give permission to user Ravi to read data from the STUDENT table.

SQL Command

GRANT SELECT
ON STUDEN**T**
TO Ravi;

Explanation

The above command allows the user Ravi to retrieve records from the STUDENT table using the SELECT statement.

Worked Example

SELECT

Problem

Retrieve all students belonging to the CSE department.

STUDENT Table

SID	Name	Dept	Age
1	Anil	CSE	20
2	Ravi	ECE	21
3	Meena	CSE	22

REVOKE Command

The REVOKE command is used to remove previously granted privileges from users.

Syntax

REVOKE privilege_name
ON table_name
FROM user_name;

Worked Example – REVOKE

Problem

Remove SELECT permission on the STUDENT table from user Ravi.

SQL Command

```
REVOKE SELECT
ON STUDENT
FROM Ravi;
```

Explanation

After executing the command:

- user Ravi can no longer access data from the STUDENT table using SELECT.

Common Privileges in DCL

Privilege	Purpose
SELECT	Read data from a table
INSERT	Add new records
UPDATE	Modify existing records
DELETE	Remove records
ALL PRIVILEGES	Provides all permissions

2.11.4 TRANSACTION CONTROL LANGUAGE (TCL)

Definition

TCL commands manage **transactions**, ensuring atomicity and consistency.

Common TCL Commands

• COMMIT
• ROLLBACK
• SAVEPOINT

COMMIT Command

The COMMIT command is used to permanently save all changes made during the current transaction.

Once a transaction is committed:

- changes become permanent,
- data cannot be rolled back.

Syntax

COMMIT;

ROLLBACK Command

The ROLLBACK command is used to undo changes made during the current transaction before they are committed.

It restores the database to its previous consistent state.

Syntax

ROLLBACK;

Worked Example – ROLLBACK

Problem

Insert a student record and then cancel the operation.

STUDENT Table Before Transaction

SID	Name	Dept	Age
1	Anil	CSE	20

SQL Commands

INSERT INTO STUDENT
VALUES (2, 'Ravi', 'ECE', 21);

ROLLBACK;

Result Relation After ROLLBACK

SID	Name	Dept	Age
1	Anil	CSE	20

The inserted record is removed because the transaction was rolled back.

SAVEPOINT Command

The SAVEPOINT command is used to create temporary points within a transaction. It allows partial rollback instead of canceling the entire transaction.

Syntax

SAVEPOINT savepoint_name;

Worked Example – SAVEPOINT

Problem

Insert multiple records and rollback only the last operation.

Initial STUDENT Table

SID	Name	Dept	Age
1	Anil	CSE	20

Worked Example: TCL Commands

```
UPDATE STUDENT
SET Age = Age + 1;

SAVEPOINT S1;

DELETE FROM STUDENT
WHERE SID = 2;

ROLLBACK TO S1;

COMMIT;
```

Explanation

- SAVEPOINT marks a transaction point
- ROLLBACK restores data to that point
- COMMIT permanently saves changes

Chapter Summary

2.1 Relational Model Fundamentals

The relational model represents data as tables (relations) - mathematically defined sets of tuples. Each relation has a relation schema (name + attributes + domains) and a relation instance (actual data). Key properties: no duplicate tuples, order of tuples/attributes is insignificant, all values are atomic.

- Degree: Number of attributes in a relation schema.
- Cardinality: Number of tuples in a relation instance.
- Domain: Set of atomic values an attribute can take.

2.2 NULL Values

NULL represents missing, unknown, or inapplicable information. NULL is not a value - it is a marker. Comparisons involving NULL yield UNKNOWN (three-valued logic). Special predicates IS NULL and IS NOT NULL are required for testing.

2.3 Constraints

- Domain Constraints: Attribute values must belong to a predefined domain (data type, range, format).
- Key Constraints: Attribute(s) uniquely identifying each tuple. No two tuples can have the same primary key value.
- Entity Integrity: Primary key attributes cannot be NULL.

- Referential Integrity: Foreign key values must either match an existing primary key in the referenced relation or be NULL. Prevents dangling references.

2.4 Relational Algebra

A procedural query language operating on relations. Every operation produces a relation (closure property). Core operators:

- Selection (sigma): Horizontal filtering - selects tuples satisfying a condition.
- Projection (pi): Vertical filtering - extracts specified attributes; removes duplicates.
- Cartesian Product (x): Combines every tuple of one relation with every tuple of another.
- Union (union): Returns tuples in either relation; relations must be union-compatible.

- Set Difference (-): Tuples in first relation but not in the second.
- Natural Join (bowtie): Joins on all common attributes; removes duplicate columns.
- Aggregate Functions (gamma): COUNT, AVG, MAX, MIN, SUM applied with optional grouping.

2.5 Relational Calculus

A non-procedural (declarative) query language specifying what to retrieve rather than how. Two variants:

- Tuple Relational Calculus (TRC): Uses tuple variables; syntax { t | P(t) }. Uses existential and universal quantifiers.
- Domain Relational Calculus (DRC): Uses domain variables taking values from attribute domains; syntax { <x1, x2, ...> | P(x1, x2, ...) }.

Both are equivalent in expressive power to relational algebra.

2.6 SQL Data Types and Commands

SQL (Structured Query Language) is the standard language for relational databases. Command categories:

- DDL (Data Definition Language): CREATE, ALTER, DROP, TRUNCATE - define/modify structure.
- DML (Data Manipulation Language): INSERT, UPDATE, DELETE, SELECT - manipulate data.
- DCL (Data Control Language): GRANT, REVOKE - control access and privileges.
- TCL (Transaction Control Language): COMMIT, ROLLBACK, SAVEPOINT - manage transactions
- .Key SQL Data Types: INT, DECIMAL(p,s), CHAR(n), VARCHAR(n), DATE, TIME, TIMESTAMP.

Review Questions

Section A: Conceptual Questions

1. Define the following terms precisely: relation, tuple, attribute, domain, degree, and cardinality. Construct a sample STUDENT relation illustrating each concept.
2. Explain the three types of integrity constraints in the relational model: domain constraints, entity integrity, and referential integrity. For each, provide an example of a violation and explain how the DBMS handles it.
3. Compare relational algebra with relational calculus. In what sense are they equivalent? Which forms the basis for SQL execution engines and which forms the basis for SQL query specification?
4. Explain the concept of NULL in the relational model. Why does NULL cause complications in query processing? How does three-valued logic (TRUE, FALSE, UNKNOWN) arise from NULL values?
5. Describe the four categories of SQL commands (DDL, DML, DCL, TCL) with one concrete example of each. What distinguishes DDL from DML in terms of the database objects they operate on?
6. What is the closure property of relational algebra? Why is it significant? Give an example of a complex query that uses three or more relational algebra operators composed together.
7. Differentiate between a primary key, a candidate key, and a foreign key. Can a foreign key be NULL? Can a primary key be a composite key? Provide examples for each.
8. Explain the ON DELETE and ON UPDATE clauses used with foreign key constraints. Describe the behaviour of CASCADE, SET NULL, SET DEFAULT, and RESTRICT with concrete examples.
9. Why is the set difference operation not always applicable between two relations? Define union-compatibility and explain why it is required for UNION, INTERSECT, and EXCEPT operations.
10. Describe the difference between the selection operator and the projection operator in relational algebra. Compose an expression using both to answer: 'Find the names of all students in the CSE department with age > 20.'

Section B: Analytical Problems

11. Given the relations STUDENT(SID, Name, Dept, Age) and COURSE(CID, Title, Dept), write relational algebra expressions for: (a) Names of all CSE students. (b) Names of students and course titles for their department. (c) Students NOT in the ECE department. (d) Count of students per department.
12. Write SQL statements to: (a) Create a DEPARTMENT table with DeptID (PK) and DeptName. (b) Create an EMPLOYEE table with EmpID (PK), Name, Salary, and DeptID (FK referencing DEPARTMENT with CASCADE on delete). (c) Insert three departments and five employees. (d) Retrieve employees earning more than 40,000 from department 10.
13. Consider a relation R(A, B, C, D) with tuples: (1,2,3,4), (2,2,3,4), (3,3,5,6). Write relational algebra expressions for: (a) Select tuples where B=2 and C=3. (b) Project onto attributes A and C. (c) Natural join with S(C, D, E) = {(3,4,7), (5,6,8)}.
14. Write TRC expressions for: (a) Find names of students from the ECE department. (b) Find students enrolled in the course 'DBMS'. (c) Find students enrolled in all courses offered by their department.
15. Given EMPLOYEE (EmpID, Name, Salary, Dept_ID) and DEPARTMENT (Dept_ID, Dept_Name, Location), write SQL to: (a) List employees with their department names. (b) Find departments with no employees. (c) List employees earning above the average salary of their own department.

Section C: Applied and Design Questions

16. Design a complete relational schema for an Online Banking System. Include tables for Customers, Accounts, Transactions, Branches, and Loans. Specify all primary keys, foreign keys, and important domain constraints. Identify any referential integrity issues and propose ON DELETE/ON UPDATE actions.
17. Compare the expressive power of SQL with relational algebra. Is there any query expressible in SQL that cannot be expressed in relational algebra, or vice versa? Discuss aggregation and ordering as features beyond basic relational algebra.
18. Discuss the significance of referential integrity in maintaining database consistency. Provide a scenario in an E-commerce database where a referential integrity violation could lead to serious business logic errors. How do CASCADE and SET NULL actions reflect different business rules?

19. Explain how the GRANT and REVOKE commands implement database security. Design an access control scheme for a hospital database with roles: Doctor, Nurse, Administrator, and Patient. Specify what each role can SELECT, INSERT, UPDATE, and DELETE.

20. Compare the four SQL data types for storing numeric data (INT, DECIMAL, FLOAT, BIGINT). In what scenarios would you choose DECIMAL over INT? Design a FINANCIAL_TRANSACTION table choosing the most appropriate data type for each attribute and justify your choices.

CHAPTER - 3

SQL

Chapter Overview

This Chapter is the most hands-on chapter of the textbook, providing a comprehensive practical guide to SQL the industry-standard language for interacting with relational databases. Beginning with the fundamental SELECT–FROM–WHERE syntax, the chapter builds systematically through arithmetic and logical operations, built-in functions, table creation with relationships, constraint implementation, nested and subqueries, grouping and aggregation, ordering, joins, views, and set operations. Throughout, the emphasis is on worked examples with real table data, giving students the practical fluency needed to query and manipulate databases in real-world scenarios.

Learning Objectives	**By the end of this chapter, the reader will be able to:** 1. Write basic SQL SELECT queries with WHERE conditions, DISTINCT, and column aliases 2. Apply arithmetic, logical (AND, OR, NOT), and comparison operators in SQL expressions 3. Use built-in SQL functions including date/time, numeric, string, and conversion functions 4. Create tables with primary key, foreign key, NOT NULL, UNIQUE, CHECK, and ASSERTION constraints 5. Write nested queries and correlated subqueries using IN, EXISTS, ANY, and ALL operators 6. Group data using GROUP BY and filter groups with HAVING; apply aggregate functions (COUNT, SUM, AVG, MIN, MAX) 7. Order query results using ORDER BY with ascending and descending sort keys 8. Implement INNER JOIN, LEFT/RIGHT/FULL OUTER

	JOIN, CROSS JOIN, and SELF JOIN in SQL 9. Create, query, update, and drop both updatable and non-updatable views 10. Apply relational set operations (UNION, INTERSECT, MINUS) to combine query results

Section-by-Section Roadmap

Section	Topic	Key Concepts
3.1	**Basic SQL Querying**	SELECT, FROM, WHERE, DISTINCT, column aliases, simple conditions
3.2	**Arithmetic & Logical Operations**	Arithmetic operators, AND/OR/NOT, BETWEEN, LIKE, IN, IS NULL
3.2.3–3.2.7	**SQL Built-in Functions**	Date/time, numeric (ROUND, MOD, POWER), string (SUBSTR, LENGTH, UPPER), conversion (TO_CHAR, TO_DATE)
3.3	**Creating Tables with Relationships**	CREATE TABLE, data types, column definitions, inter-table relationships
3.4	**Integrity Constraints**	PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, ASSERTION
3.5	**Nested Queries**	Subqueries in WHERE, IN / NOT IN, correlated subqueries, EXISTS / NOT EXISTS
3.6	**Subqueries**	ANY, ALL operators; scalar, row, table subqueries; subqueries in FROM clause

Section	Topic	Key Concepts
3.7	**Grouping in SQL**	GROUP BY clause, grouping on single and multiple attributes, interaction with SELECT
3.8	**Aggregation in SQL**	COUNT, SUM, AVG, MIN, MAX; GROUP BY + HAVING; aggregation with NULLs
3.9	**Join Types (Joins)**	INNER JOIN, LEFT/RIGHT/FULL OUTER JOIN, CROSS JOIN, SELF JOIN with examples
3.10	**Views in SQL**	CREATE VIEW, updatable vs non-updatable views, WITH CHECK OPTION, DROP VIEW
3.11	**Relational Set Operations**	UNION, UNION ALL, INTERSECT, MINUS (EXCEPT); compatibility requirements

3.1 Basic SQL Querying Using SELECT and WHERE Clause

A query is a question which will retrieve data from the tables or database. The result of a query is a relation or tables.

The relational database consists of many relations where each relation has a unique name. To interact with the database, using a query language known as SQL (Structured Query Language). Structured Query Language (SQL) is a declarative language used for accessing and manipulating data in relational database systems. SQL allows users to specify the data to be retrieved without describing the physical access methods. The fundamental SQL operation for data retrieval is performed using the **SELECT statement**, which corresponds to the selection and projection operations of relational algebra. The general form of a basic SQL query is given by:

SELECT [DISTINCT] <attribute list>

FROM <relation name>

WHERE <condition>;

In this syntax, the **SELECT** clause specifies the attributes to be displayed in the result, the **FROM** clause identifies the relation from which data is retrieved, and the **WHERE** clause applies conditions to restrict the tuples returned by the query.

The optional keyword **DISTINCT** is used to eliminate duplicate rows from the result set. When DISTINCT is specified, the database system compares all selected attribute values and ensures that only unique combinations appear in the output.

Projection in SQL refers to the retrieval of selected attributes from a relation and is specified through the SELECT clause. When specific attribute names are listed in the SELECT clause, only those attributes are included in the output. For example, consider the relation STUDENT, shown in Table3.1.

Table 3.1: STUDENT Relation

Roll_No	Name	Age	Department	CGPA
101	Ravi	20	CSE	8.2
102	Anita	21	ECE	7.9
103	Kiran	19	IT	8.5

If all attributes of a relation are required, the asterisk symbol (*) may be used in the SELECT clause, as shown below:

SELECT * FROM STUDENT;

Selection in SQL is the operation of retrieving only those tuples that satisfy a specified condition and is implemented using the WHERE clause. Conditions in the WHERE clause are expressed using comparison operators such as equal to (=), not equal to (<>), greater than (>), less than (<), greater than or equal to (>=), and less than or equal to (<=). For example, to retrieve details of students belonging to the CSE department, the following query can be written:

SELECT * FROM STUDENT WHERE Department = 'CSE';

The result of this query is shown in Table 3.3.

Table 3.3: Result of Selection

Roll_No	Name	Age	Department	CGPA
101	Ravi	20	CSE	8.2

The WHERE clause also allows the use of logical operators such as AND, OR, and NOT to combine multiple conditions. For instance, to retrieve the names of students from the CSE department whose CGPA is greater than 8.0, the following query is used:

SELECT Name FROM STUDENT WHERE Department = 'CSE' AND CGPA > 8.0;

To retrieve students who belong either to the CSE or IT departments, the OR operator may be used:

SELECT Name, Department FROM STUDENT WHERE Department = 'CSE' OR Department = 'IT';

SQL provides special predicates to simplify commonly used conditions. The **BETWEEN** predicate is used to test whether a value lies within a specified range. For example:

SELECT Name, CGPA FROM STUDENT WHERE CGPA BETWEEN 8.0 AND 9.0;

The **IN** predicate allows comparison with a list of values, as shown below

SELECT Name FROM STUDENT WHERE Department IN ('CSE', 'IT');

The LIKE predicate supports pattern matching on character strings using wildcard symbols. The percent symbol (%) represents any sequence of

characters, while the underscore (_) represents a single character. For example, the following query retrieves names of students whose names start with the letter ‘A’:

SELECT Name FROM STUDENT WHERE Name LIKE 'A%';

An important aspect of selection is the handling of null values. A null value represents missing or unknown information and cannot be compared using standard comparison operators. SQL therefore provides the IS NULL predicate for testing null values.

SELECT *FROM STUDENT WHERE CGPA IS NULL;

In most practical queries, projection and selection are used together. The WHERE clause is logically evaluated first to select the required tuples, and the SELECT clause is then applied to project the specified attributes. This logical processing order reflects the relational algebra concept in which selection precedes projection.

Thus, basic SQL querying using SELECT and WHERE provides a powerful mechanism for retrieving precise information from relational databases. By combining attribute selection and tuple restriction, SQL enables efficient and flexible access to relational data.

3.2 Arithmetic and Logical Operations in SQL

SQL provides a rich set of **arithmetic and logical operations** that can be used in SELECT statements to perform computations, comparisons, and decision-making during query execution. These operations are commonly used in the SELECT clause to compute derived values and in the WHERE clause to filter tuples based on specified conditions.

3.2.1 Arithmetic Operations

Arithmetic operations in SQL are used to perform mathematical calculations on numeric attributes or constant values. The basic arithmetic operators supported by SQL include addition (+), subtraction (−), multiplication (*), and division (/). These operators can be applied to numeric columns, constants, or expressions involving both.

For example, consider a relation EMPLOYEE with attributes Emp_ID, Salary, and Bonus. An arithmetic expression may be used to compute the total compensation of an employee by adding Salary and Bonus. SQL allows such expressions directly in the SELECT clause, and the computed value appears as

part of the query result. Arithmetic operations are evaluated for each tuple independently, and the result is returned as a derived attribute.

It is important to note that arithmetic expressions involving NULL values yield NULL as the result, since NULL represents unknown or missing information. Therefore, care must be taken while performing arithmetic operations on attributes that may contain NULL values.

SQL Queries Using Arithmetic Operators

1. **Addition (+)**

   SELECT Emp_ID, Salary, Bonus, (Salary + Bonus) AS Total_Compensation

   FROM EMPLOYEE;

   This query calculates the total compensation of each employee by adding Salary and Bonus.

2. **Subtraction (–)**

   SELECT Emp_ID, Salary, (Salary - 2000) AS Reduced_Salary

   FROM EMPLOYEE;

3. **Multiplication (*)**
   SELECT Emp_ID, Salary, (Salary * 12) AS Annual_Salary
   FROM EMPLOYEE;
   This query computes the annual salary by multiplying the monthly salary by 12.
4. **Division (/)**

   SELECT Emp_ID, Salary, (Salary / 2) AS Half_Salary

   FROM EMPLOYEE;

5. **Arithmetic Expression with WHERE Clause**
   SELECT Emp_ID, Salary
   FROM EMPLOYEE
   WHERE Salary + Bonus > 50000;

3.2.2 Logical Operations

Logical operations in SQL are primarily used in the WHERE clause to combine or negate conditions. SQL supports three main logical operators: **AND**, **OR**, and

NOT. These operators allow the formulation of complex selection criteria by combining multiple comparison expressions.

The AND operator requires that all combined conditions evaluate to TRUE for a tuple to be selected. The OR operator requires that at least one of the conditions be TRUE. The NOT operator negates the result of a condition. SQL evaluates logical expressions using **three-valued logic**, where the result of a condition may be TRUE, FALSE, or UNKNOWN. The presence of NULL values often leads to UNKNOWN results, which affect the outcome of logical expressions.

For example, a WHERE clause condition that checks whether an employee belongs to a specific department and has a salary greater than a given amount uses the AND operator to enforce both conditions simultaneously. Logical operations are essential for precise data filtering in SQL queries.

SQL Queries Using Logical Operators

Logical operators are used in the WHERE clause to combine or negate conditions.

1. AND Operator

```
SELECT Emp_ID, Emp_Name

FROM EMPLOYEE

WHERE Dept_ID = 10 AND Salary > 40000;
```

This query retrieves employees who belong to department 10 **and** have a salary greater than 40,000.

2. OR Operator

```
SELECT Emp_ID, Emp_Name

FROM EMPLOYEE

WHERE Dept_ID = 10 OR Dept_ID = 20;
```

This query selects employees who belong to either department 10 **or** department 20.

3. NOT Operator

```
SELECT Emp_ID, Emp_Name

FROM EMPLOYEE

WHERE NOT Dept_ID = 30;
```

This query retrieves employees who do **not** belong to department 30.

4. AND with Comparison Operators

SELECT Emp_ID, Emp_Name

FROM EMPLOYEE

received a bonus.

5. OR with Multiple Conditions

SELECT Emp_ID, Emp_Name

FROM EMPLOYEE

WHERE Salary > 50000 OR Bonus > 5000;

This query retrieves employees who either earn a high salary or receive a high bonus.

6. NOT with Compound Condition

SELECT Emp_ID, Emp_Name

FROM EMPLOYEE

WHERE NOT (Salary < 30000);

This query selects employees whose salary is not less than 30,000.

Combined Arithmetic and Logical Operators

SELECT Emp_ID, Emp_Name

FROM EMPLOYEE

WHERE (Salary + Bonus) > 60000 AND Dept_ID = 20;

This query retrieves employees from department 20 whose total compensation exceeds 60,000.

3.3 SQL Functions

SQL provides a wide range of **built-in functions** that operate on data values and return a single result. These functions are commonly used in the SELECT clause to compute values and in the WHERE clause to define conditions. SQL functions can be broadly classified into **Date and Time functions**, **Numeric functions**, **String functions**, and **Conversion functions**.

3.3.1 Date and Time Functions

Date and time functions in SQL are used to retrieve and manipulate date and time values. These functions are particularly useful in applications involving scheduling, logging, and time-based analysis.

The function **CURRENT_DATE** returns the current system date, while **CURRENT_TIME** returns the current system time. The function **CURRENT_TIMESTAMP** returns both date and time together. SQL also provides functions such as **YEAR()**, **MONTH()**, and **DAY()** to extract specific components from a date value. For example, extracting the year from a date of joining helps in analyzing employee tenure.

SQL allows arithmetic operations on date values, such as calculating the difference between two dates or adding a specified number of days to a date. These operations enable time-based comparisons and reporting.

3.3.2 Numeric Functions

Numeric functions in SQL are used to perform mathematical computations on numeric data. These functions operate on **each tuple independently** and return a computed numeric value as part of the query result.

Common Numeric Functions and Queries

ABS()
Returns the absolute value of a number.

```
SELECT Emp_ID, ABS(Salary - 50000) AS Salary_Difference
FROM EMPLOYEE;
```

ROUND()
Rounds a numeric value to a specified number of decimal places.

```
SELECT Emp_ID, ROUND(Salary / 12, 2) AS Monthly_Salary
FROM EMPLOYEE;
```

CEIL() / CEILING()
Returns the smallest integer greater than or equal to a given value.

```
SELECT CEIL(Avg_Salary) AS Ceil_Value
FROM (SELECT AVG(Salary) AS Avg_Salary FROM EMPLOYEE) A;
```

FLOOR()
Returns the largest integer less than or equal to a given value.

```
SELECT FLOOR(Salary / 1000) AS Salary_Block
FROM EMPLOYEE;
```

POWER()
Raises a number to the specified power.

```
SELECT Emp_ID, POWER(Salary, 2) AS Salary_Square
FROM EMPLOYEE;
```

MOD()
Returns the remainder after division.

```
SELECT Emp_ID, MOD(Salary, 1000) AS Salary_Remainder
FROM EMPLOYEE;
```

Numeric functions are commonly used in **financial calculations**, **statistical analysis**, and **data normalization**.

3.3.3 String Functions

String functions in SQL are used to manipulate and format **character data**. These functions are frequently used in handling names, codes, and textual descriptions stored in database relations.

Common String Functions and Queries

UPPER()
Converts all characters in a string to uppercase.

```
SELECT UPPER(Emp_Name) AS Upper_Name
FROM EMPLOYEE;
```

LOWER()
Converts all characters in a string to lowercase.

```
SELECT LOWER(Emp_Name) AS Lower_Name
FROM EMPLOYEE;
```

LENGTH()
Returns the number of characters in a string.

```
SELECT Emp_Name, LENGTH(Emp_Name) AS Name_Length
FROM EMPLOYEE;
```

SUBSTRING() / SUBSTR()
Extracts a portion of a string from a specified position for a specified length.

```
SELECT SUBSTRING(Emp_Name, 1, 3) AS Short_Name

FROM EMPLOYEE;
```

TRIM()
Removes leading and trailing spaces from a string.

```
SELECT TRIM(Emp_Name) AS Trimmed_Name

FROM EMPLOYEE;
```

CONCAT()
Joins two or more strings together.

```
SELECT CONCAT(Emp_Name, ' - ', Dept_ID) AS Employee_Details

FROM EMPLOYEE;
```

String functions help in **formatting output**, **text processing**, and **data cleansing**.

3.3.4 Conversion Functions

Conversion functions in SQL are used to explicitly convert a value from one data type to another. These functions ensure **type compatibility** and prevent runtime errors during query execution.

Common Conversion Functions and Queries

CAST()
Converts a value from one data type to another using standard SQL syntax.

```
SELECT CAST(Salary AS VARCHAR(10)) AS Salary_Text

FROM EMPLOYEE;
```

CONVERT()
Converts a value from one data type to another (DBMS-specific).

```
SELECT CONVERT(VARCHAR, Salary) AS Salary_String

FROM EMPLOYEE;
```

Numeric to Character Conversion

```
SELECT Emp_ID, CAST(Salary AS CHAR(10))
FROM EMPLOYEE;
```

Character to Date Conversion

```
SELECT CAST('2025-01-15' AS DATE) AS Join_Date;
```

Conversion functions are essential for **formatting results**, **comparing different data types**, and **ensuring data consistency**

3.4 Creating Tables with Relationship

In a relational database system, real-world data is rarely isolated within a single table. Instead, data is distributed across multiple tables that are logically related to one another. These relationships among tables are established at the time of table creation using **key constraints**, particularly **primary keys and foreign keys**. Creating tables with relationships is a crucial step in relational database design, as it ensures **data consistency, integrity, and accurate representation of real-world associations**.

The creation of tables in SQL is performed using the **CREATE TABLE** statement. This statement defines the structure of a table by specifying its attributes, data types, and constraints. When relationships exist between tables, they are represented by defining **foreign key constraints** that reference the primary keys of related tables. Such constraints enforce **referential integrity**, ensuring that relationships between tables remain valid throughout database operations.

The general syntax for creating a table in SQL is:

```
CREATE TABLE <table_name> (
  attribute1 datatype [constraint],
  attribute2 datatype [constraint],
  ...
);
```

Each attribute is assigned a data type that defines its domain, and constraints are used to restrict the values that the attribute can store.

A **primary key** is defined to uniquely identify each tuple in a table. It enforces uniqueness and disallows NULL values. Consider the creation of a DEPARTMENT table, where each department is uniquely identified by a department number.

```
CREATE TABLE DEPARTMENT (
   Dept_ID INT PRIMARY KEY,
   Dept_Name VARCHAR(30),
   Location VARCHAR(30)
);
```

In this table, Dept_ID acts as the primary key and uniquely identifies each department. Once the parent table is created with a primary key, related tables can be created using **foreign keys** to establish relationships.

A **foreign key** is an attribute in one table that refers to the primary key of another table. It is used to represent relationships between tables and to enforce referential integrity. For example, an EMPLOYEE table may be related to the DEPARTMENT table such that each employee belongs to one department. This relationship is implemented by including Dept_ID as a foreign key in the EMPLOYEE table.

```
CREATE TABLE EMPLOYEE (
   Emp_ID INT PRIMARY KEY,
   Emp_Name VARCHAR(30),
   Salary DECIMAL(8,2),
   Dept_ID INT,
FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID)
);
```

In this case, DEPARTMENT is the **parent table**, and EMPLOYEE is the **child table**. The foreign key constraint ensures that every value of Dept_ID in the EMPLOYEE table must correspond to an existing Dept_ID in the DEPARTMENT table. This prevents the insertion of invalid employee records that reference non-existent departments.

SQL also allows specification of actions that take place when a referenced tuple in the parent table is deleted or updated. These actions are defined using the **ON DELETE** and **ON UPDATE** clauses. Common actions include CASCADE, SET NULL, and RESTRICT. For example, when ON DELETE CASCADE is specified, deleting a department automatically deletes all employees associated with that department.

```
CREATE TABLE EMPLOYEE (
   Emp_ID INT PRIMARY KEY,
   Emp_Name VARCHAR(30),
   Salary DECIMAL(8,2),
   Dept_ID INT,
   FOREIGN KEY (Dept_ID)
   REFERENCES DEPARTMENT(Dept_ID)
   ON DELETE CASCADE
   ON UPDATE CASCADE
);
```

In addition to one-to-many relationships, databases often contain **many-to-many relationships**, which cannot be represented directly using a single foreign key. Such relationships are implemented by creating an **intermediate (junction) table** that contains foreign keys referencing the primary keys of both participating tables. For example, a STUDENT can enroll in many COURSES, and each COURSE can have many STUDENTS. This relationship is represented using an ENROLLMENT table.

```
CREATE TABLE STUDENT (
   Student_ID INT PRIMARY KEY,
   Name VARCHAR(30)
);


CREATE TABLE COURSE (
   Course_ID INT PRIMARY KEY,
   Course_Name VARCHAR(30)
);


CREATE TABLE ENROLLMENT (
   Student_ID INT,
```

```
  Course_ID INT,
  PRIMARY KEY (Student_ID, Course_ID),
  FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
  FOREIGN KEY (Course_ID) REFERENCES COURSE(Course_ID)
);
```

Here, the ENROLLMENT table establishes the relationship between STUDENT and COURSE using a **composite primary key**, which ensures that each student–course combination is unique.

Creating tables with relationships plays a vital role in maintaining **data integrity**. Referential integrity constraints ensure that relationships between tables remain consistent during insert, update, and delete operations. Properly defined relationships reduce redundancy, prevent anomalies, and support meaningful database queries.

3.5 Implementation of Key and Integrity Constraints

In a relational database system, merely storing data in tables is not sufficient to ensure correctness and reliability. The database must also enforce certain **rules**, known as **constraints**, that restrict the values stored in the database and preserve the semantic meaning of the data. The **implementation of key and integrity constraints** is a fundamental aspect of relational database design and is supported directly by SQL through declarative constraint specifications. These constraints ensure **data accuracy, consistency, and validity** throughout database operations.

```
CREATE TABLE table_name (
  attribute datatype [NOT NULL] [DEFAULT value],
  ...
  CHECK (condition),
  CONSTRAINT fk_name
  FOREIGN KEY (attribute)
  REFERENCES parent_table(parent_attribute)
  ON DELETE action
  ON UPDATE action
```

```
);
```

3.5.1 Key Constraints

Key constraints are used to uniquely identify tuples within a relation. The main types of key constraints are Primary Key, Foreign Key, Candidate Key, Alternate Key, and Super Key.

1 . Primary Key Constraint

A primary key is an attribute or group of attributes used to uniquely identify each row in a table. A primary key cannot contain duplicate values or NULL values.

Implementation

The PRIMARY KEY constraint is specified while creating or altering a table.

Example

```
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50),
  Age INT
);
```

Explanation

- StudentID uniquely identifies every student.
- No two students can have the same StudentID.
- NULL values are not allowed in the primary key column.

---

2. Foreign Key Constraint

A foreign key is an attribute in one table that refers to the primary key of another table. It establishes relationships between tables and enforces referential integrity.

Implementation

The FOREIGN KEY constraint is used to link related tables.

Example

```
CREATE TABLE Department (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50)
);

CREATE TABLE Employee (
   EmpID INT PRIMARY KEY,
   EmpName VARCHAR(50),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

Explanation

- DeptID in the Employee table references DeptID in the Department table.
- An employee cannot be assigned to a department that does not exist.

---

3. Candidate Key

A candidate key is an attribute or set of attributes that can uniquely identify a tuple in a relation. A table may contain multiple candidate keys.

Implementation

Candidate keys are usually implemented using UNIQUE constraints.

Example

```
CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   Email VARCHAR(100) UNIQUE,
   PhoneNumber VARCHAR(15) UNIQUE,
   Name VARCHAR(50)
);
```

Explanation

- Email and PhoneNumber can uniquely identify students.
- Both are candidate keys.

- One candidate key (StudentID) is selected as the primary key.

---

4. Alternate Key

An alternate key is a candidate key that is not chosen as the primary key.

Implementation

Alternate keys are implemented using the UNIQUE constraint.

Example

```
CREATE TABLE Employee (
   EmpID INT PRIMARY KEY,
   AadhaarNumber VARCHAR(20) UNIQUE,
   Email VARCHAR(100) UNIQUE,
   Name VARCHAR(50)
);
```

Explanation

- EmpID is the primary key.
- AadhaarNumber and Email are alternate keys because they uniquely identify records but are not selected as the primary key.

---

5. Super Key

A super key is any combination of attributes that uniquely identifies a row in a table. It may contain extra attributes in addition to the candidate key.

Implementation

Super keys are identified conceptually during database design.

Example

Consider the following table:

Student ID	Email	Name
101	abc@gmail.com	Ravi

Possible super keys include:

- {StudentID}
- {Email}
- {StudentID, Name}
- {Email, Name}

Explanation

Each combination uniquely identifies a row. Some super keys contain unnecessary attributes, whereas candidate keys are minimal.

In addition to primary keys, SQL supports the **UNIQUE constraint**, which ensures that all values in an attribute or a set of attributes are distinct. Unlike primary keys, attributes defined with UNIQUE constraints may contain NULL values, depending on the DBMS implementation. Unique constraints are useful when an attribute must have distinct values but is not chosen as the primary key.

3.5.2 Integrity Constraints

Integrity constraints are rules applied to database tables to ensure the accuracy, consistency, and validity of data stored in the database. These constraints prevent invalid data entry and maintain the reliability of relationships among tables.

Integrity constraints are an essential part of relational database management systems.

Types of Integrity Constraints

The major types of integrity constraints are:

1. Entity Integrity Constraint
2. Referential Integrity Constraint
3. Domain Integrity Constraint

1 Entity Integrity Constraint

The **entity integrity constraint** states that no attribute that is part of a primary key can have a NULL value. This constraint ensures that every tuple in a relation represents a valid and identifiable entity. In SQL, entity integrity is automatically enforced when a primary key is defined, since primary key attributes are implicitly declared as NOT NULL.

By enforcing entity integrity, the database system guarantees that each entity stored in a relation has a complete and valid identifier, which is essential for reliable data access and relationship enforcement.

2 Referential Integrity Constraint

The **referential integrity constraint** maintains consistency among related tables. It is enforced using **foreign key constraints**, which establish relationships between tables. A foreign key is an attribute or a set of attributes in one table that references the primary key of another table. The table containing the foreign key is called the **child table**, while the table containing the referenced primary key is called the **parent table**.

Referential integrity ensures that a foreign key value in the child table must either match an existing primary key value in the parent table or be NULL, if allowed. This prevents the occurrence of **dangling references**, where a tuple refers to a non-existent entity.

SQL enforces referential integrity through the FOREIGN KEY clause. Any attempt to insert, update, or delete data that violates the referential constraint is rejected by the database system.

Actions on Referential Integrity Violations

SQL allows the database designer to specify actions that should be taken when a referenced tuple in the parent table is updated or deleted. These actions are defined using the **ON DELETE** and **ON UPDATE** clauses. Common actions include:

- **CASCADE**, which propagates the delete or update operation to the related tuples in the child table
- **SET NULL**, which sets the foreign key values in the child table to NULL
- **SET DEFAULT**, which assigns a default value to the foreign key attributes
- **RESTRICT or NO ACTION**, which prevents the operation if related tuples exist

These options provide flexibility in handling referential integrity while reflecting real-world business rules.

3 Domain Integrity Constraint

The **domain integrity constraint** ensures that the values stored in an attribute belong to a defined domain. This is enforced in SQL through **data types**, **NOT NULL constraints**, **CHECK constraints**, and **DEFAULT values**. For example, an attribute storing salary values may be restricted to positive numbers using a CHECK constraint. Domain integrity prevents invalid data from being entered into the database.

Data Type Constraint

attribute_name datatype

NOT NULL Constraint

attribute_name datatype NOT NULL

DEFAULT Constraint

attribute_name datatype DEFAULT value

Check Constraint

The **CHECK constraint** allows the specification of a Boolean condition that must be satisfied by all tuples in a table. This constraint is used to enforce application-specific rules at the database level. For instance, a CHECK constraint can ensure that an employee's salary is greater than zero or that a student's CGPA lies within a valid range. SQL evaluates the CHECK condition whenever data is inserted or updated.

CHECK Constraint (Attribute Level)

attribute_name datatype CHECK (condition)

CHECK Constraint (Table Level)

CHECK (condition)

3.5.3 Assertion of Integrity Constraints

All key and integrity constraints in SQL are **declarative**, meaning they are specified as part of the table definition rather than through procedural code. The responsibility of enforcing these constraints lies with the **DBMS**, which automatically checks constraint violations during insert, update, and delete

operations. This approach simplifies application development and ensures uniform enforcement of rules across all database users and applications.

Importance of Implementing Key and Integrity Constraints

The implementation of key and integrity constraints provides several important benefits. It ensures **data consistency and correctness**, prevents invalid data entry, eliminates redundancy-related anomalies, and maintains meaningful relationships among tables. By enforcing constraints at the database level, SQL ensures that data integrity is preserved regardless of how the database is accessed.

3.6 Nested Queries

A **nested query** (also called a **subquery**) is an SQL query that is **embedded within another SQL query**. The outer query is known as the **parent query**, while the inner query is referred to as the **subquery**. Nested queries are used when the result of one query is required as input to another query. This capability significantly enhances the expressive power of SQL and allows the formulation of complex queries in a concise and declarative manner.

A subquery is typically enclosed within parentheses and may appear in different parts of an SQL statement, most commonly in the **WHERE clause**, but also in the **FROM** and **SELECT** clauses. The result of a subquery may be a single value, a list of values, or a relation, depending on its structure and usage. SQL evaluates the subquery first and then uses its result to evaluate the outer query.

Basic Syntax of a Nested Query

The general form of a nested query using the WHERE clause is:

```
SELECT attribute_list
FROM relation_name
WHERE attribute operator (SELECT attribute
                          FROM relation_name
                          WHERE condition);
```

The inner query produces a result that is compared with values in the outer query using relational or logical operators.

Types of Nested Queries

Nested queries can be classified based on the **number of values returned by the subquery** and on whether the subquery is **correlated** with the outer query.

Single-Row Subqueries

A **single-row subquery** returns exactly one value. Such subqueries are commonly used with comparison operators such as =, >, <, >=, and <=.

For example, consider the relation EMPLOYEE(Emp_ID, Emp_Name, Salary, Dept_ID). To find employees whose salary is greater than the average salary of all employees, the following nested query can be used:

```
SELECT Emp_Name
FROM EMPLOYEE
WHERE Salary >
    (SELECT AVG(Salary)
     FROM EMPLOYEE);
```

In this query, the subquery computes the average salary, and the outer query selects employees whose salary exceeds this value. The subquery is evaluated once, and its result is used by the outer query.

Multiple-Row Subqueries

A **multiple-row subquery** returns more than one value. Such subqueries are used with operators like IN, ANY, and ALL.

The IN operator is used to test whether a value matches any value in the result of a subquery. For example, to retrieve employees who work in departments located in ‘Block A’:

```
SELECT Emp_Name
FROM EMPLOYEE
WHERE Dept_ID IN
    (SELECT Dept_ID
     FROM DEPARTMENT
     WHERE Location = 'Block A');
```

Here, the subquery returns a list of department identifiers, and the outer query selects employees whose department matches any of those identifiers.

The ANY and ALL operators allow comparisons between a value and a set of values returned by a subquery. The ANY operator evaluates to true if the comparison is true for **at least one value**, while the ALL operator requires the comparison to be true for **all values**.

Nested Queries Using EXISTS

The **EXISTS** operator is used to test whether a subquery returns at least one tuple. It returns TRUE if the subquery result is non-empty and FALSE otherwise. EXISTS is often used with correlated subqueries.

For example, to find departments that have at least one employee:

```
SELECT Dept_Name
FROM DEPARTMENT D
WHERE EXISTS
    (SELECT *
     FROM EMPLOYEE E
     WHERE E.Dept_ID = D.Dept_ID);
```

In this query, the subquery checks for the existence of at least one employee in each department

Correlated Subqueries

A **correlated subquery** is a subquery that depends on values from the outer query. Unlike simple subqueries, correlated subqueries are evaluated **once for each tuple processed by the outer query**.

For example, to find employees whose salary is greater than the average salary of their own department:

```
SELECT Emp_Name
FROM EMPLOYEE E
WHERE Salary >
    (SELECT AVG(Salary)
     FROM EMPLOYEE
```

```
    WHERE Dept_ID = E.Dept_ID);
```

In this case, the subquery uses the Dept_ID value from the outer query. As a result, the subquery is executed repeatedly, once for each employee tuple.

Nested Queries in the FROM Clause

Nested queries can also appear in the FROM clause, where they are treated as **derived tables**. The result of the subquery is given an alias and used like a regular relation.

For example:

```
SELECT Dept_ID, Avg_Salary
FROM (SELECT Dept_ID, AVG(Salary) AS Avg_Salary
    FROM EMPLOYEE
    GROUP BY Dept_ID) AS Dept_Avg;
```

This query first computes the average salary for each department and then retrieves the results from the derived relation.

Nested Queries in the SELECT Clause

Subqueries may also be used in the SELECT clause to compute values for each tuple in the result.

For example:

```
SELECT Emp_Name,
    (SELECT AVG(Salary)
     FROM EMPLOYEE) AS Avg_Salary
FROM EMPLOYEE;
```

Here, the average salary is computed once and displayed alongside each employee record.

Advantages of Nested Queries

Nested queries allow complex conditions to be expressed naturally and concisely. They closely reflect the logical structure of many real-world queries and improve query readability. Since SQL is declarative, nested queries enable users to specify complex requirements without explicitly describing execution steps.

3.7 Subqueries

A **subquery** is an SQL query that is **embedded within another SQL statement**. It is also referred to as an **inner query**, while the query that contains it is called the **outer query**. Subqueries are used when the result of one query is required to determine the result of another query. This feature enhances the expressive power of SQL and enables users to formulate complex queries in a clear and structured manner.

A subquery is always enclosed within parentheses and can appear in various clauses of an SQL statement, most commonly in the **WHERE clause**, but also in the **FROM** and **SELECT** clauses. The result produced by a subquery may be a **single value**, a **set of values**, or a **relation**, depending on the form of the subquery and its context of use. In general, the DBMS evaluates the subquery first and then uses its result while evaluating the outer query.

Basic Syntax of a Subquery

The general syntax of a subquery used in the WHERE clause is as follows:

```
SELECT attribute_list
FROM relation_name
WHERE attribute operator
    (SELECT attribute
     FROM relation_name
     WHERE condition);
```

The operator used in the outer query depends on whether the subquery returns a single value or multiple values.

Classification of Subqueries

Subqueries can be classified based on the **number of rows returned** and on whether they are **correlated** with the outer query.

Single-Row Subqueries

A **single-row subquery** returns exactly one value. Such subqueries are used with **single-row comparison operators** such as =, >, <, >=, and <=. These subqueries are typically used when the inner query computes an aggregate value such as AVG, MAX, or MIN.

For example, consider the relation EMPLOYEE(Emp_ID, Emp_Name, Salary). To find employees whose salary is greater than the average salary of all employees, the following subquery is used:

```
SELECT Emp_Name
FROM EMPLOYEE
WHERE Salary >
    (SELECT AVG(Salary)
     FROM EMPLOYEE);
```

In this query, the subquery calculates the average salary, and the outer query selects employees whose salary exceeds that value.

Multiple-Row Subqueries

A **multiple-row subquery** returns more than one value. Such subqueries are used with operators like **IN**, **ANY**, and **ALL**.

The **IN** operator checks whether a value matches any value in the set returned by the subquery. For example, to retrieve employees working in departments located in 'Block A':

```
SELECT Emp_Name
FROM EMPLOYEE
WHERE Dept_ID IN
    (SELECT Dept_ID
     FROM DEPARTMENT
     WHERE Location = 'Block A');
```

The **ANY** operator returns true if the comparison is satisfied for at least one value returned by the subquery, whereas the **ALL** operator requires the comparison to be satisfied for all values returned by the subquery.

Subqueries Using EXISTS

The **EXISTS** operator is used to test whether a subquery returns at least one tuple. It evaluates to TRUE if the result of the subquery is non-empty and FALSE otherwise. EXISTS is often used with correlated subqueries.

For example, to find departments that have at least one employee:

```
SELECT Dept_Name
FROM DEPARTMENT D
WHERE EXISTS
    (SELECT *
     FROM EMPLOYEE E
     WHERE E.Dept_ID = D.Dept_ID);
```

In this query, the subquery checks the existence of related tuples in the EMPLOYEE table for each department.

Correlated Subqueries

A **correlated subquery** is a subquery that depends on one or more values from the outer query. Unlike simple subqueries, correlated subqueries are evaluated **once for each row processed by the outer query**.

For example, to find employees whose salary is greater than the average salary of their respective departments:

```
SELECT Emp_Name
FROM EMPLOYEE E
WHERE Salary >
    (SELECT AVG(Salary)
     FROM EMPLOYEE
     WHERE Dept_ID = E.Dept_ID);
```

Here, the subquery refers to E.Dept_ID from the outer query, making it correlated. As a result, the subquery is repeatedly executed for each employee.

Subqueries in the FROM Clause

Subqueries can also appear in the **FROM clause**, where they are treated as **derived tables** or temporary relations. The result of such a subquery must be given an alias and can be queried like a regular table.

For example:

```
SELECT Dept_ID, Avg_Salary
FROM (SELECT Dept_ID, AVG(Salary) AS Avg_Salary
```

```
    FROM EMPLOYEE
    GROUP BY Dept_ID) AS Dept_Avg;
```

In this case, the subquery computes the average salary for each department, and the outer query retrieves the results from the derived relation.

Subqueries in the SELECT Clause

Subqueries may also be used in the **SELECT clause** to compute scalar values that are displayed along with each tuple in the result.

For example:

```
SELECT Emp_Name,
    (SELECT AVG(Salary)
     FROM EMPLOYEE) AS Avg_Salary
FROM EMPLOYEE;
```

The subquery computes the average salary once, and the result is displayed for each employee.

Comparison of Subqueries and Joins

Subqueries and joins are often interchangeable in SQL. However, subqueries provide a more **natural and readable representation** for certain types of queries, especially those involving conditions such as existence or comparison with aggregate values. Joins are generally preferred for performance optimization, while subqueries emphasize clarity and logical structure.

3.8 Grouping in SQL

In SQL, **grouping** is a mechanism used to organize tuples of a relation into groups based on the values of one or more attributes. Grouping is primarily used in conjunction with **aggregate functions** to perform computations such as counting, summing, averaging, finding minimums, and maximums over sets of tuples rather than on individual tuples. The grouping operation enables meaningful data analysis and summarization, which is essential for reporting and decision-making applications.

The grouping operation in SQL is implemented using the **GROUP BY clause**. When a GROUP BY clause is specified in a query, the database system partitions the tuples of the relation into groups such that all tuples in a group have the same

values for the grouping attributes. Aggregate functions are then applied independently to each group, producing one result row per group.

Aggregate Functions and Grouping

Aggregate functions operate on a set of tuples and return a single value. Common aggregate functions supported by SQL include **COUNT**, **SUM**, **AVG**, **MIN**, and **MAX**. When aggregate functions are used without a GROUP BY clause, they operate on the entire relation and return a single row. When used with GROUP BY, they operate on each group separately.

For example, consider an EMPLOYEE relation with attributes Emp_ID, Emp_Name, Salary, and Dept_ID. If the average salary of all employees is required, the AVG function can be used without grouping. However, if the average salary of employees in each department is required, grouping by Dept_ID becomes necessary.

Syntax of GROUP BY Clause

The general syntax of a query using grouping is:

SELECT grouping_attribute(s), aggregate_function(attribute)

FROM relation_name

WHERE condition

GROUP BY grouping_attribute(s);

In this syntax, the GROUP BY clause specifies the attributes used to form groups. The WHERE clause, if present, filters tuples **before** grouping is performed.

Grouping Example

Consider the EMPLOYEE relation shown below:

Emp_ID	Emp_Name	Salary	Dept_ID
101	Ravi	45000	10
102	Anita	40000	10
103	Kiran	42000	20
104	Meena	48000	20

To find the average salary of employees in each department, the following query is used:

```
SELECT Dept_ID, AVG(Salary)
FROM EMPLOYEE
GROUP BY Dept_ID;
```

This query groups employees by Dept_ID and computes the average salary for each group. The result contains one row for each department.

Rules for Using GROUP BY

SQL enforces certain rules when using the GROUP BY clause. Every attribute appearing in the SELECT clause must either be included in the GROUP BY clause or be an argument of an aggregate function. This rule ensures that the values displayed in the SELECT clause are well-defined for each group. Violation of this rule results in a syntax or semantic error.

HAVING Clause

The **HAVING clause** is used to apply conditions to groups formed by the GROUP BY clause. While the WHERE clause filters individual tuples **before grouping**, the HAVING clause filters entire groups **after grouping** has been performed. Thus, HAVING is used to specify conditions on aggregate values.

The general syntax including HAVING is:

```
SELECT grouping_attribute(s), aggregate_function(attribute)
FROM relation_name
WHERE condition
GROUP BY grouping_attribute(s)
HAVING group_condition;
```

Example Using HAVING

To find departments whose average salary is greater than 45,000, the following query can be written:

```
SELECT Dept_ID, AVG(Salary)
FROM EMPLOYEE
GROUP BY Dept_ID
HAVING AVG(Salary) > 45000;
```

Example Using HAVING

To find departments whose average salary is greater than 45,000, the following query can be written:

```
SELECT Dept_ID, AVG(Salary)

FROM EMPLOYEE

GROUP BY Dept_ID

HAVING AVG(Salary) > 45000;
```

In this query, grouping is first performed on Dept_ID, aggregate values are computed, and then the HAVING condition filters out groups that do not satisfy the condition.

Difference Between WHERE and HAVING

The WHERE and HAVING clauses serve different purposes in grouped queries. The WHERE clause filters tuples before grouping and cannot contain aggregate functions. The HAVING clause filters groups after grouping and typically contains aggregate conditions. Understanding this distinction is essential for writing correct SQL queries involving grouping.

Grouping on Multiple Attributes

SQL allows grouping based on more than one attribute. In such cases, tuples are grouped based on the unique combinations of the specified attributes. This form of grouping is useful when summaries are required at multiple levels.

For example:

```
SELECT Dept_ID, Job_Type, COUNT(*)

FROM EMPLOYEE

GROUP BY Dept_ID, Job_Type;
```

This query groups employees by both department and job type and counts the number of employees in each group.

NULL Values and Grouping

When grouping is performed, tuples with NULL values in grouping attributes form a separate group. Aggregate functions ignore NULL values except for COUNT(*), which counts all tuples including those with NULL values.

3.9 Aggregation in SQL

In SQL, **aggregation** refers to the process of computing a **single value from a set of tuples**. Aggregation is performed using **aggregate functions**, which operate on a collection of values and return a single summarized result. Aggregation is fundamental to database querying, as it enables users to derive meaningful information such as totals, averages, counts, minimums, and maximums from large volumes of data.

Aggregate functions may be applied to the entire relation or to **groups of tuples** formed using the GROUP BY clause. When aggregation is performed without grouping, the result is a single-row output. When aggregation is combined with grouping, the result contains one row for each group.

Aggregate Functions

SQL provides a set of built-in **aggregate functions** that operate on numeric or non-numeric attributes. The most commonly used aggregate functions are **COUNT**, **SUM**, **AVG**, **MIN**, and **MAX**. These functions ignore NULL values, except for COUNT(*), which counts all tuples including those containing NULL values.

The COUNT function returns the number of tuples in a relation or group. COUNT(attribute) counts only non-NULL values of the specified attribute, whereas COUNT(*) counts all tuples. The SUM function returns the total of non-NULL values in a numeric attribute. The AVG function computes the average of non-NULL numeric values. The MIN and MAX functions return the smallest and largest values, respectively, from the specified attribute.

Syntax of Aggregate Functions

The general syntax for using aggregate functions in SQL is:

SELECT aggregate_function(attribute)

FROM relation_name

WHERE condition;

When aggregation is applied to groups, the syntax includes the GROUP BY clause:

SELECT grouping_attribute(s), aggregate_function(attribute)

FROM relation_name

WHERE condition

GROUP BY grouping_attribute(s);

Emp_ID	Emp_Name	Salary	Dept_ID
101	Ravi	45000	10
102	Anita	40000	10
103	Kiran	42000	20
104	Meena	48000	20

In these queries, the WHERE clause filters tuples before aggregation is performed.

Aggregation Without GROUP BY

When an aggregate function is used without a GROUP BY clause, it operates on **all tuples of the relation** and returns a single result. For example, consider the EMPLOYEE relation shown below.

To find the average salary of all employees, the following query is used:

SELECT AVG(Salary)

FROM EMPLOYEE;

This query returns a single value representing the average salary across the entire EMPLOYEE table.

Aggregation with GROUP BY

When aggregate functions are used together with the GROUP BY clause, aggregation is performed **separately for each group**. Grouping partitions the relation into subsets based on the values of the grouping attributes, and aggregate functions compute a result for each subset.

For example, to find the total salary paid in each department, the following query can be written:

SELECT Dept_ID, SUM(Salary)

FROM EMPLOYEE

GROUP BY Dept_ID;

This query groups employees by Dept_ID and computes the sum of salaries for each department, producing one result row per department.

Rules for Using Aggregate Functions

SQL enforces specific rules when aggregate functions are used in a SELECT statement. Any attribute that appears in the SELECT clause but is not part of an aggregate function must appear in the GROUP BY clause. This rule ensures that the query result is unambiguous and well-defined for each group.

Aggregate functions cannot be used in the WHERE clause because WHERE filters tuples before aggregation. Instead, conditions involving aggregate values must be specified using the HAVING clause.

HAVING Clause and Aggregation

The **HAVING clause** is used to apply conditions on aggregated results. It filters groups formed by the GROUP BY clause, whereas the WHERE clause filters individual tuples before grouping.

The general syntax is:

SELECT grouping_attribute(s), aggregate_function(attribute)

FROM relation_name

GROUP BY grouping_attribute(s)

HAVING aggregate_condition;

For example, to find departments whose average salary exceeds 45,000:

SELECT Dept_ID, AVG(Salary)

FROM EMPLOYEE

GROUP BY Dept_ID

HAVING AVG(Salary) > 45000;

In this query, aggregation is performed first, and then groups that do not satisfy the HAVING condition are removed.

DISTINCT and Aggregate Functions

The DISTINCT keyword can be used within aggregate functions to consider only **unique values**. For example, COUNT(DISTINCT Dept_ID) counts the number of distinct departments, ignoring duplicates.

SELECT COUNT(DISTINCT Dept_ID)

FROM EMPLOYEE;

This query returns the number of unique departments in the EMPLOYEE table.

NULL Values and Aggregation

Aggregate functions generally ignore NULL values. For example, AVG(Salary) computes the average using only non-NULL salary values. However, COUNT(*) includes all tuples, regardless of NULL values. Understanding the treatment of NULL values is important for obtaining correct aggregation results.

Aggregation in Nested Queries

Aggregate functions are often used within subqueries to compute values that are then used in outer queries. For example, a subquery may compute the maximum salary, and the outer query may retrieve employees earning that salary.

```
SELECT Emp_Name

FROM EMPLOYEE

WHERE Salary =

    (SELECT MAX(Salary)

     FROM EMPLOYEE);
```

Here, the aggregate function MAX is used within a subquery to determine the highest salary.

3.10 Ordering in SQL

In SQL, **ordering** refers to the process of arranging the tuples in the result of a query in a specified sequence. Ordering is an important feature in data retrieval because the result of an SQL query, by default, is treated as a set of tuples with **no guaranteed order**. When a specific order of rows is required, SQL provides the **ORDER BY clause**, which allows the user to sort the query result based on one or more attributes.

The ORDER BY clause is applied after the result set has been produced by selection, projection, grouping, and aggregation operations. It does not affect the data stored in the database but only controls the presentation of the query output.

Syntax of ORDER BY Clause

The general syntax of a query with ordering is:

```
SELECT attribute_list
```

FROM relation_name

WHERE condition

GROUP BY grouping_attribute(s)

HAVING group_condition

ORDER BY attribute_list [ASC | DESC];

In this syntax, the ORDER BY clause appears at the **end of the SQL statement**. The attributes specified in ORDER BY determine the sorting criteria, and the optional keywords **ASC** and **DESC** indicate ascending or descending order, respectively. By default, ordering is performed in ascending order.

Ordering on a Single Attribute

Ordering may be performed on a single attribute. For example, consider the EMPLOYEE relation shown

Emp_ID	Emp_Name	Salary	Dept_ID
101	Ravi	45000	10
102	Anita	40000	10
103	Kiran	42000	20
104	Meena	48000	20

To display employees in ascending order of salary, the following query can be used:

SELECT Emp_Name, Salary

FROM EMPLOYEE

ORDER BY Salary;

To display the same result in descending order of salary:

SELECT Emp_Name, Salary

FROM EMPLOYEE

ORDER BY Salary DESC;

Ordering on Multiple Attributes

SQL allows ordering based on multiple attributes. In such cases, tuples are first sorted according to the first attribute, and ties are resolved by sorting on the subsequent attributes.

For example, to order employees first by department and then by salary within each department:

SELECT Emp_Name, Dept_ID, Salary

FROM EMPLOYEE

ORDER BY Dept_ID ASC, Salary DESC;

This query groups employees by department in ascending order and sorts salaries in descending order within each department.

Ordering Using Column Position

In SQL, attributes in the ORDER BY clause may be specified using their **position numbers** as they appear in the SELECT list. This approach is useful when attribute names are lengthy or when derived columns are used.

For example:

SELECT Emp_Name, Salary

FROM EMPLOYEE

ORDER BY 2 DESC;

Here, the number 2 refers to the second attribute in the SELECT list, which is Salary.

Ordering with Aggregate Functions

Ordering can also be applied to results involving aggregate functions. When GROUP BY is used, ordering is performed on the grouped result.

For example, to display departments in descending order of average salary:

SELECT Dept_ID, AVG(Salary) AS Avg_Salary

FROM EMPLOYEE

GROUP BY Dept_ID

ORDER BY Avg_Salary DESC;

In this query, ordering is applied after aggregation

ORDER BY and NULL Values

The treatment of NULL values in ordering depends on the DBMS implementation. Generally, NULL values may appear either at the beginning or at the end of the sorted result. Some DBMSs allow explicit control over the placement of NULL values using keywords such as **NULLS FIRST** or **NULLS LAST**.

Example:

SELECT Emp_Name, Salary

FROM EMPLOYEE

ORDER BY Salary DESC NULLS LAST;

ORDER BY with DISTINCT

The ORDER BY clause can be used together with DISTINCT to sort the unique values returned by a query.

For example:

SELECT DISTINCT Dept_ID

FROM EMPLOYEE

ORDER BY Dept_ID;

This query retrieves distinct department identifiers and displays them in sorted order.

Rules and Restrictions of ORDER BY

The ORDER BY clause must appear **after** the SELECT, FROM, WHERE, GROUP BY, and HAVING clauses. Attributes specified in ORDER BY generally should appear in the SELECT list, especially when DISTINCT is used. Ordering is applied to the final result set and does not change the underlying table.

Rules and Restrictions of ORDER BY

The ORDER BY clause must appear **after** the SELECT, FROM, WHERE, GROUP BY, and HAVING clauses. Attributes specified in ORDER BY generally should appear in the SELECT list, especially when DISTINCT is used. Ordering is applied to the final result set and does not change the underlying table.

3.11 Implementation of Different Types of Joins in SQL

In a relational database, data is distributed across multiple tables to reduce redundancy and improve data integrity. To retrieve meaningful information from such distributed data, it is often necessary to **combine tuples from two or more relations** based on related attributes. This operation is known as a **join**. In SQL, joins are implemented using- -the **JOIN operation**, which conceptually corresponds to the join operation of relational algebra. SQL provides several types of joins to support different data retrieval requirements, each with distinct semantics and usage.

Inner Join

An **inner join** retrieves tuples from two relations only when there is a **matching value in both relations** based on a specified join condition. Tuples that do not satisfy the join condition are excluded from the result. Inner joins are the most commonly used joins in SQL.

The general syntax of an inner join is:

SELECT attribute_list

FROM table1 INNER JOIN table2

ON join_condition;

Consider two relations, EMPLOYEE(Emp_ID, Emp_Name, Dept_ID) and DEPARTMENT(Dept_ID, Dept_Name). To retrieve employee names along with their department names, an inner join can be used as follows:

SELECT Emp_Name, Dept_Name

FROM EMPLOYEE INNER JOIN DEPARTMENT

ON EMPLOYEE.Dept_ID = DEPARTMENT.Dept_ID;

This query returns only those employees who are associated with a valid department. Employees without a matching department are not included in the result.

Equi Join

An **equi join** is a special case of an inner join in which the join condition uses the **equality operator (=)**. Although SQL does not provide a separate keyword for equi join, it is implemented using an inner join with an equality condition.

SELECT Emp_Name, Dept_Name

FROM EMPLOYEE, DEPARTMENT

WHERE EMPLOYEE.Dept_ID = DEPARTMENT.Dept_ID;

In this form, the join condition is specified in the WHERE clause. The result contains columns from both tables where the specified attributes are equal.

Natural Join

A **natural join** automatically joins two relations based on **all attributes with the same name and compatible data types** in both relations. SQL eliminates duplicate columns from the result.

The syntax of a natural join is:

SELECT attribute_list

FROM table1 NATURAL JOIN table2;

Example:

SELECT Emp_Name, Dept_Name

FROM EMPLOYEE NATURAL JOIN DEPARTMENT;

Here, SQL implicitly uses the common attribute Dept_ID to perform the join. While natural joins simplify query writing, they must be used with caution, as unintended joins may occur if multiple attributes share the same name.

Outer Join

An **outer join** retrieves not only matching tuples but also **non-matching tuples** from one or both relations, padding missing attribute values with NULLs. Outer joins are used when it is necessary to preserve tuples that would otherwise be excluded by an inner join.

SQL supports three types of outer joins: **left outer join**, **right outer join**, and **full outer join**.

Left Outer Join

A **left outer join** returns all tuples from the **left table** and the matching tuples from the right table. If no matching tuple exists in the right table, NULL values are included for the right table's attributes.

SELECT Emp_Name, Dept_Name

FROM EMPLOYEE LEFT OUTER JOIN DEPARTMENT

ON EMPLOYEE.Dept_ID = DEPARTMENT.Dept_ID;

This query returns all employees, including those who are not assigned to any department.

Right Outer Join

A **right outer join** returns all tuples from the **right table** and the matching tuples from the left table. Non-matching tuples from the left table are represented with NULL values.

SELECT Emp_Name, Dept_Name

FROM EMPLOYEE RIGHT OUTER JOIN DEPARTMENT

ON EMPLOYEE.Dept_ID = DEPARTMENT.Dept_ID;

This query returns all departments, including those that currently have no employees.

Full Outer Join

A **full outer join** returns all tuples from both relations. Matching tuples are combined, and non-matching tuples from either relation are included with NULL values where no match exists.

SELECT Emp_Name, Dept_Name

FROM EMPLOYEE FULL OUTER JOIN DEPARTMENT

ON EMPLOYEE.Dept_ID = DEPARTMENT.Dept_ID;

This join ensures that no data from either table is lost.

Cross Join

A **cross join** produces the **Cartesian product** of two relations. Every tuple from the first relation is combined with every tuple from the second relation. Cross joins do not require a join condition.

The syntax is:

SELECT attribute_list

FROM table1 CROSS JOIN table2;

Example:

SELECT Emp_Name, Dept_Name

FROM EMPLOYEE CROSS JOIN DEPARTMENT;

Cross joins are rarely used in practical applications due to the large size of the result but are useful in specific analytical scenarios.

Self Join

A **self join** is a join in which a table is joined with itself. It is useful for representing **recursive or hierarchical relationships**, such as employees and their managers.

To implement a self join, the table must be referenced using **aliases**.

Example:

SELECT E1.Emp_Name AS Employee, E2.Emp_Name AS Manager

FROM EMPLOYEE E1, EMPLOYEE E2

WHERE E1.Manager_ID = E2.Emp_ID;

Here, the EMPLOYEE table is joined with itself to associate employees with their managers.

Join Implementation Using WHERE Clause vs JOIN Clause

Joins in SQL can be implemented using either the **JOIN keyword** or by specifying join conditions in the **WHERE clause**. While both approaches yield the same result for inner joins, the JOIN syntax is preferred as it improves **readability, clarity, and maintainability** of queries, especially when multiple tables are involved.

Importance of Joins in SQL

Joins are essential for retrieving related data stored across multiple tables. They enable meaningful data integration, support complex queries, and reflect the relationships defined in the database schema. Efficient use of joins is fundamental to relational database querying and application development.

3.12 Views in SQL (Updatable and Non-Updatable Views)

In SQL, a **view** is a **virtual table** that is derived from one or more base tables (relations) using a SELECT query. A view does not store data physically; instead, it stores the **definition of a query**, and the data displayed by the view is dynamically retrieved from the underlying base tables whenever the view is referenced. Views are used to provide **logical data independence**, enhance

security, simplify complex queries, and present data in a customized form to different users.

A view behaves like a table from the user's perspective and can be queried using standard SQL statements such as SELECT. However, since views are derived from base tables, their ability to support data modification operations depends on the nature of their definition.

Creation of a View

A view is created using the **CREATE VIEW** statement. The general syntax is:

CREATE VIEW view_name AS

SELECT attribute_list

FROM relation_name

WHERE condition;

Once created, the view can be used in queries in the same way as a base table.

For example, consider an EMPLOYEE table with attributes Emp_ID, Emp_Name, Salary, and Dept_ID. A view that displays only employee names and salaries can be defined as follows:

CREATE VIEW Emp_Salary_View AS

SELECT Emp_Name, Salary

FROM EMPLOYEE;

This view presents a simplified representation of the EMPLOYEE table.

Purpose and Advantages of Views

Views serve several important purposes in database systems. They allow users to see only relevant data, thereby enhancing **security** by restricting access to sensitive attributes. Views also provide **query simplification**, as complex queries involving joins and conditions can be encapsulated within a view definition. In addition, views support **logical data independence**, since changes to the underlying base tables may not require changes to user queries that reference the view.

3.12.1Updatable Views

An **updatable view** is a view through which **data modification operations** such as INSERT, UPDATE, and DELETE can be performed. When such operations

are executed on an updatable view, the changes are propagated to the underlying base table(s). Whether a view is updatable depends on the structure of its defining query.

In general, a view is considered updatable if it is derived from a **single base table** and does not contain constructs that make tuple mapping ambiguous. The view definition must include all necessary attributes required to uniquely identify tuples in the base table, typically including the primary key.

For example, consider the following view:

```
CREATE VIEW CSE_Employees AS

SELECT Emp_ID, Emp_Name, Salary

FROM EMPLOYEE

WHERE Dept_ID = 'CSE';
```

This view is updatable because it is based on a single table and does not use aggregate functions, grouping, or joins. An update operation on this view, such as increasing salaries, directly affects the corresponding rows in the EMPLOYEE table.

```
UPDATE CSE_Employees

SET Salary = Salary + 5000;
```

Conditions for an Updatable View

A view is generally updatable if it satisfies the following conditions:

- It is based on a **single base table**
- It does not use **DISTINCT**
- It does not contain **aggregate functions**
- It does not use **GROUP BY** or **HAVING**
- It does not involve **joins**
- It does not include **set operations** such as UNION
- It includes all **NOT NULL attributes** without default values from the base table

When these conditions are met, SQL can unambiguously map view rows to base table rows.

3.12.2.Non-Updatable Views

A **non-updatable view** is a view through which data modification operations are **not permitted**. Such views are read-only, meaning they can be used only for data retrieval. Views become non-updatable when their definitions include operations that prevent the DBMS from determining how updates should be applied to the underlying tables.

Views involving **joins, aggregate functions, grouping, DISTINCT, or set operations** are typically non-updatable.

For example, consider a view that shows the average salary of employees in each department:

CREATE VIEW Dept_Avg_Salary AS

SELECT Dept_ID, AVG(Salary) AS Avg_Salary

FROM EMPLOYEE

GROUP BY Dept_ID;

This view is non-updatable because it uses an aggregate function and GROUP BY clause. Updating the Avg_Salary value does not correspond to a unique update in the base table.

Similarly, a view defined using a join is non-updatable:

CREATE VIEW Emp_Dept_View AS

SELECT Emp_Name, Dept_Name

FROM EMPLOYEE E JOIN DEPARTMENT D

ON E.Dept_ID = D.Dept_ID;

Since the view is derived from multiple tables, SQL cannot determine which base table should be updated for a given modification.

Since the view is derived from multiple tables, SQL cannot determine which base table should be updated for a given modification.

CREATE VIEW CSE_Employees AS

SELECT Emp_ID, Emp_Name, Salary

FROM EMPLOYEE

WHERE Dept_ID = 'CSE'

WITH CHECK OPTION;

In this case, an update that changes the Dept_ID of an employee to a value other than 'CSE' through the view is rejected, ensuring that all rows visible through the view continue to satisfy the view condition.

Dropping a View

A view can be removed from the database using the **DROP VIEW** statement:

DROP VIEW view_name;

Dropping a view does not affect the underlying base tables.

Comparison Between Updatable and Non-Updatable Views

Updatable views allow INSERT, UPDATE, and DELETE operations and are typically defined on a single base table without complex SQL constructs. Non-updatable views are read-only and usually involve joins, aggregation, grouping, or set operations. While updatable views support data modification, non-updatable views are mainly used for reporting and analysis.

3.13 Relational Set Operations in SQL

In the relational model, relations are treated as **sets of tuples**, and therefore set-theoretic operations play an important role in data manipulation and retrieval. **Relational set operations** allow the results of two or more SELECT queries to be combined using principles derived from **set theory**. SQL supports a limited but powerful set of such operations, enabling users to merge, compare, and filter query results in a structured manner. The primary relational set operations supported by SQL are **UNION**, **INTERSECT**, and **EXCEPT** (called **MINUS** in some DBMSs such as Oracle).

Relational set operations are applied to the **results of SELECT queries**, not directly to base tables. Each participating query produces a relation, and the set operation combines these relations to produce a final result.

Conditions for Applying Set Operations

For relational set operations to be valid in SQL, certain conditions must be satisfied. The relations produced by the SELECT queries must be **union-compatible**. This means that both queries must return the **same number of attributes**, and the corresponding attributes must have **compatible data types**. The attribute names of the result are taken from the first SELECT query. These rules ensure that tuples from different queries can be meaningfully compared and combined.

UNION Operation

The **UNION** operation combines the result sets of two SELECT queries and returns all distinct tuples that appear in either result. Duplicate tuples are automatically eliminated, consistent with set semantics.

The general syntax of the UNION operation is:

SELECT attribute_list

FROM relation1

WHERE condition

UNION

SELECT attribute_list

FROM relation2

WHERE condition;

For example, consider two relations, CSE_STUDENT and IT_STUDENT, each storing student identifiers. To retrieve a list of all students belonging to either department, the UNION operation can be used:

SELECT Student_ID

FROM CSE_STUDENT

UNION

SELECT Student_ID

FROM IT_STUDENT;

The result contains unique student identifiers that appear in either table.

UNION ALL

SQL also provides **UNION ALL**, which is a variant of UNION that **does not eliminate duplicate tuples**. It preserves all tuples from both result sets and is therefore more efficient when duplicate elimination is not required.

The syntax is:

SELECT attribute_list

FROM relation1

UNION ALL

SELECT attribute_list

FROM relation2;

UNION ALL is often used when duplicates are meaningful or when performance is a concern.

INTERSECT Operation

The **INTERSECT** operation returns only those tuples that appear in **both** result sets of the two SELECT queries. It corresponds to the set intersection operation in relational algebra.

The general syntax is:

SELECT attribute_list

FROM relation1

INTERSECT

SELECT attribute_list

FROM relation2;

For example, to find students who are enrolled in both the CSE and IT programs:

SELECT Student_ID

FROM CSE_STUDENT

INTERSECT

SELECT Student_ID

FROM IT_STUDENT;

The result includes only those student identifiers common to both relations.

EXCEPT (MINUS) Operation

The **EXCEPT** operation returns tuples that appear in the result of the first SELECT query but **not** in the result of the second query. This operation corresponds to the set difference operation in relational algebra. Some DBMSs use the keyword **MINUS** instead of EXCEPT.

The general syntax is:

SELECT attribute_list

FROM relation1

EXCEPT

SELECT attribute_list

FROM relation2;

For example, to retrieve students who belong to the CSE department but not to the IT department:

SELECT Student_ID

FROM CSE_STUDENT

EXCEPT

SELECT Student_ID

FROM IT_STUDENT;

The result contains only those students unique to the CSE department.

Use of DISTINCT and Set Semantics

By default, the UNION, INTERSECT, and EXCEPT operations apply **DISTINCT semantics**, meaning that duplicate tuples are removed from the result. This behavior aligns with the mathematical definition of sets. The exception is UNION ALL, which retains duplicates. Understanding this distinction is important for both correctness and performance considerations.

Ordering of Results in Set Operations

When set operations are used, the **ORDER BY clause can be applied only once**, and it must appear at the **end of the entire compound query**. Ordering is performed on the final result produced after applying the set operation.

For example:

SELECT Student_ID

FROM CSE_STUDENT

UNION

SELECT Student_ID

FROM IT_STUDENT

ORDER BY Student_ID;

Relational Algebra Perspective

From the relational algebra point of view, SQL set operations closely correspond to classical set operations. UNION corresponds to the union operator, INTERSECT to the intersection operator, and EXCEPT to the difference operator. These operations- emphasize the set-based nature of relational databases and reinforce the theoretical foundation of SQL.

Importance of Relational Set Operations

Relational set operations provide a concise and expressive way to combine query results without requiring joins or nested queries. They are particularly useful when comparing datasets, merging similar data from different sources, and performing analytical queries. Proper use of these operations enhances query clarity and aligns SQL queries with relational theory.

Chapter Summary

3.1 Basic SQL Queryings

SQL uses SELECT-FROM-WHERE syntax for data retrieval. SELECT specifies attributes (projection); FROM identifies the relation; WHERE applies filtering conditions (selection). DISTINCT eliminates duplicate rows. Logical operators AND, OR, NOT combine conditions. Special predicates: BETWEEN (range), IN (list membership), LIKE (pattern matching with % and _), IS NULL.

3.2 Arithmetic and SQL Functions

- Arithmetic Operators: +, -, *, / applied to numeric columns or constants. NULL in arithmetic yields NULL.
- Date/Time Functions: CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP, YEAR(), MONTH(), DAY().
- Numeric Functions: ABS(), ROUND(), CEIL(), FLOOR(), POWER(), MOD().
- String Functions: UPPER(), LOWER(), LENGTH(), SUBSTRING(), TRIM(), CONCAT().
- Conversion Functions: CAST(), CONVERT() for type conversion between data types.

3.3 Creating Tables with Relationships

Tables are created with CREATE TABLE, specifying attributes, data types, and constraints. PRIMARY KEY ensures uniqueness and non-nullability. FOREIGN KEY establishes relationships and enforces referential integrity. Many-to-many relationships require a junction (intermediate) table with a composite primary key. ON DELETE CASCADE/SET NULL/RESTRICT define behavior when referenced tuples are deleted.

3.4 Integrity Constraints in SQL

- NOT NULL: Prevents null values in a column.
- UNIQUE: Ensures all values in a column are distinct (allows NULL unlike PRIMARY KEY).
- CHECK: Enforces application-specific conditions (e.g., Salary > 0).
- DEFAULT: Specifies a default value if none is provided.
- PRIMARY KEY: Combines NOT NULL + UNIQUE; entity integrity.
- FOREIGN KEY: Referential integrity; links child table to parent table.

3.5 & 3.6 Nested Queries and Subqueries

- Single-Row Subqueries: Returns one value; used with =, >, <, etc. (e.g., finding employees earning above average salary).
- Multiple-Row Subqueries: Returns multiple values; used with IN, ANY, ALL.
- EXISTS: Tests whether a subquery returns at least one tuple.
- Correlated Subqueries: Subquery references columns from the outer query; executed once per outer tuple.
- Subqueries in FROM Clause: Treated as derived tables with an alias.
- Subqueries in SELECT Clause: Compute scalar values displayed alongside each tuple.

3.7 Grouping and 3.8 Aggregation

GROUP BY partitions tuples into groups based on attribute values; aggregate functions operate per group. Rules: non-aggregated attributes in SELECT must appear in GROUP BY. HAVING filters groups after aggregation (like WHERE for groups). Aggregate functions ignore NULL except COUNT(*). Common functions: COUNT, SUM, AVG, MIN, MAX. DISTINCT within aggregates counts unique values.

3.9 Ordering and Joins

ORDER BY sorts the result by one or more attributes (ASC by default, DESC for descending). Applied after all other clauses. NULL treatment depends on DBMS. Join types:

- Inner Join: Only matching tuples from both tables.
- Equi Join: Inner join with equality condition; join condition in WHERE clause.
- Natural Join: Automatic join on common attribute names; eliminates duplicate columns.
- Left Outer Join: All tuples from left table + matching from right (NULL for no match).
- Right Outer Join: All tuples from right table + matching from left (NULL for no match).
- Full Outer Join: All tuples from both tables; NULL where no match.
- Cross Join: Cartesian product; every combination.
- Self-Join: Table joined with itself using aliases; useful for hierarchical relationships.

3.10 Views

A view is a virtual table defined by a SELECT query. It does not store data physically. Advantages: security (hide sensitive columns), query simplification, logical data independence. Updatable Views: Based on single table, no DISTINCT/GROUP BY/aggregate/joins. Changes propagate to base table. Non-Updatable Views: Read-only; involve joins, aggregates, GROUP BY, or set operations. WITH CHECK OPTION prevents updates that would violate the view condition.

3.11 Relational Set Operations

- UNION: All distinct tuples from either result set (duplicate elimination).
- UNION ALL: All tuples including duplicates.
- INTERSECT: Tuples appearing in both result sets.
- EXCEPT/MINUS: Tuples in first result but not in second.

All set operations require union-compatible relations. ORDER BY can only appear once at the end.

Review Questions

Section A: Conceptual Questions

1. Explain the logical order of evaluation of SQL clauses: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY. Why can aggregate functions not appear in the WHERE clause?
2. Differentiate between WHERE and HAVING clauses. When would you use each? Provide a concrete query where using WHERE instead of HAVING (or vice versa) would produce an incorrect result.
3. What is the difference between a correlated subquery and a non-correlated subquery? Describe the performance implications of each. When would you prefer a subquery over a join?
4. Explain the conditions under which a SQL view is updatable. For each condition (no DISTINCT, single table, no GROUP BY, etc.), explain the underlying reason why the condition is necessary for updates to be propagated correctly.
5. Compare INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, and FULL OUTER JOIN using a scenario with Employees and Departments where some employees have no department and some departments have no employees. What tuples appear in the result of each join type?
6. Describe the five aggregate functions (COUNT, SUM, AVG, MIN, MAX). What happens when NULL values are present? How does COUNT(*) differ from COUNT(attribute)? How does DISTINCT affect these functions?
7. Explain the EXISTS and NOT EXISTS operators. How do they differ from IN and NOT IN? Can queries using IN always be rewritten using EXISTS? Provide an example transformation.
8. What are the three SQL set operations (UNION, INTERSECT, EXCEPT)? What is union-compatibility? How does UNION differ from UNION ALL? When would UNION ALL be preferred?
9. Describe the LIKE operator with its wildcard characters. Write SQL queries to find: names starting with 'A', names ending with 'son', names containing 'ar' in the middle, and exactly 5-character names.
10. Explain self-joins with a concrete example involving an employee-manager hierarchy. Why must table aliases be used? Write SQL to find all employees and their manager's name.

Section B: Analytical Problems

Use the following schema: EMPLOYEE(EmpID, Name, Salary, DeptID, ManagerID) and DEPARTMENT(DeptID, DeptName, Budget).

11. Write SQL queries to: (a) Find employees earning more than their manager. (b) Find departments with more than 3 employees. (c) Find the second highest salary. (d) Find employees who earn above their department's average salary.
12. Write queries using subqueries: (a) Find employees in the same department as 'Alice'. (b) Find departments where every employee earns above 40,000. (c) Find employees who are managers of at least one other employee.
13. Create a view DEPT_SUMMARY that shows DeptID, DeptName, number of employees, average salary, and maximum salary per department. Then write a query on this view to find departments with an average salary above 50,000.
14. Write SQL to implement: (a) UNION of all employee names and department names. (b) INTERSECT to find IDs that appear in both employees and some other table. (c) EXCEPT to find departments with no employees.
15. Write queries using all types of joins between EMPLOYEE and DEPARTMENT to demonstrate: which employees appear only in INNER JOIN results vs. LEFT OUTER JOIN results. Explain the difference.

Section C: Applied and Design Questions

16. Design a complete SQL schema for an Online Learning Platform (Students, Courses, Instructors, Enrollments, Assignments, Submissions). Include all constraints (PK, FK, CHECK, NOT NULL, DEFAULT). Create views for: 'courses by instructor', 'student progress', and 'top performers'.
17. A retail database contains Products(PID, Name, Price, Stock, CategoryID) and Sales(SaleID, PID, Quantity, SaleDate, CustomerID). Write SQL to generate: (a) Monthly sales report. (b) Top 10 best-selling products. (c) Products never sold. (d) Average sale value per customer.
18. Discuss the performance implications of using subqueries vs. joins vs. CTEs (Common Table Expressions) for complex queries. In what situations does the query optimizer treat them equivalently? When might explicit joins be faster than correlated subqueries?

19. Create a stored procedure (or write equivalent SQL) that: accepts a department ID, returns all employees in that department ordered by salary descending, and computes a running total of salaries. Discuss how SQL window functions (RANK, ROW_NUMBER, SUM OVER) could simplify this.
20. A bank requires a query to detect potentially fraudulent accounts: accounts with more than 10 transactions in a single day OR transactions above three times the account's average transaction amount. Write the complete SQL query and create a non-updatable view for fraud monitoring.

CHAPTER - 4

SCHEMA REFINEMENT

Chapter Overview

This Chapter addresses a fundamental question of relational database design: given a set of attributes, how should they be grouped into relations to avoid problems? The answer is normalization — a systematic theory-driven process of refining schemas to eliminate redundancy and prevent update anomalies. The chapter introduces functional dependency as the core theoretical tool, then defines a hierarchy of normal forms (1NF, 2NF, 3NF, BCNF) with increasingly strong guarantees. It concludes with formal decomposition theory, distinguishing lossless-join decomposition from lossy decomposition, and dependency-preserving decomposition.

Learning Objectives	**By the end of this chapter, the reader will be able to:** 1. Explain insertion, deletion, and update anomalies arising from poorly designed schemas 2. Define functional dependency and identify FDs from a relation schema 3. Distinguish full, partial, and transitive functional dependencies 4. Test a relation for 1NF, 2NF, 3NF, and BCNF compliance and identify violations 5. Decompose a relation into 2NF, 3NF, or BCNF by removing violating dependencies 6. Explain the properties and trade-offs of each normal form 7. Define lossless-join decomposition and verify using the tabular test 8. Explain dependency-preserving decomposition and its importance for constraint enforcement

Section-by-Section Roadmap

Section	Topic	Key Concepts
4.1	**Introduction to Schema Refinement**	Motivation, goals of normalization, bad schema characteristics
4.2	**Purpose of Normalization**	Insertion, update, deletion anomalies; data redundancy problems
4.3	**Functional Dependency**	FD definition, notation ($X \rightarrow Y$), inference rules, types of FDs
4.3.3	**Types of Functional Dependencies**	Full FD, partial FD, transitive FD key distinctions for normal forms
4.4.1	**First Normal Form (1NF)**	Atomicity requirement; eliminating repeating groups and multi-valued attributes
4.4.2	**Second Normal Form (2NF)**	Eliminating partial dependencies on composite primary keys
4.4.3	**Third Normal Form (3NF)**	Eliminating transitive dependencies; every non-key attribute depends on the key

Section	Topic	Key Concepts
4.4.4	**Boyce–Codd Normal Form (BCNF)**	Stricter than 3NF; every determinant must be a candidate key
4.5	**Comparison of Normal Forms**	Side-by-side comparison of 1NF, 2NF, 3NF, BCNF requirements and trade-offs
4.6–4.7	**Decomposition**	Need for decomposition; lossless-join vs lossy decomposition; tabular test
4.8	**Dependency-Preserving Decomposition**	Ensuring FDs can be checked on decomposed relations without joins

4.1 INTRODUCTION TO SCHEMA REFINEMENT

In relational database design, the initial schema obtained from requirements analysis or ER-to-relational mapping may contain **redundancies, anomalies, and inconsistencies**. Schema refinement, commonly known as **normalization**, is the systematic process of restructuring database schemas to improve their quality while preserving information and minimising redundancy.

The primary objective of schema refinement is to design relations that:

- Avoid unnecessary duplication of data
- Prevent undesirable update, insertion, and deletion anomalies
- Maintain data integrity and consistency
- Support efficient query processing

Schema refinement relies heavily on the concept of **functional dependencies**, which formally capture semantic relationships among attributes. By analysing these dependencies and comparing them against a hierarchy of increasingly strict rules called **normal forms**, a designer can identify and eliminate precisely those parts of a schema that cause problems.

4.2 DATA REDUNDANCY AND UPDATE ANOMALIES

Before studying the mechanics of normalisation, it is worth understanding exactly what goes wrong when a schema contains redundancy. The problems are known collectively as **update anomalies** and appear in three forms.

Motivating Example. Consider the following relation that stores students, their departments, and the courses they are enrolled in:

SID	StudentName	Dept	Course	Instructor
1	Anil Kumar	CSE	DBMS	Dr. Rao
1	Anil Kumar	CSE	Operating Systems	Dr. Kumar
1	Anil Kumar	CSE	Computer Networks	Dr. Patel
2	Ravi Sharma	ECE	VLSI Design	Dr. Mehta
2	Ravi Sharma	ECE	Digital Circuits	Dr. Mehta

Table 4.1 STUDENT_COURSE - a single unnormalised relation storing students, departments, and courses.

4.2.1 Update Anomaly

Anil Kumar's department is stored in three separate rows, one for each course he takes. If he transfers from CSE to IT, every one of those rows must be updated simultaneously. Missing even a single row leaves the database in an inconsistent state in which Anil appears to belong to two departments at once.

4.2.2 Insertion Anomaly

Suppose a new student, Priya Nair (SID = 3), has been admitted to the Mathematics department but has not yet enrolled in any course. There is no row that can be inserted because Course and Instructor form part of the composite key, a student record cannot exist in this relation without at least one associated course. The student's departmental information is effectively lost until enrolment.

4.2.3 Deletion Anomaly

If Ravi Sharma drops Digital Circuits, his last remaining course with Dr. Mehta, deleting that row also destroys the only stored fact that Ravi is in the ECE department. Once the row is gone, that departmental information is permanently lost from this relation.

All three anomalies share the same root cause: the relation stores information about more than one distinct concept- students, departments, courses, and enrolments in a single table. Normalisation separates these concepts into individual relations.

4.3 FUNCTIONAL DEPENDENCIES

4.3.1 Definition of Functional Dependency

A **functional dependency (FD)** is a constraint between two sets of attributes in a relation. It is not derived from any single snapshot of the data but is a semantic constraint reflecting real-world facts.

Formal Definition. Given a relation R, an attribute set X **functionally determines** an attribute set Y, written $X \rightarrow Y$, if and only if, for every valid instance of R, whenever two tuples t_1 and t_2 agree on X they are also guaranteed to agree on Y:

$X \rightarrow Y$ iff $t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]$ for all $t_1, t_2 \in R$

This means that knowing the value of X is sufficient to determine the value of Y uniquely. Functional dependencies must be declared by the designer on the basis of domain knowledge, not inferred mechanically from the data.

4.3.2 Illustrative Example

Consider a STUDENT relation with attributes SID, Name, and Dept. In any university, a student identification number uniquely identifies one student, so the following dependencies hold:

SID → Name (the student's name is determined by their SID)

SID → Dept (the department is determined by the SID)

The reverse dependencies do not hold: Name → SID fails because two students could share a name; Dept → Name fails because a department contains many students.

4.3.3 Types of Functional Dependencies

(a) Trivial Functional Dependency

A dependency X → Y is **trivial** if $Y \subseteq X$, that is, if Y is already a subset of X. Trivial dependencies carry no useful information because Y is already contained in the determinant.

Example: {SID, Name} → Name (trivial - Name is part of the left-hand side)

(b) Non-Trivial Functional Dependency

A dependency X → Y is **non-trivial** if $Y \not\subset X$, that is, Y contains at least one attribute not present in X. Non-trivial dependencies are the ones that carry meaningful semantic content.

Example: SID → Name (non-trivial - Name does not appear in {SID})

(c) Fully Functional Dependency

Y is **fully functionally dependent** on X if Y depends on the whole of X and not on any proper subset of X. Full dependency matters most when X is a composite key.

Example. In an ENROLMENT relation with composite key (SID, CID), the attribute Grade is fully functionally dependent on (SID, CID) because knowing only SID or only CID is not enough to determine a student's grade in a course, both are required:

(SID, CID) → Grade (fully functional)

(d) Partial Functional Dependency

Y is **partially functionally dependent** on X if Y depends on only a proper subset of the composite key X. Partial dependencies indicate that the attribute does not truly need the entire key.

(SID, CID) → StudentName (partial - only SID is needed for StudentName)

(SID, CID) → CourseName (partial - only CID is needed for CourseName)

(e) Transitive Functional Dependency

A dependency X → Z is **transitive** if there exists an intermediate attribute set Y such that X → Y and Y → Z hold but Y → X does not (Y is not a superkey). Z depends on X only indirectly, through Y.

Example: EID → DeptID and DeptID → DeptName

Therefore EID → DeptName (transitively, through DeptID)

4.4 NORMAL FORMS BASED ON FUNCTIONAL DEPENDENCIES

Normalisation organises relations into a hierarchy of **normal forms**. Each level removes a specific category of redundancy. A relation that satisfies the rules of a given level automatically satisfies all lower levels. The progression is:

Unnormalized → 1NF → 2NF → 3NF → BCNF

Each transition involves identifying a specific type of problematic dependency and decomposing the relation to eliminate it.

4.4.1 FIRST NORMAL FORM (1NF)

Definition

A relation is in **First Normal Form (1NF)** if:

- All attribute values are atomic (indivisible)

- There are no repeating groups, comma-separated lists, nested relations, or multivalued attributes

1NF is the baseline requirement for a valid relation in the relational model. It captures the idea that every cell in a table must hold exactly one value, not a set or a list.

Example 4.1 Violation of 1NF and Its Repair

Step 1: Unnormalised relation- the Courses attribute contains a comma-separated list of values.

SID	StudentName	Courses
1	Anil Kumar	DBMS, Operating Systems, Computer Networks
2	Ravi Sharma	VLSI Design, Digital Circuits
3	Priya Nair	Calculus, Linear Algebra

Table 4.2 STUDENT-

Courses is multivalued, violating 1NF.

The Courses column violates 1NF because it stores multiple course names in a single cell. Queries such as 'find all students enrolled in DBMS' require splitting and scanning strings rather than a clean indexed lookup- a clear sign that atomic values are absent.

Step 2: Converted to 1NF- each course occupies its own row. The primary key becomes the composite (SID, Course).

SID	StudentName	Course
1	Anil Kumar	DBMS
1	Anil Kumar	Operating Systems
1	Anil Kumar	Computer Networks

2	Ravi Sharma	VLSI Design
2	Ravi Sharma	Digital Circuits
3	Priya Nair	Calculus
3	Priya Nair	Linear Algebra

Table 4.3 STUDENT converted to 1NF.

Every cell now holds a single atomic value. The student name appears on multiple rows- this redundancy will be addressed when the relation is brought into 2NF.

4.4.2 SECOND NORMAL FORM (2NF)

Definition

A relation is in **Second Normal Form (2NF)** if:

- It is in 1NF
- Every non-prime attribute is fully functionally dependent on the entire primary key- no non-prime attribute depends on only a part of a composite primary key

A **non-prime attribute** is any attribute that does not appear in any candidate key of the relation. 2NF is relevant only when the primary key is composite; a relation with a single-attribute primary key is automatically in 2NF once it satisfies 1NF.

Example 4.2 - Violation of 2NF and Decomposition

Consider the ENROLMENT relation below, which is in 1NF but contains partial dependencies.

SID	CID	StudentName	CourseName	Grade
1	C01	Anil Kumar	DBMS	A
1	C02	Anil Kumar	Operating Systems	B+

2	C01	Ravi Sharma	DBMS	A-
3	C03	Priya Nair	Calculus	A

Table 4.4 ENROLMENT in 1NF- composite primary key (SID, CID).

The functional dependencies present are:

SID → StudentName (partial- depends on part of the composite key)

CID → CourseName (partial- depends on part of the composite key)

(SID, CID) → Grade (fully functional- requires the entire key)

Because StudentName and CourseName each depend on only part of the composite key, the relation is not in 2NF. The consequence is visible: Anil Kumar's name appears twice, and DBMS appears twice. Inserting a new course requires providing a student; deleting the last student in a course destroys the course name.

Decomposition into 2NF- move each partial dependency into its own relation:

SID	StudentName
1	Anil Kumar
2	Ravi Sharma
3	Priya Nair

Table 4.5 STUDENT(SID, StudentName)- primary key: SID

CID	CourseName
C01	DBMS
C02	Operating Systems
C03	Calculus

Table 4.6 COURSE(CID, CourseName)- primary key: CID

SID	CID	Grade
1	C01	A
2	C02	B+
3	C01	A-
4	C03	A

Table 4.7 ENROLMENT(SID, CID, Grade)- primary key: (SID, CID)

In each decomposed relation, every non-prime attribute depends on the complete primary key. Student names and course names are now stored exactly once, so updates to either require changing only a single row.

4.4.3 THIRD NORMAL FORM (3NF)

Definition

A relation is in **Third Normal Form (3NF)** if:

- It is in 2NF
- No non-prime attribute is transitively dependent on the primary key

A relation in 2NF may still contain redundancy if one non-prime attribute determines another non-prime attribute. When such a chain- key → A → B- exists, B depends on the key only indirectly through A. Removing these transitive chains brings the relation into 3NF.

Example 4.3 Violation of 3NF and Decomposition

EID	EmpName	DeptID	DeptName	DeptLocation
E01	Anil Kumar	D01	Computer Science	Block A
E02	Ravi Sharma	D01	Computer Science	Block A
E03	Priya Nair	D02	Electronics	Block B
E04	Sonia Gupta	D02	Electronics	Block B

Table 4.8 EMPLOYEE in 2NF- transitive dependency present.

The primary key is EID and the functional dependencies are:

EID → EmpName, DeptID

DeptID → DeptName, DeptLocation

DeptName and DeptLocation are determined by DeptID, which is itself determined by EID. These attributes are therefore **transitively dependent** on EID through DeptID. The result is that 'Computer Science Block A' appears in two rows. Moving a department requires updating multiple employee rows.

Decomposition into 3NF:

EID	EmpName	DeptID
E01	Anil Kumar	D01
E02	Ravi Sharma	D01
E03	Priya Nair	D02
E04	Sonia Gupta	D02

Table 4.9 EMPLOYEE(EID, EmpName, DeptID)- primary key: EID

DeptID	DeptName	DeptLocation
D01	Computer Science	Block A
D02	Electronics	Block B

Table 4.10 DEPARTMENT(DeptID, DeptName, DeptLocation)- primary key: DeptID

Department information is now stored exactly once. Relocating a department requires updating a single row in DEPARTMENT, regardless of how many employees belong to it. Both relations are in 3NF.

4.4.4 BOYCE–CODD NORMAL FORM (BCNF)

Definition

A relation R is in **Boyce–Codd Normal Form (BCNF)** if, for every non-trivial functional dependency X → Y that holds in R, X is a **superkey** of R. In other words, the only non-trivial functional dependencies allowed are those whose determinant is a superkey.

BCNF is a stricter version of 3NF. The difference becomes apparent only in relations with overlapping candidate keys- a situation that arises less frequently than partial or transitive dependencies but can still produce redundancy. Every relation in BCNF is automatically in 3NF, but not vice versa.

Example 4.4 A Relation in 3NF but Not in BCNF

A university maintains the following policy: each course section is taught by exactly one instructor; each instructor uses exactly one room. The TEACHING relation records which instructor teaches which course in which room.

Course	Instructor	Room
DBMS	Dr. Rao	R-101
DBMS	Dr. Kumar	R-102
Operating Systems	Dr. Kumar	R-102
Computer Networks	Dr. Patel	R-103

Table 4.11 TEACHING - in 3NF but not in BCNF.

The functional dependencies are:

(Course, Instructor) → Room - composite candidate key

Instructor → Room - each instructor has a fixed room

The dependency Instructor → Room violates BCNF because Instructor is not a superkey of TEACHING. Dr. Kumar's room assignment R-102 is therefore stored in two rows, creating redundancy. If Dr. Kumar moves to R-201, two rows must be updated.

Decomposition into BCNF:

Instructor	Room
Dr. Rao	R-101
Dr. Kumar	R-102
Dr. Patel	R-103

Table 4.12 INSTRUCTOR_ROOM(Instructor, Room)- primary key: Instructor

Course	Instructor
DBMS	Dr. Rao
DBMS	Dr. Kumar
Operating Systems	Dr. Kumar
Computer Networks	Dr. Patel

Table 4.13 COURSE_INSTRUCTOR(Course, Instructor)- primary key: (Course, Instructor)

Every non-trivial functional dependency in each decomposed relation now has a superkey on its left-hand side. Both relations are in BCNF.

Note: Decomposing to BCNF is always lossless-join but may not be dependency-preserving. If a functional dependency spans two decomposed relations, enforcing it requires a join. The designer must evaluate this trade-off.

4.5 DECOMPOSITION OF RELATIONS

4.5.1 Need for Decomposition

Normalisation repeatedly splits a single relation into two or more smaller relations. Not every split is acceptable. Two essential properties determine whether a decomposition is correct: the **lossless-join property** and the **dependency-preservation property**.

4.5.2 Lossless-Join Decomposition

A decomposition of R into R_1 and R_2 is **lossless-join** if the natural join $R_1 \bowtie R_2$ recovers exactly the original relation R - no spurious tuples are introduced and no tuples are lost.

A sufficient condition for lossless join when R is decomposed into R_1 and R_2 is that the attributes common to R_1 and R_2 functionally determine at least one of the two relations:

$$(R_1 \cap R_2) \rightarrow (R_1 - R_2) \quad \text{or} \quad (R_1 \cap R_2) \rightarrow (R_2 - R_1)$$

The decomposition in Example 4.3 is lossless because DeptID- the shared attribute- is the primary key of DEPARTMENT. The dependency DeptID →

DeptName, DeptLocation ensures that joining on DeptID will restore the original rows exactly without adding phantom rows.

A **lossy** decomposition is one where joining the pieces produces extra rows not present in the original. These are called *spurious tuples* and they represent facts that were never in the original data. Any lossy decomposition must be rejected.

4.5.3 Dependency-Preserving Decomposition

A decomposition is **dependency-preserving** if every functional dependency of the original relation can be checked by examining the constraints on a single decomposed relation, without the need to join any two relations.

Dependency preservation is important in practice. If enforcing a business rule requires joining two tables, the database engine must perform that join every time a row is inserted or updated, an expensive operation. When the decomposition is dependency-preserving, constraints can be verified locally, which is faster and simpler.

Example 4.5 - Testing Dependency Preservation

Original relation: R(A, B, C) with functional dependencies A → B and B → C.

Decomposition: R_1(A, B) and R_2(B, C).

- A → B can be checked in R_1 alone.
- B → C can be checked in R_2 alone.

Both dependencies are preserved without requiring a join. This is a dependency-preserving decomposition.

4.6 SURROGATE KEYS

A **surrogate key** is an artificially generated primary key with no inherent business meaning. It is typically a sequential integer (auto-increment) or a universally unique identifier (UUID) assigned by the database engine when a new row is created.

Surrogate keys are introduced when the natural candidate key is cumbersome. for example, when it is a long multi-attribute composite, when its value might change over time due to business rule changes, or when it contains sensitive data that should not appear as a foreign key in other tables.

Example 4.6 - Natural Key versus Surrogate Key

Suppose the natural identifier for a student is their roll number, formatted as '2024-CSE-0042'. This value might be reassigned if the student transfers programme. A surrogate key insulates internal references from such changes.

StudentID (surrogate)	RollNo (natural)	StudentName	Programme
1001	2024-CSE-0042	Anil Kumar	B.Tech CSE
1002	2024-ECE-0015	Ravi Sharma	B.Tech ECE
1003	2024-MA-0007	Priya Nair	B.Sc Mathematics

Table 4.14 STUDENT with both a surrogate primary key and a natural key.

Even if Anil Kumar's RollNo is corrected or reassigned, all foreign-key references that use StudentID = 1001 remain valid. The natural key RollNo should still be declared as a UNIQUE constraint to preserve its identifying role within the business domain.

4.7 MULTIVALUED DEPENDENCIES AND FOURTH NORMAL FORM

4.7.1 Definition of Multivalued Dependency

A **multivalued dependency (MVD)** captures a kind of redundancy that functional dependencies cannot express. It arises when one attribute independently determines two separate sets of values for two other attributes, and those two sets have no relationship with each other.

Formal Definition. $X \rightarrow\rightarrow Y$ (read: X multi-determines Y) holds in R if, for every pair of tuples that agree on X, swapping their Y values produces tuples that are also in R. Equivalently, for any given value of X, the set of Y values is completely independent of the values of the remaining attributes of R.

Example 4.7 - Multivalued Dependency in Practice

A faculty member can have several areas of specialisation (Skill) and can also supervise several research projects (Project). Skills and projects are entirely independent- adding a new skill implies nothing about a project, and vice versa.

FID	Skill	Project
F01	Databases	Smart Library
F01	Databases	Campus IoT
F01	Machine Learning	Smart Library
F01	Machine Learning	Campus IoT
F02	Networks	5G Research

Table 4.15 FACULTY_SKILL_PROJECT- multivalued dependencies cause redundancy.

The MVDs that hold here are FID →→ Skill and FID →→ Project. Because skills and projects are independent, every combination of a given faculty member's skill and project must appear as a row. Adding a new skill for F01 requires inserting two new rows- one per existing project- and adding a new project similarly requires one new row per existing skill. This is the characteristic redundancy that Fourth Normal Form eliminates.

4.7.2 Fourth Normal Form (4NF)

Definition

A relation R is in **Fourth Normal Form (4NF)** if:

- It is in BCNF
- For every non-trivial multivalued dependency X →→ Y in R, X is a superkey of R

The relation FACULTY_SKILL_PROJECT above is in BCNF (it has no problematic functional dependencies) but violates 4NF because FID →→ Skill holds and FID is not a superkey. The fix is to separate the two independent multivalued facts into their own relations.

Decomposition into 4NF:

FID	Skill
F01	Databases
F01	Machine Learning
F02	Networks

Table 4.16 FACULTY_SKILL(FID, Skill) - primary key: (FID, Skill)

FID	Project
F01	Smart Library
F01	Campus IoT
F02	5G Research

Table 4.17 FACULTY_PROJECT(FID, Project) - primary key: (FID, Project)

Each relation now records only one multivalued fact per faculty member. Adding a new skill for F01 requires inserting one row in FACULTY_SKILL only; FACULTY_PROJECT is completely unaffected.

4.8 FIFTH NORMAL FORM(5NF)

Fourth Normal Form handles the case where two independent sets of values produce redundancy. Fifth Normal Form (5NF) addresses a more subtle variant-redundancy caused by **join dependencies** that involve three or more relations.

4.8.1 Definition

A relation R is in **Fifth Normal Form (5NF)**, also called **Project-Join Normal Form (PJNF)**, if every join dependency in R is implied by the candidate keys of R. Equivalently, R cannot be losslessly decomposed into any smaller collection of relations unless the decomposition is forced by the candidate keys themselves.

Example 4.8 - Fifth Normal Form

A wholesaler records which supplier sells which part, which project uses which part, and which supplier is contracted with which project. The three facts are mutually constrained: if Supplier S1 sells Part P1 AND Project PJ1 uses Part P1 AND S1 is contracted with PJ1, then the combination (S1, P1, PJ1) must appear as a row. This three-way constraint cannot be expressed as any functional or multivalued dependency.

Supplier	Part	Project
S1	P1	PJ1
S1	P1	PJ2
S1	P2	PJ1
S2	P1	PJ1

Table 4.18 SUPPLY(Supplier, Part, Project)- exhibits a join dependency.

The join dependency here is ⋈{(Supplier, Part), (Supplier, Project), (Part, Project)}. Decomposing into three relations gives:

Supplier	Part
S1	P1
S1	P2
S2	P1

Table 4.19 SUPPLIER_PART(Supplier, Part)

Supplier	Project
S1	PJ1
S1	PJ2
S2	PJ1

Table 4.20 SUPPLIER_PROJECT(Supplier, Project)

Table 4.21

Part	Project
P1	PJ1
P1	PJ2
P2	PJ1

PART_PROJECT(Part, Project)

The original SUPPLY relation can be recovered exactly by joining these three relations on their shared attributes. The 5NF decomposition ensures that no redundant rows remain and no spurious tuples are introduced.

Note: Fifth Normal Form violations are rare in practice. They arise only in complex many-to-many-to-many associations. Most production schemas are normalised to BCNF or 3NF, and 5NF is considered primarily for theoretical completeness.

4.9 COMPARISON OF NORMAL FORMS

The table below consolidates the key properties of each normal form discussed in this chapter.

Normal Form	Built On	Condition Eliminated	Symptom if Violated
1NF	—	Multivalued / composite values	Comma-separated lists in a column
2NF	1NF	Partial functional dependency	Non-prime attribute depends on part of composite key
3NF	2NF	Transitive functional dependency	Non-prime attribute depends on another non-prime
BCNF	3NF	Non-superkey functional determinant	FD whose left-hand side is not a superkey
4NF	BCNF	Non-trivial multivalued dependency	Two independent repeating groups in one relation
5NF	4NF	Non-key join dependency	Three-way cyclic constraint across entities

Table 4.22 Summary of Normal Forms 1NF through 5NF.

Normalisation is not a one-size-fits-all prescription. Most practitioners target 3NF or BCNF because these levels remove the anomalies most commonly encountered while keeping the schema straightforward. Higher normal forms (4NF, 5NF) are applied selectively when the domain genuinely exhibits multivalued or join dependencies.

It is also worth noting that normalisation is a design tool, not a performance guarantee. Fully normalised schemas sometimes demand joins that a denormalised design could avoid. A skilled designer knows when to apply normalisation and when a measured degree of redundancy is acceptable for performance- and documents that decision explicitly.

Chapter Summary

4.1 Introduction to Schema Refinement

Schema refinement (normalization) is the systematic process of restructuring database schemas to eliminate redundancy, anomalies, and inconsistencies while preserving information. Goals: avoid data duplication, prevent update/insertion/deletion anomalies, maintain data integrity, support efficient queries.

4.2 Anomalies from Data Redundancy

- Update Anomaly: Updating the same fact in multiple places can cause inconsistency if any copy is missed.
- Insertion Anomaly: Cannot insert certain data without inserting other unrelated data.
- Deletion Anomaly: Deleting a tuple may unintentionally remove valuable, unrelated information.

4.3 Functional Dependencies (FD)

A functional dependency X -> Y means: for any two tuples with the same X values, they must have the same Y values. X functionally determines Y. Types:

- Trivial FD: Y is a subset of X (e.g., {SID, Name} -> Name). Always holds.
- Non-Trivial FD: Y is not a subset of X (e.g., SID -> Name).
- Fully Functional Dependency: Y depends on the entire
- X, not any proper subset. Important for 2NF.
- Partial Functional Dependency: Y depends on only part of a composite key. Violates 2NF.
- Transitive Dependency: X -> Y -> Z (Y is not a key). Violates 3NF.

4.4 Normal Forms

- 1NF (First Normal Form): All attribute values are atomic; no repeating groups or multi-valued attributes. Every relation in RDBMS must be in 1NF.
- 2NF (Second Normal Form): In 1NF + no partial functional dependencies. Every non-key attribute depends on the entire primary key. Only relevant when the primary key is composite.
- 3NF (Third Normal Form): In 2NF + no transitive dependencies. Every non-key attribute depends directly on the primary key, not on another non-key attribute.

- BCNF (Boyce-Codd Normal Form): For every non-trivial FD X -> Y, X must be a superkey. Stricter than 3NF; eliminates more anomalies but may not always preserve all dependencies.
- 4NF (Fourth Normal Form): In BCNF + no non-trivial multivalued dependencies (MVDs). Handles independence between multi-valued facts.
- 5NF (Fifth Normal Form / PJNF): In 4NF + all join dependencies are implied by candidate keys. Cannot be decomposed further without loss.

4.5 Multivalued Dependencies (MVD)

An MVD X ->-> Y exists when for a given X value, the set of Y values is independent of other attributes. Example: STUDENT_SKILL(SID, Skill, Hobby): SID ->-> Skill and SID ->-> Hobby because skills and hobbies are independent facts about a student. 4NF decomposes relations with MVDs into separate relations.

4.6 Decomposition

Normalization requires decomposing relations into smaller, better-structured relations. Two essential properties of correct decomposition:

- Lossless Join Property: Natural joining of decomposed relations must reconstruct the original relation exactly (no spurious tuples). Test: common attribute(s) must functionally determine at least one side. Condition: (X1 intersection X2) -> X1 OR (X1 intersection X2) -> X2.
- Dependency Preservation: All FDs of the original relation can be enforced in the decomposed relations without requiring a join.

4.7 Surrogate Keys

Artificial, system-generated identifiers (auto-increment integers, UUIDs) with no business meaning. Advantages: stable (not affected by business rule changes), simplify joins, improve performance. Disadvantages: do not carry semantic meaning, require additional columns.

Review Questions

Section A: Conceptual Questions

1. Explain the three data anomalies (update, insertion, deletion) that arise from data redundancy. Using the relation STUDENT_COURSE(SID, SName, CID, CName, Grade, Instructor), demonstrate each anomaly with a concrete example.
2. Define functional dependency formally. What is the difference between a trivial and a non-trivial functional dependency? Between a full and a partial functional dependency? Between a direct and a transitive dependency?
3. State the definition of each normal form: 1NF, 2NF, 3NF, BCNF, 4NF, and 5NF. For each, identify the specific type of anomaly or dependency it eliminates.
4. What is BCNF? How does it differ from 3NF? Can a relation be in 3NF but not in BCNF? Provide an example relation that is in 3NF but violates BCNF. What is the consequence of not achieving BCNF?
5. Define lossless join decomposition. How is it tested formally? Provide an example of a decomposition that is lossless and one that is lossy. Explain the consequences of a lossy decomposition.
6. What is dependency preservation in decomposition? Why is it important? Provide an example where achieving BCNF requires sacrificing dependency preservation. What is the impact of non-preserved dependencies?
7. Define a multivalued dependency (MVD). How does it differ from a functional dependency? Under what conditions does an MVD cause redundancy? Give a real-world example.
8. What is a surrogate key? When would you prefer a surrogate key over a natural primary key? What are the disadvantages of surrogate keys?
9. Compare 3NF and BCNF from the perspective of anomaly prevention and decomposition properties. Under what conditions is BCNF decomposition always lossless? When might 3NF be preferred over BCNF?
10. Explain the process of converting a relation from unnormalized form through 1NF, 2NF, 3NF, and BCNF, step by step. What checks must be performed at each step?

Section B: Analytical Problems

11. Consider the relation R(A, B, C, D, E) with FDs: {A -> B, B -> C, C -> D, A -> E}. (a) Find the closure of A. (b) Identify the primary key. (c) Is R in 2NF? 3NF? BCNF? Justify. (d) Decompose R into BCNF. Verify lossless join property.
12. Given COURSE_OFFERING(CID, Title, InstructorID, InstructorName, Room, Time, DeptID, DeptBudget), with FDs: CID -> {Title, InstructorID, Room, Time, DeptID}; InstructorID -> InstructorName; DeptID -> DeptBudget. (a) Identify all partial and transitive dependencies. (b) Decompose into 3NF. (c) Is your decomposition in BCNF? If not, decompose further.
13. Relation SUPPLY(SupplierID, SupplierName, PartID, PartName, ProjectID, Quantity) with FDs: SupplierID -> SupplierName, PartID -> PartName. (a) What normal form is this relation in? (b) Identify all anomalies with examples. (c) Decompose into 3NF preserving all dependencies.
14. Test whether the following decomposition of R(A, B, C) with FD A -> B into R1(A, B) and R2(A, C) is: (a) Lossless. (b) Dependency preserving. Show all steps including the natural join reconstruction.
15. Given the relation STUDENT_SKILLS(SID, Skill, Language) where SID ->-> Skill and SID ->-> Language (Skill and Language are independent). (a) Is this relation in 3NF? BCNF? Why or why not? (b) Identify the MVD and decompose into 4NF.

Section C: Applied and Design Questions

16. A company's HR department maintains a spreadsheet with columns: EmployeeID, EmployeeName, Department, DeptManager, ManagerPhone, Project, ProjectBudget, HoursWorked. Perform complete normalization from UNF to 3NF, identifying all FDs, performing each decomposition step, and explaining what anomaly is eliminated at each step.
17. Discuss a scenario where normalizing to BCNF would break a dependency that is important for the application. Propose a strategy that balances normalization against practical requirements (e.g., triggers, application-level enforcement, controlled denormalization).
18. Compare the tradeoffs of high normalization (3NF/BCNF) versus controlled denormalization in: (a) OLTP systems (Online Transaction Processing). (b) OLAP systems (Online Analytical Processing / Data Warehouses). Explain why data warehouses often intentionally use denormalized schemas (star schema, snowflake schema).

19. Design a normalized relational schema (at least 3NF) for a Social Media Platform with users, posts, comments, likes, followers, and hashtags. Identify all FDs, demonstrate that your schema is in 3NF, and check for any remaining anomalies.
20. Explain the role of Armstrong's Axioms (Reflexivity, Augmentation, Transitivity) in deriving all implied functional dependencies from a given set of FDs. Using these axioms, prove that the union rule (if X->Y and X->Z, then X->YZ) and decomposition rule are valid.

CHAPTER - 5

TRANSACTION CONCEPT

Chapter Overview

This Chapter addresses how database systems maintain correctness when multiple users access and modify data simultaneously — a fundamental challenge in any multi-user system. The chapter introduces the transaction as the unit of database work, defines the ACID properties that guarantee transaction correctness, and analyses the problems that arise when transactions execute concurrently without control. It then develops the theory of serializability — the standard correctness criterion for concurrent execution — and examines schedule categories based on their recoverability properties. The chapter concludes with isolation levels and testing methods for conflict serializability.

Learning Objectives	**By the end of this chapter, the reader will be able to:** 1. Define a transaction and describe its five execution states (Active, Partially Committed, Committed, Failed, Aborted) 2. Explain and apply the four ACID properties: Atomicity, Consistency, Isolation, Durability 3. Identify concurrency problems: dirty read, lost update, unrepeatable read, phantom read 4. Distinguish serial from non-serial schedules and explain why serializability is the correctness goal 5. Test a schedule for conflict serializability using a precedence (serialization) graph 6. Define view serializability and explain its relationship to conflict serializability

	7. Classify schedules as recoverable, cascadeless, or strict based on their recovery properties 8. Describe SQL isolation levels (Read Uncommitted, Read Committed, Repeatable Read, Serializable) and the anomalies each permits

Section-by-Section Roadmap

Section	Topic	Key Concepts
5.1	**Transaction Concepts**	Transaction definition as logical work unit; read/write operations; states
5.1.1	**Transaction States**	Active → Partially Committed → Committed / Failed → Aborted lifecycle
5.2	**ACID Properties**	Atomicity, Consistency, Isolation, Durability — the correctness guarantees
5.3	**Concurrent Executions**	Why concurrency is needed; interleaving; concurrency anomalies (dirty read, lost update, etc.)
5.3.1–5.3.2	**Schedules**	Serial schedules, non-serial schedules, concurrent schedules; schedule notation
5.4	**Serializability**	Correctness criterion; conflict vs view serializability
5.4.1	**Conflict Serializability**	Conflicting operations; precedence graph; cycle detection test
5.4.2	**View Serializability**	View equivalence; relationship to conflict serializability
5.5	**Recoverability**	Recoverable, cascade less (cascade-free),

Section	Topic	Key Concepts
	of Schedules	and strict schedules
5.5.4	**Transaction Rollback**	Cascade rollback scenario; why strict schedules are preferred
5.6	**Implementation of Isolation**	Isolation levels in SQL; Read Uncommitted, Read Committed, Repeatable Read, Serializable
5.7	**Testing for Serializability**	Conflict testing method using precedence graphs; cycle = non-serializable

5.1 TRANSACTION CONCEPTS

A modern database system is expected to support multiple users accessing and modifying data concurrently while ensuring correctness, consistency, and reliability. To achieve this objective, database systems organize operations into transactions. A transaction represents a logical unit of work that accesses and possibly updates the database. The correctness of database execution is defined primarily in terms of the correct execution of transactions.

5.1.1 Transaction States

During its execution, a transaction passes through several well-defined states, each representing a particular phase in the transaction's life cycle. These states allow the database management system to track execution progress and to apply appropriate recovery actions in case of failures.

Initially, a transaction enters the Active state. In this state, the transaction is executing its read and write operations on database objects. As long as the transaction continues to execute normally, it remains in the Active state.

After the last statement of the transaction has been executed, the transaction enters the Partially Committed state. At this point, all required operations have been completed, but the changes made by the transaction have not yet been permanently recorded on stable storage. A failure at this stage may still prevent the transaction from being completed successfully.

If all updates are successfully recorded and the database system ensures that the effects of the transaction are permanent, the transaction enters the Committed state. Once a transaction is committed, its effects on the database persist even in the presence of subsequent system failures.

If an error occurs during execution, such as arithmetic overflow, integrity constraint violation, or system crash, the transaction moves to the Failed state. A failed transaction cannot proceed further and must be undone.

Following failure detection, the transaction enters the Aborted state. In this state, the database system rolls back all changes made by the transaction, restoring the database to the state that existed before the transaction began execution. After abortion, the transaction may be restarted or terminated permanently.

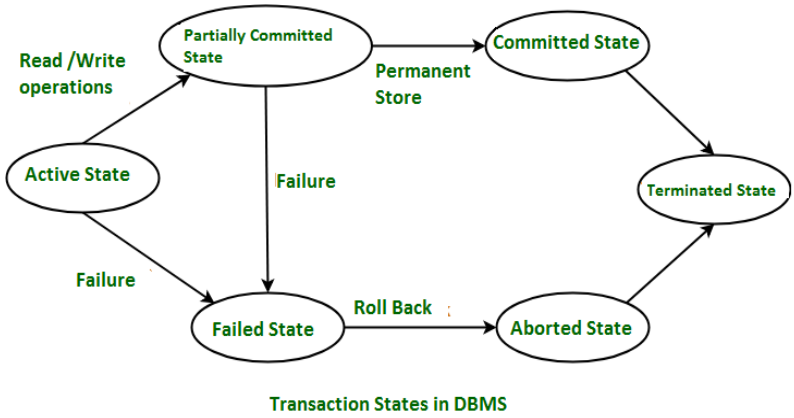


Figure 5.1: Transaction State Diagram

The diagram illustrates the possible transitions between transaction states, emphasizing the role of failure handling and recovery in database systems.

Illustrative Example (Banking Transaction):
Consider a transaction that transfers ₹5,000 from Account A to Account B. The transaction begins in the Active state while reading and updating account balances. After executing the final update statement, it reaches the Partially Committed state. If the system crashes before the updates are written to disk, the transaction moves to the Failed state and is subsequently Aborted, restoring the original balances. Only when the transaction reaches the Committed state do the updated balances become permanent.

5.2 ACID Properties of Transactions

To guarantee correctness and reliability of transaction execution, every transaction must satisfy a set of fundamental properties known as the ACID properties. These properties form the theoretical foundation of transaction management in relational database systems.

Atomicity ensures that a transaction is treated as an indivisible unit of execution. Either all operations of the transaction are performed, or none are performed. Partial execution of a transaction is not permitted. Atomicity is crucial in the presence of failures, as it prevents the database from being left in an inconsistent intermediate state.

Consistency ensures that the execution of a transaction preserves the integrity constraints of the database. A transaction transforms the database from one consistent state to another consistent state. If the database is consistent before the transaction begins, it must remain consistent after the transaction completes successfully.

Isolation ensures that the intermediate states of a transaction are not visible to other concurrently executing transactions. Each transaction appears to execute in isolation, as if it were the only transaction in the system. Isolation prevents anomalies such as dirty reads, lost updates, and inconsistent retrievals.

Durability guarantees that once a transaction has been committed, its effects are permanent and will survive subsequent system failures. Even in the event of power loss or system crash, the results of committed transactions must not be lost.

The relationship between ACID properties and real-world behaviour is summarized in the following table.

Table 5.1: ACID Properties and Real-World Interpretation

Property	Description	Real-World Example
Atomicity	All operations execute completely or not at all	Money not debited unless cash is dispensed
Consistency	Database constraints are preserved	Account balance never becomes negative
Isolation	Concurrent transactions do not interfere with each other	Seat booking avoids double allocation
Durability	Committed changes persist despite failures	Balance update remains after power failure

Example (ATM Withdrawal Scenario):

When a customer withdraws ₹2,000 from an ATM, atomicity ensures that the account is debited only if the cash is successfully dispensed. Consistency ensures that the withdrawal does not violate account constraints such as minimum balance requirements. Isolation ensures that another concurrent transaction does not see intermediate balance values during the withdrawal process. Durability ensures that once the withdrawal is completed, the updated account balance remains stored even if the system crashes immediately afterward.

5.3 CONCURRENT EXECUTIONS

In a database system that serves multiple users, executing transactions strictly one after another leads to poor resource utilization and low system throughput. To overcome this limitation, modern database management systems allow concurrent execution of transactions, where operations from different transactions are interleaved in time. Concurrent execution is a fundamental design principle in DBMSs and is essential for achieving high performance in multi-user environments.

One of the primary benefits of concurrent execution is improved throughput. By overlapping the execution of multiple transactions, the system ensures that the CPU and I/O devices are effectively utilized, thereby reducing idle time.

Concurrent execution also improves average response time, especially for short transactions, since they need not wait for long-running transactions to complete. Furthermore, concurrency enhances scalability, allowing the database system to support a large number of users without significant performance degradation.

Despite these advantages, concurrent execution introduces significant challenges related to data consistency. When transactions execute concurrently, they may access and update the same data items, leading to incorrect results if their interactions are not properly controlled. Typical concurrency anomalies include lost updates, where the effect of one transaction is overwritten by another; dirty reads, where a transaction reads data written by another transaction that has not yet committed; and inconsistent retrievals, where a transaction observes different values of the same data item at different points in time due to concurrent updates. These anomalies violate the consistency guarantees expected from a database system.

To address these challenges, database systems enforce correctness criteria such as serializability, which ensures that the outcome of concurrent execution is equivalent to some serial execution of the same transactions. Concurrency control protocols are designed to balance performance gains from concurrency with the need for correctness.

Illustrative Example (Ticket Booking Scenario):

Consider two users attempting to book the last available seat on a flight. Let Transaction T1 and Transaction T2 both read the number of available seats before either transaction updates it. If both transactions find that one seat is available and proceed to book it, the system may end up allocating the same seat twice. This situation demonstrates how concurrent execution, without proper

control, can lead to incorrect and inconsistent database states. A concurrency control mechanism ensures that once one transaction books the seat, the other transaction is either delayed or aborted.

5.3.1Schedules in Concurrent Execution

To formally analyze concurrent executions, database systems use the concept of a schedule. A schedule represents the exact order in which the operations of multiple transactions are executed by the DBMS. While allowing interleaving of operations, a schedule must preserve the program order of operations within each transaction.

A schedule includes the following operations:

- Read(X)
- Write(X)
- Commit
- Abort

Schedules form the basis for reasoning about correctness, serializability, and recoverability.

5.3.2Types of Schedules

Based on execution order and correctness guarantees, schedules are classified into several types.

(a) Serial Schedule

A serial schedule executes one transaction completely before starting another transaction. There is no interleaving of operations.

Example Serial Schedule (T1 followed by T2):

Time	Transaction T1	TransactionT2
1	Read(X)	
2	Write(X)	
3	Commit	
4		Read(X)
5		Write(X)
6		Commit

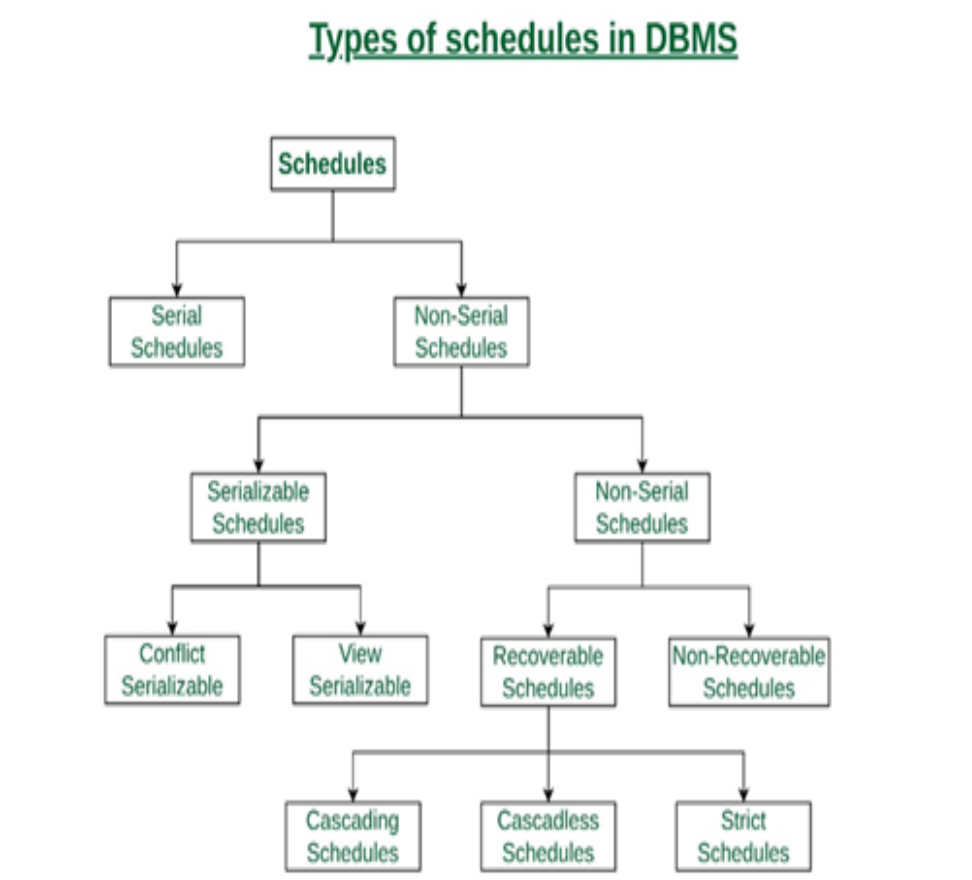


Figure 5.6: Serial Schedule Execution Flow

Serial schedules always preserve database consistency but do not exploit concurrency, resulting in poor performance.

(b) Non-Serial (Concurrent) Schedule: A **non-serial schedule** allows operations of multiple transactions to be interleaved, provided the order of operations within each transaction is preserved.

Example Non-Serial Schedule:

Time	Transaction T1	Transaction T2
1	Read(X)	
2		Read(X)
3	Write(X)	
4		Write(X)
5	Commit	
6		Commit

Figure 5.7: Non-Serial Schedule with Interleaving

Non-serial schedules improve system performance but may lead to anomalies if not properly controlled.

5.4 SERIALIZABILITY

Serializability is the formal correctness criterion for concurrent schedules. It ensures that even though transactions are executed concurrently, the final database state is equivalent to that of some serial execution of the same transactions.

5.4.1 Conflict Serializability

A schedule is conflict serializable if it can be transformed into a serial schedule by repeatedly swapping non-conflicting operations. Two operations conflict if:

1. They belong to different transactions,
2. They access the same data item,
3. At least one of them is a write operation.

Example: Conflict Serializable Schedule

Schedule S:

Time	T1	T2
1	Read(X)	
2		Read(X)
3	Write(X)	
4		Write(X)

To test conflict serializability, a precedence graph is constructed.

Conflict Analysis

Two operations are said to conflict if:

1. They belong to different transactions,
2. They access the same data item,
3. At least one of them is a write operation.

Now, identify the conflicting operation pairs in Schedule S:

1. Write(X) by T1 conflicts with Read(X) by T2, since both access X and one

is a write.

2. Write(X) by T1 conflicts with Write(X) by T2, since both access X and both are write operations.

In both conflicting cases, the operation of T1 occurs before the corresponding operation of T2 in the schedule.

Serial Order Comparison

The relative order of all conflicting operations in Schedule S is consistent with the serial execution:

T1 → T2

By swapping only non-conflicting operations, Schedule S can be transformed into the serial schedule where transaction T1 executes completely before transaction T2.

Conclusion

Since the order of all conflicting operations in the given schedule matches the order in the serial schedule T1 followed by T2, the schedule is conflict serializable.

Final Result:

✓ The schedule is Conflict Serializable

5.4.2 View Serializability

A schedule is view serializable if it is view equivalent to a serial schedule. Two schedules are view equivalent if:

1. The same transaction performs the first read of each data item,
2. Each read reads the value written by the same transaction,
3. The same transaction performs the final write on each data item.

Schedule S

Time	Transaction T1	Transaction T2
1	Read(X)	
2		Read(X)
3	Write(X)	

4	Commit	
5		Commit

View Serializability Analysis

To check whether Schedule S is view serializable, we verify the three conditions of view equivalence with respect to a serial schedule.

Consider the serial schedule **T1 → T2**.

1. Initial Read Condition

- In Schedule S, T1 reads the initial value of X.
- In the serial schedule T1 → T2, T1 also reads the initial value of X.

✓ Initial read condition is satisfied.

2. Read-From Condition

- In Schedule S, T2 reads the value of X written by T1?

  No.
  T2 reads X before T1 writes it, so T2 reads the initial value of X.

- In the serial schedule T1 → T2, T2 would read the value written by T1.

This serial order does **not** preserve the read-from relationship.

Now consider the serial schedule T2 → T1.

- In Schedule S, both T1 and T2 read the initial value of X.
- In the serial schedule T2 → T1, both transactions also read the initial value of X.

✓ Read-from condition is satisfied for serial order T2 → T1.

3. Final Write Condition

- In Schedule S, the final write on X is performed by T1.
- In the serial schedule T2 → T1, the final write on X is also performed by T1.

Final write condition is satisfied.

Conclusion

Since all three view equivalence conditions are satisfied with respect to the serial order **T2** → T1, the given schedule is VIEW SERIALIZABLE.

Final Result:

✓ View Serializable

View serializability is more general than conflict serializability. Some schedules involving **blind writes** may be view serializable but not conflict serializable.

Schedule S

Time	Transaction T1	Transaction T2
1	Write(X)	
2		Write(X)
3	Commit	
4		Commit

View Serializability Analysis

To check view serializability, the following three conditions must be satisfied with respect to some serial schedule.

1. Initial Read Condition

No transaction reads X in the schedule . Hence, this condition is trivially satisfied.

2. Read-From Condition

Since there are no read operations, there are no read-from relationships to preserve. Thus, this condition is also satisfied.

3. Final Write Condition

In Schedule S, the final write on X is performed by T2. In the serial schedule T1 → T2, the final write on X is also performed by T2.

Since the final writer of X is the same in both schedules, this condition is satisfied.

Conflict Serializability Check

The two write operations on X conflict with each other. Conflicting write operations cannot be reordered using conflict swaps without changing the final result.

Hence, the schedule cannot be transformed into a serial schedule using conflict reordering.

Conclusion

- The given schedule satisfies all conditions of view equivalence with the serial order T1 → T2.
- Therefore, the schedule is VIEW SERIALIZABLE.
- However, it is NOT CONFLICT SERIALIZABLE due to conflicting write operations.

Final Result:

✓ View Serializable

✗ Not Conflict Serializable

Important Observation: Every conflict-serializable schedule is view-serializable, but not every view-serializable schedule is conflict-serializable.

5.4.3 Importance of Serializability

Serializability allows database systems to combine the performance benefits of concurrency with the correctness guarantees of serial execution. Most practical concurrency control protocols enforce conflict serializability, since it is easier to test using precedence graphs and simpler to implement.

5.5 RECOVERABILITY OF SCHEDULES

Correctness of concurrent transaction execution is not determined by serializability alone. A schedule must also ensure that failures and rollbacks do not leave the database in an inconsistent state. This requirement is captured through the concept of recoverability.

5.5.1 Recoverable Schedules

A recoverable schedule is one in which a transaction commits only after all transactions whose updates it has read have committed. This guarantees that if a transaction must be rolled back, all dependent transactions can also be rolled

back safely.

In other words, if transaction T2 reads a data item written by transaction T1, then T2 must not commit before T1 commits

Example: Recoverable Schedule

Time	Transaction T1	Transaction T2
1	Write(X)	
2		Read(X)
3	Commit	
4		Commit

Here, T2 reads X written by T1 and commits after T1 commits. If T1 were to fail, T2 could be rolled back safely before committing.

5.5.2 Cascadeless Schedules

A cascade less schedule is a stricter form of recoverable schedule. In such schedules, transactions are allowed to read only committed data. As a result, cascading rollbacks are completely avoided.

Cascade less schedules improve system performance during recovery because rolling back one transaction does not require rolling back others.

Example: Cascade less Schedule

Time	Transaction T1	Transaction T2
1	Write(X)	
2	Commit	
3		Read(X)
4		Commit

Since T2 reads X only after T1 commits, no rollback dependency exists.

5.5.3 Strict Schedules

A strict schedule enforces the strongest constraint. In a strict schedule, a transaction is not allowed to read or write a data item until the transaction that last wrote that item has committed or aborted.

Strict schedules simplify recovery significantly because undo operations never affect committed data.

Example: Strict Schedule

Time	Transaction T1	Transaction T2
1	Write(X)	
2	Commit	
3		Write(X)
4		Commit

Here, T2 accesses X only after T1 commits, ensuring strictness.

5.5.4 Transaction Rollback Scenario

Consider the following non-cascadeless schedule:

Time	Transaction T1	Transaction T2
1	Write(X)	
2		Read(X)
3	Abort	
4		Abort

Since T2 read uncommitted data written by T1, failure of T1 forces rollback of T2, resulting in cascading rollback. Such behavior is undesirable and is prevented in cascadeless and strict schedules.

5.6 IMPLEMENTATION OF ISOLATION

Isolation ensures that concurrent transactions do not interfere with each other. In practice, DBMSs implement isolation using standard isolation levels, each providing a different balance between correctness and performance.

5.6.1 Isolation Levels

The SQL standard defines four isolation levels.

1. **Read Uncommitted:** Transactions may read uncommitted changes made by other transactions. This level provides maximum concurrency but minimal consistency.

2. **Read Committed**: Transactions can read only committed data. Dirty reads are prevented, but non-repeatable reads and phantom reads may occur.
3. **Repeatable Read**: Ensures that if a transaction reads a data item, it will see the same value throughout its execution. Phantom reads may still occur.
4. **Serializable**: The highest isolation level. Transactions are executed as if they were serial, preventing all anomalies.

5.6.2 Isolation Levels and Anomaliess

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Yes	Yes	Yes
Read Committed	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

Table 5.6: Isolation Levels vs Concurrency

Anomalies

5.7 TESTING FOR SERIALIZABILITY

To ensure correctness of concurrent schedules, DBMSs must test whether a schedule is serializable. In practice, conflict serializability testing is widely used.

5.7.1 Conflict Testing Method

Conflict testing is based on identifying conflicting operations and constructing a precedence graph.

Steps:

1. Identify all conflicting operation pairs.
2. Create a node for each transaction.
3. Add a directed edge Ti → Tj if an operation of Ti precedes and conflicts

with an operation of Tj.

4. Check for cycles.

If the graph has no cycles, the schedule is conflict serializable.

5.7.2 Example: Testing a Schedule for Serializability

Consider the following schedule:

Time	Transaction T1	Transaction T2
1	Read(A)	
2		Read(A)
3	Write(A)	
4		Write(A)
5	Commit	
6		Commit

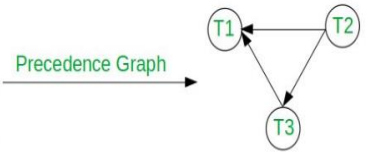
Conflicting operations:

- Write(A) by T1 conflicts with Read(A) by T2
- Write(A) by T1 conflicts with Write(A) by T2

Precedence relation:

T1	T2	T3
	R(A)	
	W(A)	
		R(C)
	W(B)	
		W(A)
		W(C)
R(A)		
R(B)		
W(A)		
W(B)		

Schedule S



- T1 → T2

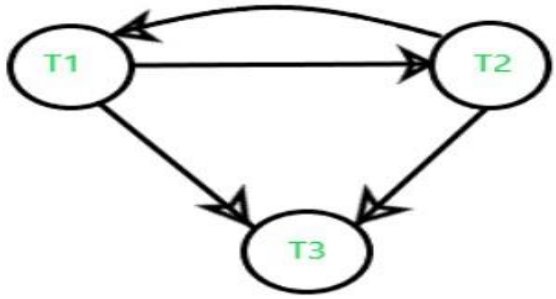


Figure 5.10: Precedence Graph for Serializability Testing

Since the precedence graph contains no cycle, the schedule is conflict serializable and equivalent to the serial order T1 followed by T2.

5.7.3. Examples Covering All Serializability Cases

To clearly distinguish between conflict serializability and view serializability, it is essential to study schedules that fall into different correctness categories. The following examples systematically prove:

1. A schedule that is both conflict serializable and view serializable
2. A schedule that is view serializable but not conflict serializable
3. A schedule that is neither conflict serializable nor view serializable

Example 1: Schedule that is BOTH Conflict Serializable and View Serializable

Consider two transactions T1 and T2 operating on data item X.

Transaction T1:
Read(X), Write(X)

Transaction T2:
Read(X), Write(X)

Schedule S_1

Time	Transaction T1	Transaction T2
1	Read(X)	
2	Write(X)	
3		Read(X)
4		Write(X)
5	Commit	
6		Commit

Conflict Serializability Analysis

Conflicting operation pairs:

- Write(X) by T1 precedes Read(X) by T2
- Write(X) by T1 precedes Write(X) by T2

Precedence relation:

- T1 → T2

Since the precedence graph contains no cycle, Schedule S_1 is conflict serializable, equivalent to the serial order T1 followed by T2.

View Serializability Analysis

1. **Initial Read**
    - T1 reads the initial value of X in both schedules.
2. **Reads-From Relationship**
    - T2 reads the value of X written by T1 in both schedules.
3. **Final Write**
    - T2 performs the final write on X in both schedules.

All view equivalence conditions are satisfied.

Conclusion :

Schedule S_1 is:

- ✓ Conflict Serializable
- ✓ View Serializable

This represents the most common and desirable class of schedules in practice.

Example 2: Schedule that is View Serializable BUT NOT Conflict Serializable

This example involves **blind writes**, which are central to understanding the difference between the two concepts.

Transaction T1: Write(X)

Transaction T2: Write(X)

Schedule S_2

Time	Transaction T1	Transaction T2
1	Write(X)	
2		Write(X)
3	Commit	
4		Commit

Conflict Serializability Analysis

Conflicting operations:

- Write(X) by T1 conflicts with Write(X) by T2

Precedence relations:

- T1 → T2 (because T1's write occurs before T2's write)

If we reverse the order:

- T2 → T1 would be required

Since the conflicting writes cannot be swapped without changing the outcome, the schedule cannot be transformed using conflict swaps.

View Serializability Analysis

1. **Initial Read**
    - No reads occur → condition satisfied.
2. **Reads-From Relationship**
    - No reads occur → condition satisfied.
3. **Final Write**
    - T2 performs the final write in both Schedule S_2 and serial schedule T1 → T2.

Thus, Schedule S_2 is view equivalent to the serial schedule T1 followed by T2.

Conclusion:

Schedule S_2 is:

- ✗ Not Conflict Serializable
- ✓ View Serializable

This example proves that view serializability is more general than conflict serializability.

Example 3: Schedule that is NEITHER Conflict Serializable NOR View Serializable

Consider two transactions T1 and T2 operating on data item X.

Transaction T1: Read(X), Write(X)

Transaction T2: Read(X), Write(X)

Schedule S_3

Time	Transaction T1	Transaction T2
1	Read(X)	
2		Read(X)
3	Write(X)	
4		Write(X)
5	Commit	
6		Commit

Conflict Serializability Analysis

Conflicting pairs:

- T1 Write(X) → T2 Read(X)
- T2 Write(X) → T1 Read(X)

Precedence relations:

- T1 → T2
- T2 → T1

This forms a **cycle** in the precedence graph.

Therefore, Schedule S_3 is not conflict serializable.

View Serializability Analysis

Check view equivalence with possible serial schedules:

Serial Order T1 → T2

- T2 reads X written by T1

But in S_3, T2 reads the initial value of X.

Serial Order T2 → T1

- T1 reads X written by T2

But in S_3, T1 reads the initial value of X.

Thus:

- Reads-from relationships are not preserved.
- Final write order does not match any serial schedule.

Conclusion :

Schedule S_3 is:

- ✗ Not Conflict Serializable
- ✗ Not View Serializable

Such schedules are incorrect and must be prevented by concurrency control mechanisms.

5.8 Lock-Based Protocols

Lock-based protocols are the most widely used concurrency control mechanisms in database systems. In these protocols, a transaction must obtain a lock on a data item before accessing it. Locks prevent other transactions from performing conflicting operations on the same data item simultaneously.

Two fundamental types of locks are used in lock-based protocols:

1. Shared Lock (S-lock)

   A shared lock allows a transaction to read a data item. Multiple transactions may hold shared locks on the same data item simultaneously, provided no transaction holds an exclusive lock on that item.

2. Exclusive Lock (X-lock)

   An exclusive lock allows a transaction to both read and write a data item. When a transaction holds an exclusive lock, no other transaction is

permitted to hold either a shared or exclusive lock on the same data item.

5.8.1. Two-Phase Locking (2PL) Protocol

The Two-Phase Locking (2PL) protocol ensures conflict serializability by dividing the execution of a transaction into two distinct phases:

1. Growing Phase

   During this phase, a transaction may acquire locks but is not allowed to release any lock.

2. Shrinking Phase

   During this phase, a transaction may release locks but is not allowed to acquire any new locks.

Once a transaction releases its first lock, it enters the shrinking phase and cannot obtain additional locks.

Important Property: All schedules produced under the Two-Phase Locking protocol are conflict serializable.

Example: Readers–Writers Problem

The Readers–Writers problem illustrates the use of shared and exclusive locks.

- Multiple reader transactions are allowed to read a data item concurrently using shared locks.
- A writer transaction must obtain an exclusive lock and therefore must wait until all reader transactions release their shared locks.

5.8.1.1 TYPES OF TWO-PHASE LOCKING (2PL) PROTOCOLS

While basic 2PL ensures serializability, it does not by itself prevent cascading rollbacks or deadlocks. To address these issues, several variants of the Two-Phase Locking protocol have been proposed and widely adopted in practice.

1 .Basic Two-Phase Locking (Basic 2PL)

In Basic Two-Phase Locking, a transaction follows the two-phase rule strictly:

- During the growing phase, the transaction may acquire shared and exclusive locks but cannot release any lock.
- During the shrinking phase, the transaction may release locks but cannot acquire any new lock.

Characteristics:

- Guarantees conflict serializability.
- Does not guarantee freedom from cascading rollbacks.
- Deadlocks may occur.

Advantages

- Ensures conflict serializability
- Allows higher concurrency than stricter variants

Disadvantages

- Cascading rollbacks may occur
- Recovery is complex
- Deadlocks are possible

Example:
A transaction acquires an exclusive lock on data item X, performs updates, releases the lock, and then commits. If another transaction has read X before the commit, cascading rollback may occur in case of failure.

2 .Strict Two-Phase Locking (Strict 2PL)

In Strict Two-Phase Locking, a transaction holds all exclusive (write) locks until it commits or aborts. Shared locks may be released earlier, but write locks are retained until the end of the transaction.

Characteristics:

- Guarantees conflict serializability.
- Produces strict schedules, thereby preventing cascading rollbacks.
- Simplifies recovery because uncommitted data is never read or overwritten.

Advantages

- Prevents cascading rollbacks
- Simplifies recovery
- Widely implemented in practical DBMSs

Disadvantages

- Reduced concurrency compared to Basic 2PL

Deadlocks are still possible

Example:
If a transaction updates a bank account balance, the exclusive lock on that account is held until commit. Other transactions cannot read or write that account until the transaction completes.

Important Note: Strict 2PL is the most commonly implemented form of two-phase locking in commercial DBMSs.

3. Rigorous Two-Phase Locking (Rigorous 2PL)

In Rigorous Two-Phase Locking, a transaction holds both shared and exclusive locks until it commits or aborts. No locks are released before transaction completion.

Characteristics:

- Guarantees conflict serializability.
- Ensures strict schedules.
- Commit order of transactions corresponds to their serialization order.
- Reduces concurrency compared to Strict 2PL.

Advantages

- Ensures strict schedules
- Commit order equals serialization order
- Simplest recovery mechanism

Disadvantages

- Lowest concurrency among 2PL variants
- Deadlocks may still occur
- Higher waiting time for transactions

Example:
A transaction that reads and writes multiple records holds all read and write locks until commit, ensuring that no other transaction can access those records during execution.

4. Conservative Two-Phase Locking (Static 2PL)

In Conservative Two-Phase Locking, a transaction acquires all the locks it needs before it begins execution. If any required lock cannot be obtained, the transaction waits and does not acquire any lock.

Characteristics:

- Guarantees conflict serializability.
- Prevents deadloc**ks** completely.
- Requires advance knowledge of all data items to be accessed.
- May reduce concurrency.

Advantages

- Completely eliminates deadlocks
- Prevents cascading rollbacks
- Guarantees conflict serializability

Disadvantages

- Requires advance knowledge of all required data items
- Significantly reduces concurrency
- Not suitable for dynamic transactions

Example:
A transaction that needs to read X and write Y requests locks on both X and Y before starting. If either lock is unavailable, the transaction waits without holding any lock.

5.9 TIMESTAMP-BASED CONCURRENCY CONTROL PROTOCOL

Timestamp-based concurrency control protocols ensure serializability by ordering transactions according to timestamps rather than using locks. Each transaction is assigned a unique timestamp when it starts, and this timestamp defines the logical execution order of transactions.
All conflicting operations must occur in timestamp order. If an operation violates this order, the transaction is rolled back and restarted with a new timestamp.

A key characteristic of timestamp-based protocols is that they are non-blocking.

Transactions never wait for locks; instead, incorrect ordering is handled through transaction aborts.

Timestamp Assignment

When a transaction T starts, it is assigned a timestamp TS(T).

• Smaller timestamp → older transaction

• Larger timestamp → younger transaction

For each data item X, the system maintains:

• Read_TS(X) – largest timestamp of any transaction that has successfully read X
• Write_TS(X) – largest timestamp of any transaction that has successfully written X

These values are used to validate read and write operations.

5.9.1. Basic Timestamp Ordering Protocol (TO)

The Basic Timestamp Ordering Protocol enforces serializability by ensuring that all read and write operations follow timestamp order.

Rules of the Protocol

Let transaction T with timestamp TS(T) access data item X.

Read Rule

Transaction T can read X if:

TS(T) ≥ Write_TS(X)

If TS(T) < Write_TS(X), the read is rejected and T is rolled back, because T is trying to read a value written by a younger transaction.

If the read is allowed:

Read_TS(X) = max(Read_TS(X), TS(T))

Write Rule

Transaction T can write X if:

TS(T) ≥ Read_TS(X)

AND
TS(T) ≥ Write_TS(X)

If either condition fails, the write is rejected and T is rolled back.

If allowed:

Write_TS(X) = TS(T)

Example: Basic Timestamp Ordering

Assume:

TS(T1) = 10

TS(T2) = 20

Initial values:

Read_TS(X) = 0

Write_TS(X) = 0

Schedule

Time	Transaction T1	Transaction T2
1	Write(X)	
2		Read(X)
3	Commit	
4		Commit

Step-by-Step Analysis

• T1 writes X → allowed (10 ≥ 0)

Write_TS(X) = 10

• T2 reads X → allowed (20 ≥ 10)

Read_TS(X) = 20

Conclusion

The schedule follows timestamp order T1 → T2.

✓ Conflict serializable

✓ No locks used

✓ No waiting, only ordering

5.9.2. Thomas' Write Rule (Modified Timestamp Ordering)

The Thomas' Write Rule is an optimization of the basic timestamp ordering protocol. It reduces unnecessary rollbacks by ignoring obsolete write operations.

Rule Description

When transaction T attempts to write data item X:

• If TS(T) < Write_TS(X)

→ The write is ignored (not rolled back)

• If TS(T) ≥ Write_TS(X)

→ The write is performed normally

This rule applies only to write operations.

Example: Thomas' Write Rule

Assume:

TS(T1) = 10

TS(T2) = 20

Schedule

Time	Transaction T1	Transaction T2
1		Write(X)
2	Write(X)	
3	Commit	
4		Commit

Analysis

• T2 writes X first → Write_TS(X) = 20

• T1 later attempts Write(X)

• TS(T1) < Write_TS(X)

Under basic TO → T1 would be rolled back

Under Thomas' Write Rule → T1's write is ignored

Conclusion

✓ Serializability preserved

✓ Unnecessary rollback avoided

✓ Higher throughput than basic TO

5.9.3. Multi-Version Timestamp Ordering (MVTO)

Multi-Version Timestamp Ordering maintains multiple versions of each data item, each tagged with the timestamp of the transaction that created it.

Key Concepts

- Every write creates a new version of the data item
- A read operation selects the version with the largest timestamp ≤ TS(T)
- Read and write conflicts are greatly reduced

Example: Multi-Version Timestamp Ordering

Assume:

TS(T1) = 10

TS(T2) = 20

Versions of X:

X_0 – initial value (TS = 0)

X_1 – written by T1 (TS = 10)

Schedule

Time	Transaction T1	Transaction T2
1	Write(X_1)	
2		Read(X_1)
3	Commit	
4		Commit

Analysis

• T2 reads X_1 since 10 ≤ 20

• No rollback required

• High concurrency achieved

Conclusion

✓ Highest concurrency

✓ No read-write conflicts

✗ Increased storage overhead

5.9.4 Advantages of Timestamp-Based Protocols

• Guarantees conflict serializability

• Deadlock-free (no waiting)

• Suitable for real-time systems

• Simple conceptual ordering model

5.9.5 Disadvantages of Timestamp-Based Protocols

• Frequent rollbacks under high contention

• Possible starvation of older transactions

• Overhead of timestamp management

• Multi-version methods require extra storage

5.10 OPTIMISTIC CONCURRENCY CONTROL (OCC)

Optimistic Concurrency Control (OCC) is a concurrency control technique based on the assumption that conflicts among transactions are rare. Unlike lock-based protocols, OCC does not use locks during transaction execution. Instead, transactions execute freely and are checked for conflicts only at commit time.The central idea of OCC is to allow maximum concurrency and detect conflicts after they occur, rather than preventing them in advance. If a conflict is detected during validation, the transaction is rolled back and restarted.

OCC is particularly suitable for:

• Read-mostly workloads

• Distributed and web-based applications

• Systems with short transactions and low contention

5.10.1 Phases of Optimistic Concurrency Control

Each transaction in OCC proceeds through three well-defined phases:

1. Read Phase
2. Validation Phase
3. Write Phase

These phases ensure correctness while avoiding locking overhead.

1.Read Phase

During the read phase, a transaction:

• Reads data items from the database

• Performs computations

• Stores all updates in a private local workspace

No changes are written to the database during this phase, and no locks are acquired.

Important Property:

All writes are deferred until the transaction successfully completes validation.

2. Validation Phase

The **validation phase** determines whether the transaction can safely commit. The system checks whether the transaction conflicts with other transactions that have committed or are currently validating.

If a conflict is detected, the transaction is **aborted and restarted**.

The validation ensures that the concurrent execution is equivalent to some serial execution.

3. Write Phase

If validation succeeds:

- All updates from the local workspace are copied to the database
- The transaction commits

If validation fails:

• The transaction is rolled back

• All local updates are discarded

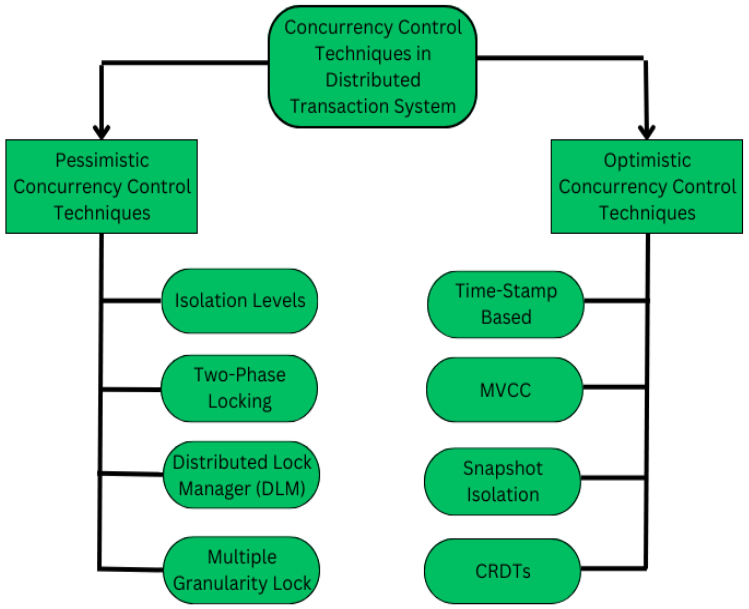


Figure 5.14: Phases of Optimistic Concurrency Control

Figure description: A flow diagram showing the progression of a transaction through the Read Phase, Validation Phase, and Write Phase, with rollback occurring if validation fails.

5.10.2 Validation Techniques in OCC

Validation is the most critical component of OCC. Transactions are validated using **timestamp ordering and conflict checks**.

Let:
• T_1 be the transaction being validated

• T_2 be a transaction that committed earlier

Three conditions are commonly used to validate T_1.

5.10.3. Validation Conditions

Transaction $\mathbf{T_1}$ can commit if **one of the following conditions holds** for every

transaction T_2 that committed before T_1:

1. **T_2 completes before T_1 begins**

   → No overlap, hence no conflict

2. **T_2 completes before T_1 enters write phase, and**

   → Write_Set(T_2) ∩ Read_Set(T_1) = ∅

3. **Write_Set(T_2) ∩ Read_Set(T_1) = ∅ andWrite_Set(T_2) ∩ Write_Set(T_1) = ∅**

If none of these conditions is satisfied, T_1 must be rolled back.

5.10.4 Example: OCC with Transaction Schedule

Consider two transactions:

Transaction T1: Read(X), Write(X)

Transaction T2: Read(X), Write(X)

Schedule under OCC

Time	T1	T2
1	Read(X)	
2		Read(X)
3	Write(X) (local)	
4		Write(X) (local)
5	Validation	
6	Commit	
7		Validation
8		Abort

Analysis

• Both transactions read X concurrently

• T1 enters validation first and commits

• T2 detects conflict during validation (write–write conflict)

• T2 is rolled back

Conclusion

✓ Serializability preserved

✓ No locks used

✗ Transaction T2 is rolled back

5.10.5. Advantages of Optimistic Concurrency Control

• No lock overhead

• Deadlock-free

• High concurrency for read-dominant workloads

• Suitable for distributed systems• Simple transaction execution logic

5.10.6. Disadvantages of Optimistic Concurrency Control

• High rollback cost under heavy contention

• Starvation possible for long transactions

• Validation overhead at commit time• Not suitable for write-intensive workloads

5.11 DEADLOCKS

In database systems that employ **lock-based concurrency control**, transactions frequently compete for shared data items. When transactions acquire locks incrementally and hold them for a period of time, situations may arise in which **no transaction can make progress**. Such a situation is known as a **deadlock**.

A **deadlock** occurs when a group of transactions is indefinitely waiting for one another to release locks, and none of them can proceed. From the system's perspective, deadlocks represent a serious problem because they:

- Reduce system throughput
- Waste CPU and memory resources
- Require explicit detection and recovery mechanisms

Deadlocks do not resolve automatically and must be handled by the DBMS.

5.11.1 Necessary Conditions for Deadlock (Coffman Conditions)

A deadlock can occur if and only if all four of the following conditions hold simultaneously. These are known as the Coffman conditions.

(a) Mutual Exclusion

At least one resource must be held in a non-shareable mode. In DBMSs, this corresponds to exclusive (X) locks, where only one transaction can access the data item at a time.

(b) Hold and Wait

A transaction that is holding one or more resources is waiting to acquire additional resources that are currently held by other transactions.

(c) No Preemption

Resources cannot be forcibly taken away from a transaction. A transaction releases resources only voluntarily, typically at commit or abort.

(d) Circular Wait

A circular chain of transactions exists such that:

- Transaction T1 waits for a resource held by T2
- Transaction T2 waits for a resource held by T3
- Transaction Tn waits for a resource held by T1

5.11.2 Detailed Deadlock Example with Schedule

Consider two transactions executing concurrently:

Transaction T1: needs data items X and Y

Transaction T2: needs data items Y and X

Schedule Leading to Deadlock

Time	Transaction T1	Transaction T2
1	Lock-X(X)	
2		Lock-X(Y)
3	Request Lock-X(Y)	
4		Request Lock-X(X)

Explanation:

- T1 holds lock on X and waits for Y
- T2 holds lock on Y and waits for X
- Neither transaction can proceed

This results in a deadlock.

5.11.3 Wait-For Graph Representation

A **wait-for graph** is a simplified directed graph used by the DBMS to detect deadlocks.

Definition:
A wait-for graph G = (V, E) where:

- Each node represents a transaction
- An edge Ti → Tj indicates that transaction Ti is waiting for a lock held by Tj

Deadlock Detection Rule: If the wait-for graph contains a **cycle**, then a deadlock exists.

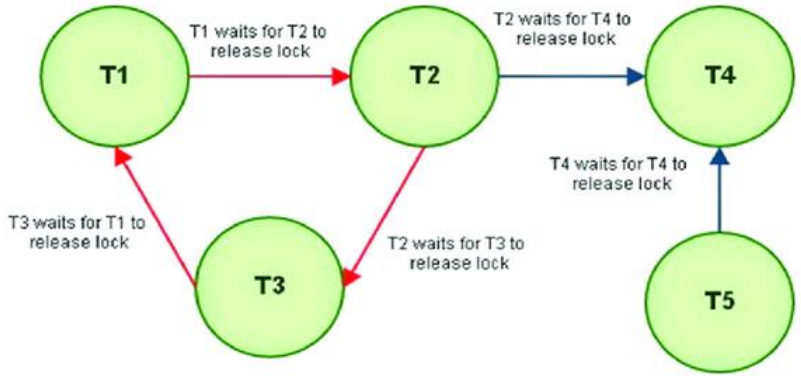


Figure 5.11: Wait-For Graph Showing a Deadlock Cycle

Figure description: The figure depicts two or more transactions forming a cycle, indicating circular wait and hence deadlock.

5.11.4 Deadlock Handling Strategies

DBMSs handle deadlocks using one of the following strategies.

(a) Deadlock Prevention

Deadlock prevention ensures that at least one Coffman condition never holds.

Common schemes include:

- **Wait–Die Scheme**: Older transaction waits; younger transaction aborts.
- **Wound–Wait Scheme**: Older transaction aborts younger transaction.

These schemes use transaction timestamps to decide which transaction waits and which aborts.

(b) Deadlock Avoidance

Deadlock avoidance ensures the system never enters an unsafe state.

Example:

- **Banker's Algorithm**, which requires prior knowledge of resource requirements.

Avoidance techniques are rarely used in DBMSs due to high overhead.

(c) Deadlock Detection and Recovery

Deadlocks are allowed to occur, detected periodically, and resolved.

Detection:

- Construct wait-for graph
- Search for cycles

Recovery:

- Abort one or more transactions
- Choose victim based on cost, priority, or number of locks held

5.12 FAILURE CLASSIFICATION

A failure is any event that prevents a transaction or the database system from completing its intended operations correctly. DBMSs must be designed to tolerate failures while preserving:

- **Atomicity** (all-or-nothing execution)
- **Consistency**
- **Durability**

Failures are categorized based on their scope and impact.

5.12.2 Transaction Failures

A **transaction failure** affects only a single transaction.

Causes:

- Logical errors (invalid input, arithmetic exception)

- Integrity constraint violations
- Explicit abort by the user or system

Effect:

- Partial effects of the transaction must be undone
- Other transactions continue unaffected

5.12.3 System Failures

A **system failure** occurs due to hardware or software crashes.

Examples:

- Power failure
- Operating system crash
- CPU failure

Characteristics:

- Contents of main memory are lost
- Disk contents remain intact

Recovery Requirement:

- Undo all incomplete transactions
- Redo committed transactions using logs

5.12.4 Media Failures

A **media failure** occurs due to damage to secondary storage.

Examples:

- Disk head crash
- Corrupted disk sectors

Characteristics:

- Permanent loss of stored data

Recovery Requirement:

- Restore database from backup

- Apply log records to reconstruct recent changes

5.12.5 Failure Classification Summary

Failure Type	Scope	Data Loss	Recovery Action
Transaction Failure	Single transaction	No	Rollback
System Failure	Entire system	No (disk intact)	Undo/Redo
Media Failure	Disk storage	Yes	Backup + Redo

5.13 STORAGE

Storage management is a fundamental responsibility of a DBMS. It determines:

- How data is stored and retrieved
- How failures are handled
- How durability is guaranteed

Efficient storage design directly affects performance and reliability.

5.13.1 Storage Hierarchy in DBMS

Database systems use a multi-level storage hierarchy:

1. **Primary Storage (Main Memory)**
    - Volatile
    - Fastest access
    - Stores buffer cache and active transactions
2. **Secondary Storage (Disk Storage)**
    - Non-volatile
    - Stores database files and log files
3. **Tertiary Storage (Backup Storage)**
    - Magnetic tape or cloud storage

- Used for backup and archival

5.13.2 Stable Storage

Stable storage is an abstraction that guarantees data persistence even in the presence of failures.

Techniques:

- Disk mirroring
- RAID architectures
- Replicated storage systems

5.13.3 Logging and Log Storage

A **log** is a sequential record of all database updates.

Each log entry typically contains:

- Transaction identifier
- Data item identifier
- Old value (before image)
- New value (after image)

Logs are essential for recovery.

5.13.4 Write-Ahead Logging (WAL)

The **Write-Ahead Logging protocol** ensures correctness during recovery.

Rules:

1. Before modifying a data item, the corresponding log record must be written to stable storage.

2. A transaction's commit record must be written before the transaction is considered committed.

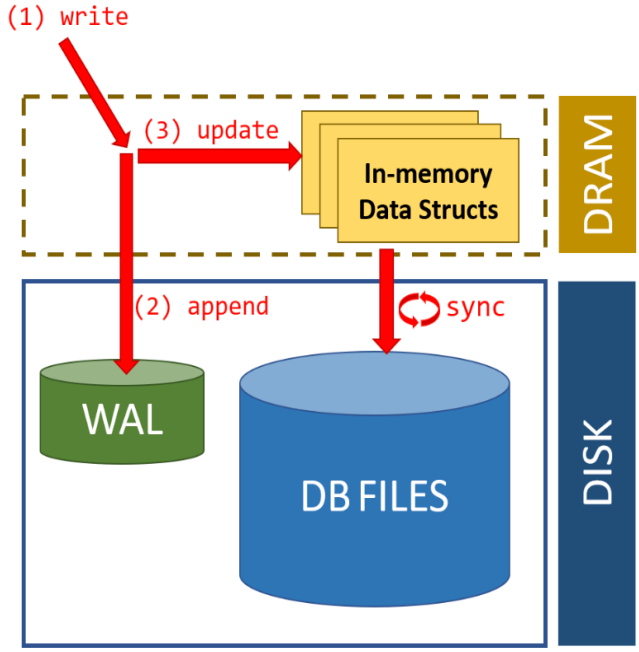


Figure 1: Write path for a typical transactional DB (e.g., MS SQL).

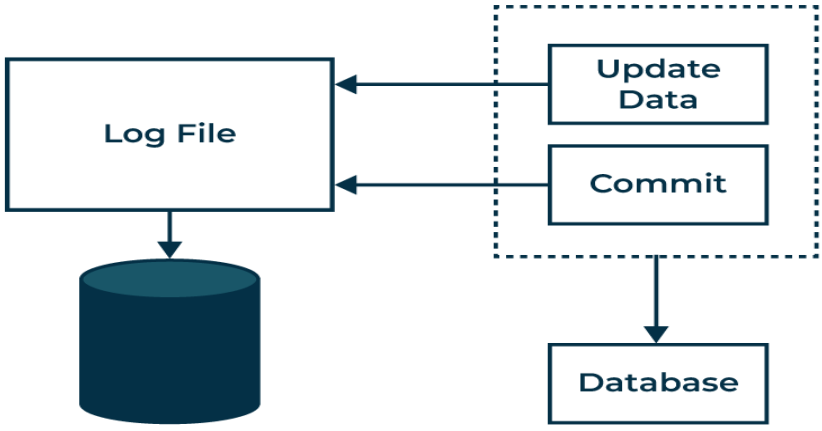


Figure 5.12: Write-Ahead Logging Mechanism

5.13.5 Checkpoints and Their Importance

A checkpoint is a point where:

- All modified buffers are flushed to disk
- A checkpoint record is written to the log

Benefits:

- Limits the amount of log to be scanned during recovery
- Reduces system restart time

5.14 RECOVERY ALGORITHMS

Recovery algorithms use log records and checkpoints to restore the database after failures. They ensure that the atomicity and durability properties of transactions are preserved.

5.14.1 Deferred Update Recovery Algorithm

In the Deferred Update approach:

- Database updates are not applied immediately
- Updates are recorded only in the log
- Actual database modifications occur after commit

Characteristics:

- No undo is required
- Redo is required for committed transactions

Example: Deferred Update

Transaction T1:

- Write(X = 50)
- Commit

If a system crash occurs before commit, no changes need to be undone because updates were never applied to the database.

Advantages:

- Simple recovery

- No cascading rollback

Disadvantages:

- Delayed visibility of updates

5.14.2 Immediate Update Recovery Algorithm

In the Immediate Update approach:

- Database updates are applied before commit
- Log records are written first (WAL rule)

Characteristics:

- Both undo and redo may be required
- Used in most practical DBMSs

Example: Immediate Update

Transaction T1:

- Write(X: 10 → 20)
- System crash before commit

During recovery:

- Undo the write using the old value from the log

Advantages:

- High concurrency
- Immediate visibility of updates

Disadvantages:

- More complex recovery

5.14.3 Log-Based Recovery with UNDO and REDO

In log-based recovery:

- UNDO is applied to all incomplete transactions
- REDO is applied to all committed transactions

Recovery Steps:

1. Scan the log backward to undo uncommitted transactions
2. Scan the log forward to redo committed transactions

5.15.4 Shadow Paging Recovery Algorithm

Shadow Paging avoids logging by maintaining two page tables:

- Shadow page table (stable)
- Current page table (updated)

On commit:

- The current page table becomes the new shadow page table

Advantages:

- No logging required
- Simple recovery

Disadvantages:

- Poor concurrency
- High storage overhead

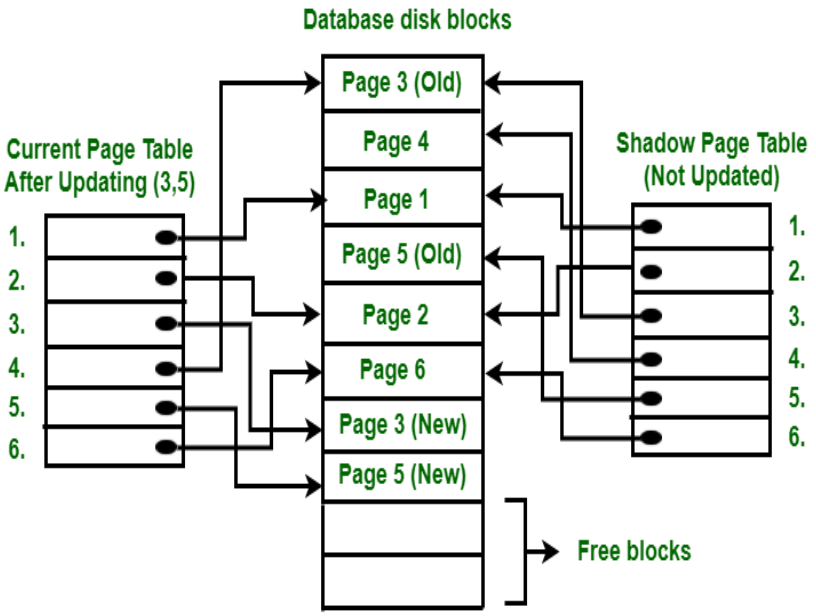


Figure 5.15: Shadow Paging Mechanism

5.14.5 Checkpoint-Based Recovery

When checkpoints are used:

- Recovery starts from the most recent checkpoint
- Reduces log scanning overhead
- Improves system restart time

5.16 INTRODUCTION TO INDEXING TECHNIQUES

5.16.1 Need for Indexing in Databases

In large databases, searching for records by scanning the entire relation (file scan) is inefficient and time-consuming. **Indexing** is a technique used to **speed up data retrieval operations** by providing an auxiliary access structure that allows the DBMS to locate records quickly.

An index functions similarly to an index in a book: instead of scanning every page, the reader directly accesses the required page.

5.16.2 Definition of Index

An index is a data structure that stores pairs of the form:

(Index key value, Pointer to record)

The index key is derived from one or more attributes of the relation, and the pointer identifies the physical location of the record on disk.

5.16.3 Advantages of Indexing

- Significantly reduces disk I/O operations
- Improves query performance for selection and join operations
- Enables efficient execution of range queries and equality searches

5.16.4 Types of Indexing Techniques

Indexing techniques are broadly classified into:

1. **Tree-based indexing** (e.g., B+ Trees)
2. **Hash-based indexing**

5.17 B+ TREE INDEXING

5.17.1 Introduction to B+ Trees

A B+ Tree is a balanced multi-level tree data structure widely used in DBMSs for indexing. It is a variant of the B-Tree, optimized for disk-based storage systems.

Key Feature: All actual data records (or pointers to records) are stored only at the leaf level, while internal nodes store only index keys and child pointers.

5.17.2 Properties of B+ Trees

Let m be the order of a B+ Tree.

- Each internal node can have at most m children
- Each internal node (except root) has at least ⌈m/2⌉ children
- Root has at least two children (unless it is a leaf)
- All leaf nodes appear at the same level
- Leaf nodes are linked sequentially (linked list)

5.17.3 Structure of a B+ Tree

- **Internal Nodes** Contain index keys and pointers to child nodes.
- **Leaf Nodes** Contain index keys and pointers to actual data records.
- **Sequential Access** Leaf nodes are connected using pointers for efficient range queries.

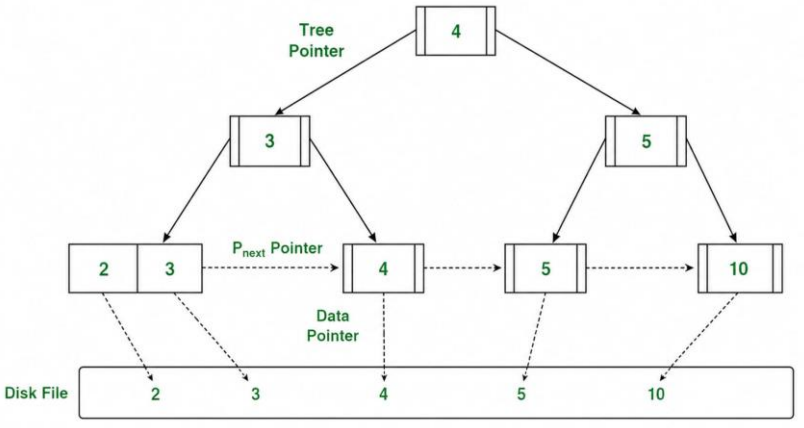


Figure 5.18: Structure of a B+ Tree

Figure description: The figure shows a multi-level B+ Tree with index keys in

internal nodes and data pointers in leaf nodes connected sequentially.

5.18 OPERATIONS ON B+ TREES

5.18.1 Search Operation in B+ Tree

Algorithm:

1. Start at the root node.
2. Compare the search key with index keys.
3. Follow the appropriate child pointer.
4. Repeat until a leaf node is reached.
5. Search the key in the leaf node.

Time Complexity: O ($\log_m$ n), where m is the order of the tree.

Example: Search Operation

Assume a B+ Tree indexing student IDs.

To search for StudentID = 105:

- Compare 105 with root keys
- Traverse to the correct child
- Reach the leaf node containing 105

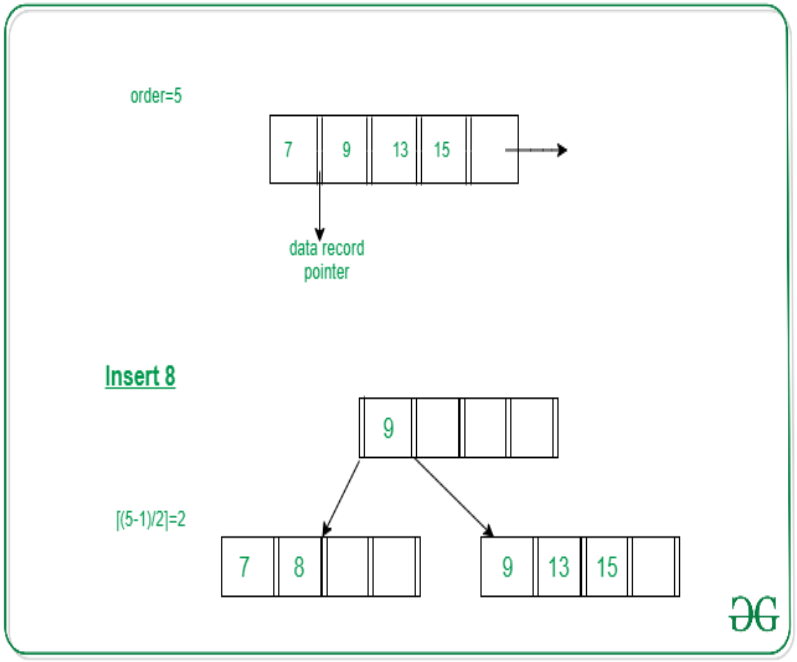
5.18.2 Insertion Operation in B+ Tree

Steps:

1. Locate the appropriate leaf node.
2. Insert the key in sorted order.
3. If the leaf node overflows:
    - Split the node
    - Promote the middle key to the parent
4. Repeat splitting up the tree if necessary.

Example: Insertion Operation

Assume order m = 4 and insert key 45.



- Leaf node becomes full after insertion
- Node is split into two nodes

- Middle key is propagated upward

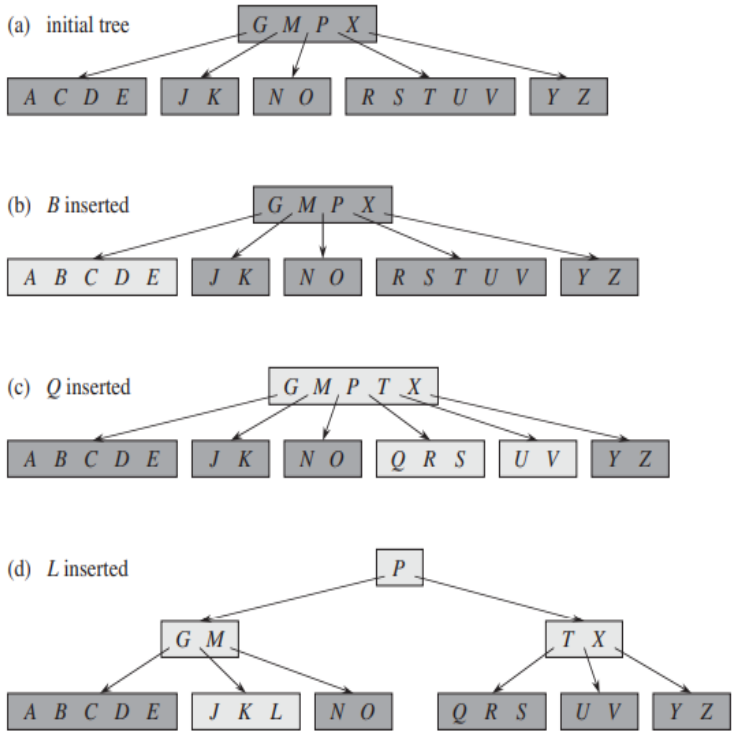


Figure 5.19: B+ Tree Insertion and Node Splitting

5.18.3 Deletion Operation in B+ Tree

Steps:

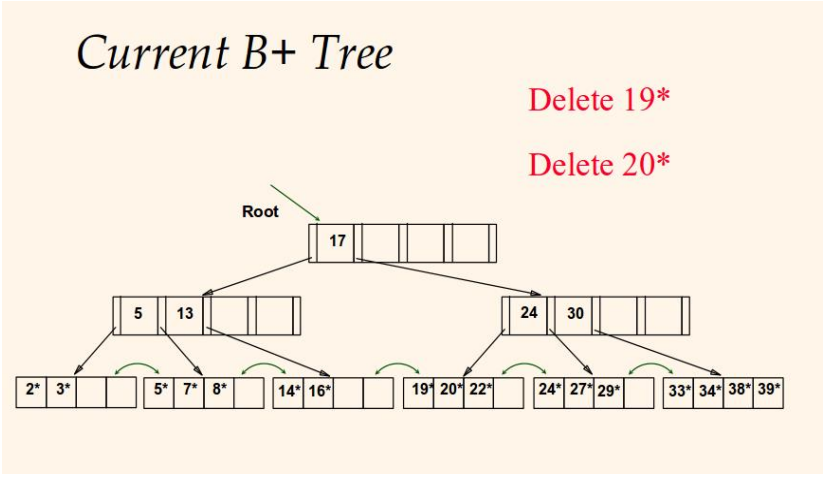
1. Locate the leaf node containing the key.
2. Remove the key.
3. If underflow occurs:
    - Borrow a key from sibling (redistribution), or
    - Merge with sibling
4. Update parent keys accordingly.

Example: Deletion Operation

If deletion causes a leaf node to contain fewer keys than required:

- Redistribute keys from a sibling node, or

- Merge nodes and adjust parent pointers.



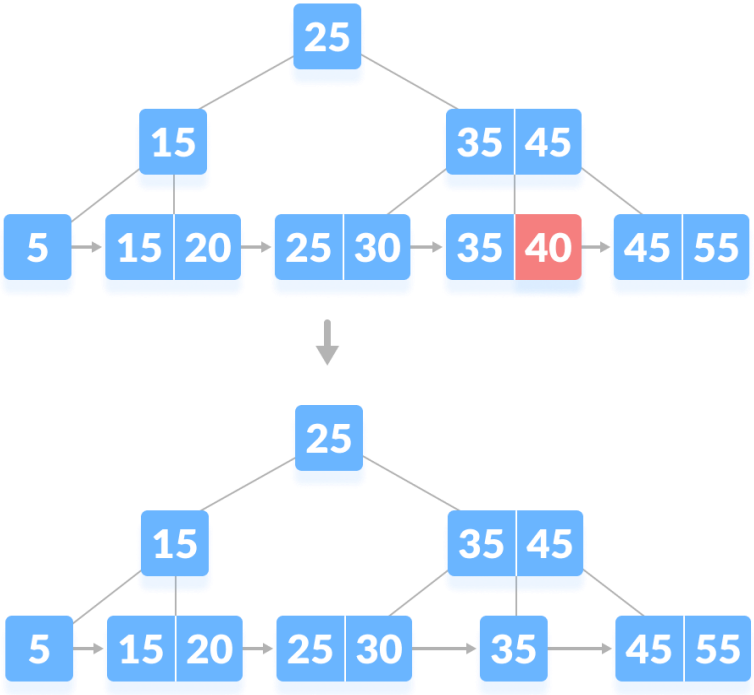


Figure 5.20: B+ Tree Deletion with Redistribution and Merge

5.18.4 Advantages of B+ Trees

- Always balanced → predictable performance
- Efficient for both equality and range queries

- Minimizes disk I/O due to high fan-out

5.18.5 Disadvantages of B+ Trees

- Slightly higher overhead for insertions and deletions
- Requires additional storage for pointers

5.19 HASH-BASED INDEXING

5.19.1 Introduction to Hash Indexing

Hash-based indexing uses a hash function to map search key values directly to storage locations called buckets.

Hashing is extremely efficient for equality search operations.

5.19.2 Hash Function

A hash function h(k) maps a key value k to a bucket number.

Example:
h(k) = k mod 10

5.19.3 Static Hashing

In static hashing, the number of buckets is fixed.

Problem:
When data grows, buckets may overflow, causing performance degradation.

Example: Static Hashing

Assume:

- h(k) = k mod 5
- Insert keys: 12, 22, 32

All keys map to the same bucket, causing overflow.

5.19.4 Dynamic Hashing

Dynamic hashing allows the hash structure to grow or shrink.

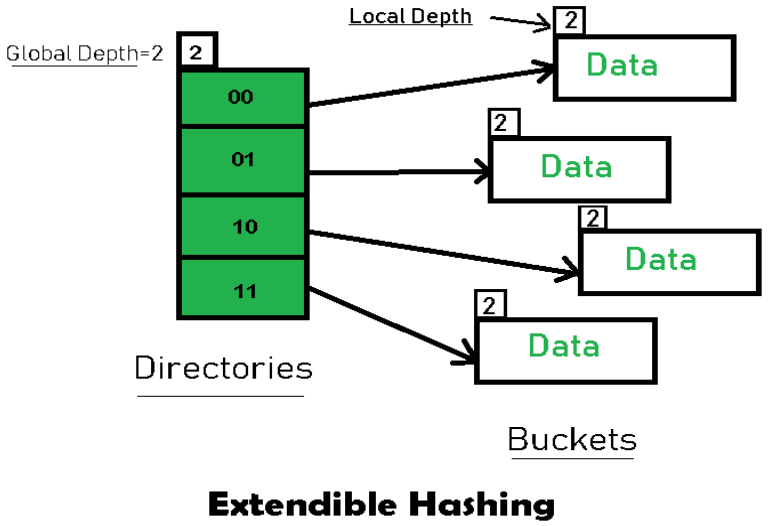
Common techniques:

- Extendible hashing
- Linear hashing

5.19.5 Extendible Hashing

Uses a directory of pointers indexed by hash values.

- Directory grows dynamically
- Buckets split when overflow occurs



Extendible Hashing

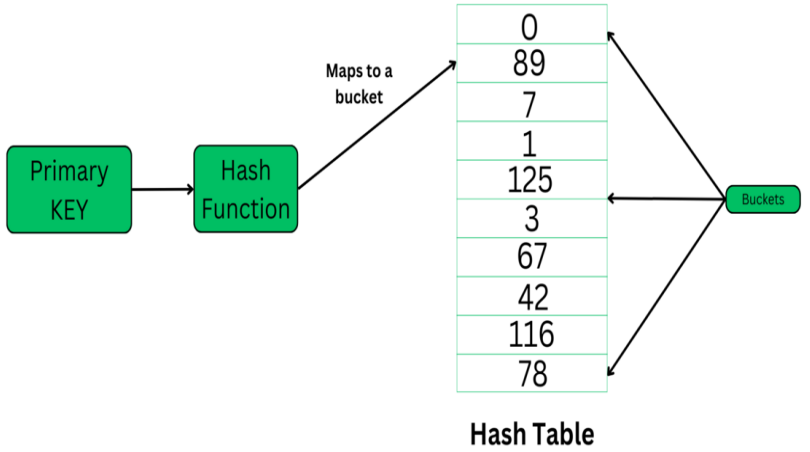


Figure 5.21: Extendible Hashing Structure

5.19.6 Linear Hashing

- Buckets split gradually
- No directory required

- More space-efficient than extendible hashing

5.20.7 Comparison: B+ Tree vs Hash Indexing

Feature	B+ Tree	Hash Index
Query Type	Range + Equality	Equality only
Structure	Balanced tree	Hash table
Disk I/O	Logarithmic	Constant (average)
Use Case	Ordered data	Exact match queries

Chapter Summary

5.1 Transaction Concepts

A transaction is a logical unit of work that accesses and possibly modifies the database. Transaction States:

- Active: Transaction is executing read/write operations.
- Partially Committed: Last statement executed; changes not yet permanently recorded.
- Committed: All updates permanently recorded on stable storage.
- Failed: Error occurred; cannot proceed.
- Aborted: Rolled back; database restored to pre-transaction state.

5.2 ACID Properties

- Atomicity: All-or-nothing execution. No partial transactions. Managed by recovery manager.
- Consistency: Transaction takes database from one consistent state to another. Integrity constraints preserved.
- Isolation: Intermediate states invisible to other transactions. Prevents dirty reads, lost updates, inconsistent retrievals.
- Durability: Committed changes are permanent even after system failures. Managed through logging and stable storage.

5.3 Concurrent Execution

Multiple transactions execute simultaneously, interleaving operations. Benefits: improved throughput, reduced response time, better resource utilization. Risks: lost updates, dirty reads, non-repeatable reads, phantom reads. Concurrency control protocols ensure correctness.

- Serial Schedule: Transactions execute one after another; no interleaving. Always correct but poor performance.
- Non-Serial Schedule: Operations interleaved; better performance but may cause anomalies without proper control.

5.4 Serializability

- Conflict Serializability: A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations. Two operations conflict if: different transactions, same data item, at least one is a write. Tested using a Precedence Graph - if no cycle, schedule is conflict serializable.
- View Serializability: More general. A schedule is view serializable if

view-equivalent to a serial schedule. Conditions: same initial reads, same read-from relationships, same final writes. Every conflict-serializable schedule is view-serializable, but not vice versa (blind writes may be view serializable but not conflict serializable).

5.5 Recoverability

- Recoverable Schedule: A transaction commits only after all transactions whose data it read have committed. Prevents dirty reads from causing irrecoverable situations.
- Cascadeless Schedule: Transactions read only committed data. Prevents cascading rollbacks. Stricter than recoverable.
- Strict Schedule: No transaction reads or writes a data item until the last writer has committed or aborted. Simplifies recovery; strongest constraint.

5.6 Isolation Levels (SQL Standard)

- Read Uncommitted: Allows dirty reads; maximum concurrency, minimal correctness.
- Read Committed: Prevents dirty reads; non-repeatable reads still possible.
- Repeatable Read: Prevents non-repeatable reads; phantom reads possible.
- Serializable: Full isolation; prevents all anomalies. Equivalent to serial execution.

5.7 Lock-Based Protocols

- Shared Lock (S-lock): Multiple transactions can read simultaneously. Incompatible with X-lock.
- Exclusive Lock (X-lock): Only one transaction; read and write. Incompatible with S and X locks.
- Two-Phase Locking (2PL): Growing phase (acquire only) + Shrinking phase (release only). Guarantees conflict serializability.
- Basic 2PL: Guarantees serializability but allows cascading rollbacks and deadlocks.
- Strict 2PL: Holds exclusive locks until commit/abort. Prevents cascading rollbacks. Most commonly used in practice.
- Rigorous 2PL: Holds all locks until commit/abort. Commit order = serialization order.
- Conservative 2PL: Acquires all needed locks before execution. Prevents

deadlocks. Requires advance knowledge of data accessed.

5.8 Timestamp-Based Protocols

Assigns each transaction a unique timestamp at start. Operations must follow timestamp order.

- Basic Timestamp Ordering: Read/write rules based on Read_TS(X) and Write_TS(X). Violations cause rollback and restart.
- Thomas' Write Rule: Optimization ignoring obsolete writes (TS(T) < Write_TS(X)) instead of rolling back. Increases throughput.
- Multi-Version Timestamp Ordering (MVTO): Maintains multiple versions of each data item. Reads select appropriate version; no read-write conflicts. Higher concurrency at cost of extra storage.

5.9 Optimistic Concurrency Control (OCC)

Assumes conflicts are rare. No locks during execution. Three phases:

- Read Phase: Transaction reads from database and stores updates locally.
- Validation Phase: Checks for conflicts with other committed transactions. If conflict detected, abort and restart.
- Write Phase: If validated, local updates committed to database.

Advantages: No lock overhead, deadlock-free, high concurrency for read-dominant workloads. Disadvantages: High rollback cost under contention, not suitable for write-intensive workloads.

5.10 Deadlocks

A group of transactions indefinitely waiting for each other to release locks. Four Coffman conditions (all must hold): Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

- Detection: Wait-for graph; deadlock exists if there is a cycle.
- Prevention: Wait-Die scheme (older waits, younger aborts) or Wound-Wait scheme (older aborts younger).
- Avoidance: Banker's algorithm; rarely used due to overhead.
- Detection and Recovery: Allow deadlocks, detect periodically, abort victim transaction.

5.11 Failure Classification and Recovery

- Transaction Failure: Affects single transaction (logical error, constraint violation, explicit abort). Recovery: rollback.
- System Failure: Hardware/OS crash; main memory lost, disk intact. Recovery: undo incomplete, redo committed transactions.

- Media Failure: Disk damage; permanent data loss. Recovery: restore from backup and apply log records.

Storage: Primary (volatile RAM), Secondary (non-volatile disk), Tertiary (backup tape/cloud). Stable Storage: Disk mirroring, RAID for persistence.

- Write-Ahead Logging (WAL): Log record must be written to stable storage before modifying data. Commit record written before transaction considered committed.
- Checkpoints: Flush buffers to disk; write checkpoint record. Limits log scanning needed on recovery.
- Deferred Update: Updates applied to database only after commit. Only REDO needed.
- Immediate Update: Updates applied before commit (with logging). Both UNDO and REDO may be needed.
- Shadow Paging: Maintains shadow and current page tables. No logging needed; poor concurrency.

5.12 Indexing: B+ Trees and Hashing

- B+ Tree: Balanced multi-level index. All data at leaf level; leaves linked sequentially. Efficient for range and equality queries. Search: O(log m n). Operations: Insert (split on overflow), Delete (merge/redistribute on underflow).
- Static Hashing: Fixed buckets using hash function h(k). Overflow problem when data grows.
- Extendible Hashing: Directory of pointers grows dynamically; buckets split on overflow.
- Linear Hashing: Buckets split gradually without directory. Space-efficient.
- B+ Tree vs Hash Index: B+ Tree supports range + equality queries; hash index for equality only. B+ Tree: logarithmic I/O; hash: constant (average).

Review Questions

Section A: Conceptual Questions

1. Describe the five transaction states (Active, Partially Committed, Committed, Failed, Aborted) and the transitions between them. In a banking fund transfer scenario, map each operation to the appropriate transaction state.
2. Explain the four ACID properties with concrete examples. For each property, describe what could go wrong in a database system if that property were not guaranteed. Which component of the DBMS is primarily responsible for enforcing each property?
3. Distinguish between conflict serializability and view serializability. Prove that every conflict-serializable schedule is view-serializable, but not vice versa. Give an example schedule that is view serializable but not conflict serializable.
4. Define recoverable, cascadeless, and strict schedules. What is cascading rollback and why is it undesirable? Order these schedule categories from most restrictive to least restrictive and explain the tradeoff with performance.
5. Explain the four SQL isolation levels (Read Uncommitted, Read Committed, Repeatable Read, Serializable) and the specific anomalies each prevents. Which level is safest? Which offers the best performance? How do DBMSs typically implement each level?
6. Describe the Two-Phase Locking (2PL) protocol and its four variants (Basic, Strict, Rigorous, Conservative). For each variant, explain what additional guarantee it provides beyond basic 2PL, and what tradeoff it makes.
7. Explain the Timestamp-Based Concurrency Control protocol. How does Thomas' Write Rule improve performance? How does Multi-Version Timestamp Ordering (MVTO) achieve higher concurrency?
8. Describe the three phases of Optimistic Concurrency Control (OCC). What are the three validation conditions? Under what workload characteristics is OCC preferable to lock-based protocols?
9. State the four Coffman conditions necessary for deadlock. For each condition, describe a deadlock prevention strategy that eliminates that condition. What are the tradeoffs of deadlock prevention vs. detection-and-recovery approaches?
10. Explain Write-Ahead Logging (WAL) and its two rules. How do checkpoints reduce recovery time? Compare Immediate Update (with

UNDO/REDO) vs. Deferred Update (REDO only) recovery algorithms in terms of correctness and performance.

Section B: Analytical Problems

11. Given the schedule: T1: R(X), W(X); T2: R(X), W(X); T3: R(X) with operations interleaved as: T1:R(X), T2:R(X), T1:W(X), T3:R(X), T2:W(X), T3 Commit, T1 Commit, T2 Commit. (a) Is this schedule conflict serializable? Build a precedence graph. (b) Is it recoverable? (c) Is it cascadeless?
12. For each schedule, determine if it is: (i) Conflict Serializable, (ii) View Serializable, (iii) Recoverable, (iv) Cascadeless: (a) T1:W(X), T2:W(X), T1:Commit, T2:Commit. (b) T1:R(X), T2:W(X), T2:Commit, T1:W(X), T1:Commit. Show all reasoning.
13. Two transactions T1 and T2 operating under Basic 2PL: T1 needs locks on A, B, C; T2 needs locks on B, C, D. T1 acquires A, B; T2 acquires C, D; T1 requests C (waits); T2 requests B (waits). (a) Is this a deadlock? Draw the wait-for graph. (b) Apply Wait-Die and Wound-Wait schemes with TS(T1)=10, TS(T2)=20.
14. Given transactions with timestamps TS(T1)=5, TS(T2)=10, TS(T3)=15, and data item X with Read_TS(X)=8, Write_TS(X)=12. Determine whether each of the following operations is permitted under Basic TO protocol and Thomas' Write Rule: (a) T1 reads X. (b) T2 reads X. (c) T1 writes X. (d) T3 writes X.
15. A database log shows the following at the time of a system crash: [T1, Begin], [T1, X, 10, 20], [T2, Begin], [T2, Y, 30, 40], [T1, Commit], [T3, Begin], [T3, Z, 50, 60], [CHECKPOINT: T2, T3 active]. Determine which transactions need UNDO and which need REDO. Show the recovery steps.

Section C: Applied and Design Questions

16. A bank's online transaction system processes thousands of fund transfers per minute. Design a concurrency control strategy specifying: (a) Which isolation level to use and why. (b) Which 2PL variant to use. (c) How to handle deadlocks. (d) The recovery strategy (WAL, checkpoints, UNDO/REDO). Discuss tradeoffs between consistency and throughput.
17. Compare lock-based protocols, timestamp-based protocols, and optimistic concurrency control for the following scenarios: (a) A stock

trading system with high write contention. (b) A read-heavy analytics dashboard. (c) A reservation system with occasional conflicts. Justify your recommendation for each.

18. Design an indexing strategy for a university database with 1 million students, 50,000 courses, and 10 million enrollment records. The most common queries are: (a) Find all courses for a given student (student_id). (b) Find enrollment count per course. (c) Find courses offered between two dates. Decide between B+ tree and hash indexes for each query. Justify.
19. A distributed e-commerce platform processes orders across multiple servers. Describe how the ACID properties are maintained in a distributed setting. What challenges arise compared to a centralized DBMS? Briefly describe the Two-Phase Commit (2PC) protocol and how it ensures atomicity in distributed transactions.
20. Design a recovery scheme for a hospital database that cannot afford data loss. Specify: (a) Logging strategy (WAL, log format). (b) Checkpoint frequency and type. (c) Recovery algorithm (which transactions to UNDO/REDO). (d) Backup strategy. (e) How media failure recovery differs from system failure recovery. Justify all decisions in terms of the hospital's requirements.

ABOUT THE BOOK

This textbook provides a comprehensive and a structured introduction to the fundamental concepts, and design principles, and implementation techniques of a Database Management Systems (DBMS). It is designed to guide the learners from foundational topics such as data models and a relational theory to advanced areas including SQL, schema refinement (normalization), and the transaction management.

ABOUT AUTHORS

Dr. Halavath Balaji

Dr. Halavath Balaji has developed an exceptional expertise in delivering of high-quality education, and guiding research, and mentoring students toward academic and professional success. As a Professor at Sreenidhi Institute of Science and Technology.

Jogu Saritha

Jogu Saritha is an accomplished academician currently serving as an Assistant Professor at Sreenidhi Institute of Science and Technology. She is also a dedicated Research Scholar at the National Institute of Technology, Jalandhar (NITJ).

Pallavi B

Pallavi.B is an experienced academician working as an Assistant Professor at Sreenidhi Institute of Science and Technology. She is also a research scholar at SR University, Warangal, where she is actively involved in the field of Machine Learning.

Lurnexa Publications

130–187, Ramulavari Gudi Centre, Gorantla,
Guntur 522034, Andhra Pradesh, India
lurnexapublication@Gmail.com
lurnexa.in

MRP. ₹750

ISBN 978-81-685077-5-3

9 788168 507753